Summary sheet

The IMMC intends to create a first-of-a-kind Global Sports League (GSL) that will last for 8-9 months and joins clubs from all continents of the globe. This requires us to create a model that ensures competitiveness, geographic diversity, and fan engagement.

To meet this challenge, we came up with a comprehensive, data-driven, and flexible framework. First, we determined the optimal sport by creating a **Sport Suitability Score (SSS)**, evaluating it on all candidates based on global fanbase, feasible play frequency, cross-continental competitiveness, and avoiding single-nation dominance. **Football (soccer)** emerged as the best option, with the highest SSS among men and women. Male football was chosen for creating the league as it provided the highest score of all tested sports for both genders. After that, team selection utilized a modified **ELO rating system**, initialized with a decade of historical match data from all national leagues and international tournaments. This allowed for the creation of a global ranking of all football clubs. We selected the top 20 teams, strictly enforcing geographic diversity (minimum two teams per continent) and inclusivity (maximum one team per country), ensuring a competitive and representative lineup. Then, for the 20-team league, we designed a 38-game **Double Round Robin (DRR)** schedule. We have calculated that for such type of timetable and for chosen teams there is nearly 90% probability of winner selection, so no further gameplays (and costs) will be required, while at the end of the season there is still quite high uncertainty who will be the champion resulting in intensification of viewers emotions. Schedule's generation employs a two-phase optimization: initially, a **Constraint Programming Solver with Satisfiability Techniques** (CP-SAT) quickly finds a feasible schedule adhering to all important constraints (one game per week, home/away streak limits, etc.). Subsequently, **Simulated Annealing (SA)** refines this schedule, minimizing both environmental and financial cost (which is proportional to the total distance traveled by the teams) and balances reducing total travel distance and ensuring fair travel distribution (lower standard deviation) among teams, while strictly maintaining feasibility. Then once a week schedule is created. We also create a second model that determines the **match time** during the day for each match, accounting for **jet lag**. The biological cycle of each team is represented as a **cosine function** and we assume that each team can shift their timezone by 4 hours. The times of the matches are obtained by taking the intersections of graphs of the two teams playing, for which the functions have a positive value and then adjusting that by the timezone in which the match will be played. Lastly 1 hour is substracted from the obtained time as it is assumed that a match lasts 2 hours with the break and the midpoint of the match passes through the most optimal time.

When the number of teams is expanded to 24 a DRR becomes too lengthy. Thus, we propose a **dynamic clustering model**. Teams are grouped into six clusters of four teams in each cluster using a **balanced K-means algorithm**, weighted by both geographical proximity (**reducing travel**) and team strength (based on league points, **ensuring competitive matchups**). The season is split into six rounds: teams play a DRR within their cluster (6 games), then are re-clustered based on accumulated points for the next round, yielding a competitive 36-game season within the required 8-9 month timespan. Then the function for match hours used for 20 teams is applied to obtain the match hours for 24 teams. A pre-calculated schedule for the whole season is also provided.

All the core tools — ELO for ranking, CP-SAT for feasibility, SA for optimization, and K-means for clustering — are inherently **highly flexible**. The entire framework can be easily generalized to other team sports (e.g., basketball, rugby) by adjusting parameters like scoring rules, game frequency, travel cost weights, clustering weight, and incorporating sport-specific constraints.

We strongly **recommend the IMMC adopt Football for the GSL** and implement our proposed scheduling models—Double Round Robin optimized via CP-SAT and SA for 20 teams, and the dynamic K-means clustering approach for 24 or more teams—to launch a successful, globally diverse, competitive, and logistically efficient league.

Letter to the Decision Makers

Deer Decision Makers of the International Multi-continental Matchmaking Committee (IMMC),

To address your challenge of developing a fair, effective, and sustainable scheduling system for the new Global Sports League (GSL), we have created a robust, data-driven framework, that is presented in this report in great detail. Our primary goal was to deliver a solution that ensures exciting competition, diverse global representation, and logistical efficiency for this ambitious project.

The first step we took was a rigorous analysis of potential team sports. We have evaluated multiple factors, such as fan base sizes, playability, and competitive balance across hundreds of teams from different potential team sports. Football (Soccer) emerged as the ideal choice, offering the greatest potential for fan engagement and worldwide appeal for this league due to its large worldwide fan base and lack of domination by a single team.

We identified the top 20 clubs using a careful ranking methodology based on ten years of match results from all significant national leagues and international tournaments. Crucially, our selection process guarantees the required geographic diversity – with at least two teams per continent and only one per nation – creating a truly global and highly competitive field from day one.

For the initial 20-team league, we propose a fair Double Round Robin format (38 games per team). We create a schedule that significantly reduces the overall travel distance, which aligns with the sustainable development goals of humankind. When choosing the hours at which the teams play, we select such dates as to ensure that both teams are equally fatigued. This means that if a team has to travel into a completely different timezone, they will not experience significant impact of the get lag. This directly addresses your goals for logistical feasibility, sustainability, and most of all equity.

For your financial security we have investigated total costs of such an endeavor. It has turned out that the total cost is proportional to the distance traveled by the teams. There are some fixed costs, like rent of the stadium or marketing, but they do not depend on the schedule form, so we haven't considered them. We also considered environmental impact the League creates and we drew a conclusion that not only does our algorithm reduce your cost, but also maximize sustainability.

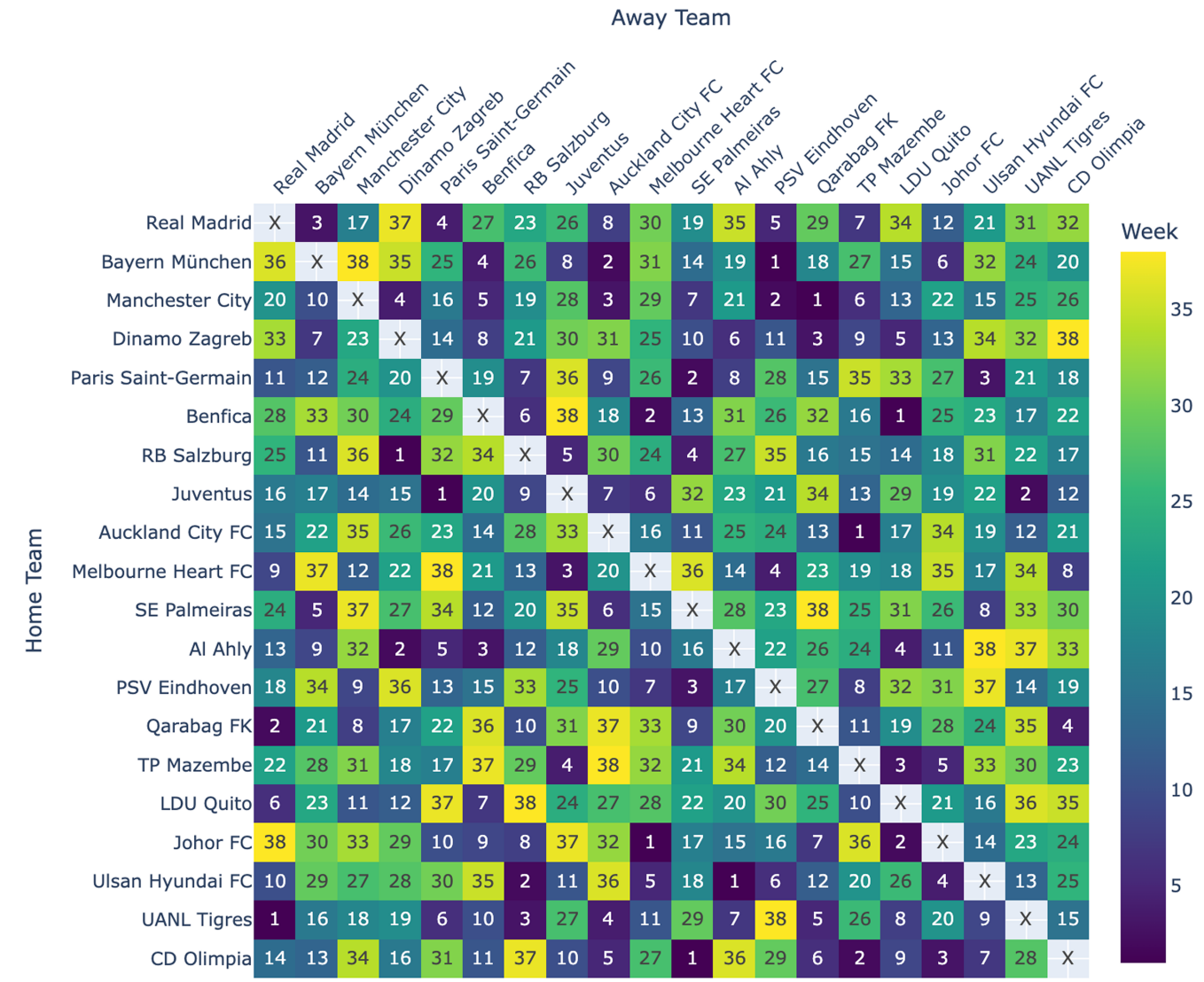
The solution we provided for you allowed us to create probabilistic model backed by simulations. Ultimately we can conclude that almost until the very end of the League there is still a significant chance for the leader loosing it's position what thrills up the audience until the really end providing lively interest and cash flow for the organizer during the entire time range of the league. However, you do not have to worry about unresolved or unfair results as there is 90% certainty of identifying the best team as the winner.

If you would like to extend the GSL to 24 teams, the full round-robin becomes too long for the 8-9 month season. Henceforth, we have designed a suitable alternative: a dynamic clustering system. Teams are periodically grouped into smaller clusters based on both geographic proximity (to cut down travel costs) and their current performance in the league (to ensure thrilling duels between teams). This maintains intense competition throughout a 36-week season that fits the maximum time span of the league while intelligently managing travel demands.

A key strength of our solution is its flexibility. The core methods used for ranking and scheduling can be adapted for other team sports and league formats in the future by simply adjusting settings like scoring rules or travel priorities, providing a durable model for the GSL. All we need is a few datasets and you can conquer other sport disciplines. Therefore, we strongly recommend launching the GSL with Football (Soccer) and adopting our proposed scheduling models: the optimized Double Round Robin for the 20-team roster and the dynamic clustering system for future expansion. We are confident this provides the best foundation for a thrilling, fair, and sustainable global league that captures the world's attention.

Sincerely,
Team #2025003

Schedule Visualization



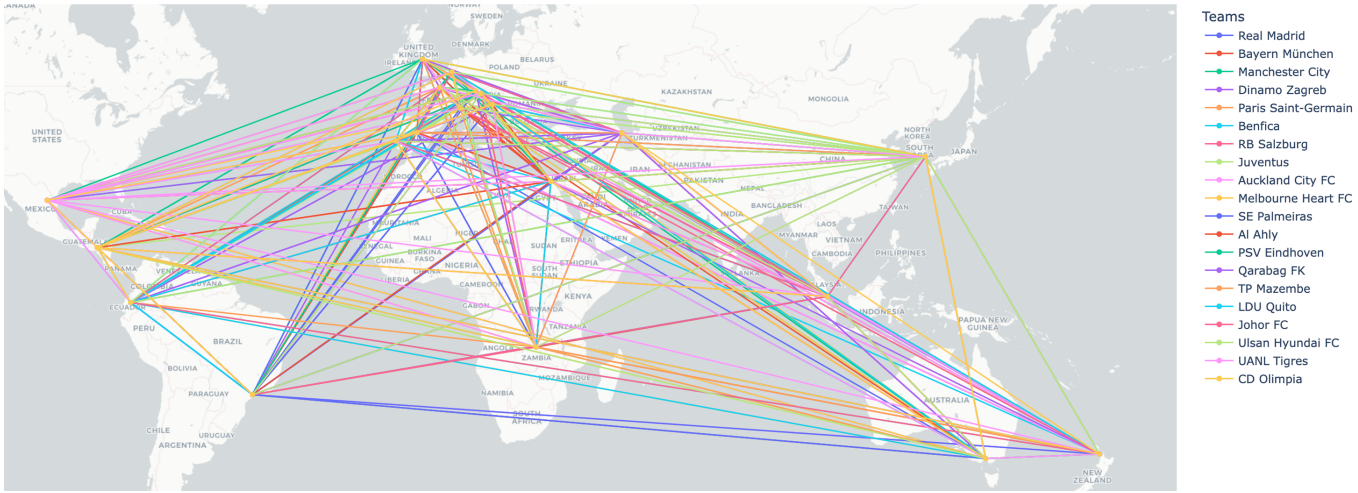


Figure 1: GSL Schedule for 20 clubs

Contents

1 Introduction

The IMMC has tasked us with the problem of creating an efficient schedule for a Global Sports League in a discipline of our choosing. Currently, there are few international competitions in the majority the sports. Most of them are single-time tournaments (cups) that bring together countries from the same continent for a short period of time. This limits its flexibility and excludes fans from certain parts of the world. What has to be distinguished is that a **league is a whole season of matches**, contrary to a short-time tournament. In our consideration, we will use this definition of a league.

It now strikes that throughout the history of our planet, there has never been a fully intercontinental league in any team sport. Most attempts are limited to continent scale (e.g. Eurobasket) or cannot be classified as *leagues* (e.g. FIFA Club World Cup). Today, the International Multi-continental Matchmaking Committee (IMMC) pioneers in creating a worldwide sports league, the Global Sports League (GSL).

Sport competitions reach far beyond just contests in running, kicking throwing or jumping. They create a sense of unity and good spirit between the participants. The International Olympic Committee's goals include peace, inclusion and development through sport.[1] Therefore, the GSL is a great opportunity not only to offer fans from all around the world an fantastic show but also to promote the chosen discipline and values related to fair competition. By observing their idols and international stars playing on their hometown stadiums, some fans will await GSL matches for weeks. The international character of the competition will also offer a significant opportunity for revenues for the IMMC and participating teams.

In our investigation we propose a complete schedule as well as an algorithm to establish the optimal league schedule.

2 Choosing sport

2.1 Sport Suitability Scores

The first task is choosing an appropriate sport, that will have a sufficient following and will give an interesting club mix from different parts of the globe. Therefore, when choosing our sport we narrowed our option to a few of the most popular team sports globally. We chose football, cricket, volleyball, basketball and rugby as potentially viable options for the GSL.[2] The next step was outlining the key quantifiable measures that will allow us to determine the best sport numerically. We deemed the following as crucial:

- Global fanbase - the chosen sport has to have a sufficient global following so that the generated revenue will be high enough to cover the cost of travel and other expenses.
- Playing frequency - The more frequent a sport can be played the easier it is to create a concise schedule.
- Competition level across continents - each continent has to be relatively good at every sport in order to create an interesting clubs combination.
- No dominant nations - it is best if the sport is not dominated by a single nation or a small group of nations.

1. International Olympic Committee, *Olympic agenda 2020 - strategic roadmap for the Olympic Movement*, accessed April 3, 2025, https://www.olympics.com/ioc/olympic-agenda-2020.

2. Aryan Singh, *The most popular sports in the world*, February 2025, accessed April 3, 2025, https://www.worldatlas.com/articles/what-are-the-most-popular-sports-in-the-world.html.

Taking into account the aforementioned factors we created two metrics that allow us to determine the best sport:

$$\textbf{Fan Volume Index (FVI)} = n \times f \tag{1}$$

Where:

- n - number of fans of the sport worldwide
- f - number of times per week the sport can be played

$$\textbf{Match Fairness Index (MFI)} = \frac{v}{\sigma_{continents}} \tag{2}$$

Where:

- v - number of different winners in the past 5 editions of quadrennial world championships in each sport
- $\sigma_{continents}$ - standard deviation of the number of countries per continent the top 50 of the national ranking of the sport

As the the names suggest the FVI takes into account the potential viewership of an event and the potential revenue coming from hosting games in the league. The higher this index is, the better the sport is suited for the GSL. On the other hand, the MFI measures how uniform the distribution of good teams across continents is as well as whether there are some countries that dominate the sport. Similarly to the FVI, the higher its value the better. These two indexes are used to calculate the Sport Suitability Scores (SSS). They are determined using the following formula:

$$\textbf{Sport Suitability Score (SSS)} = \ln(\text{FVI} \times \text{MFI}) \tag{3}$$

To calculate the SSS both the metrics are multiplied and a natural logarithm is applied, as it is a multiplicative index. This also ensures that the results are easy to compare when presented on a graph.

2.2 Result - sport selection

We utilized the Sports Suitability Score from section 2.1 in order to evaluate the aforementioned sports. The code we created can be found in appendix A.3. We collected all the data both for male and female athletes using values from appendix A.1 for men and appendix A.2 for women. The results are presented on Figure 2.

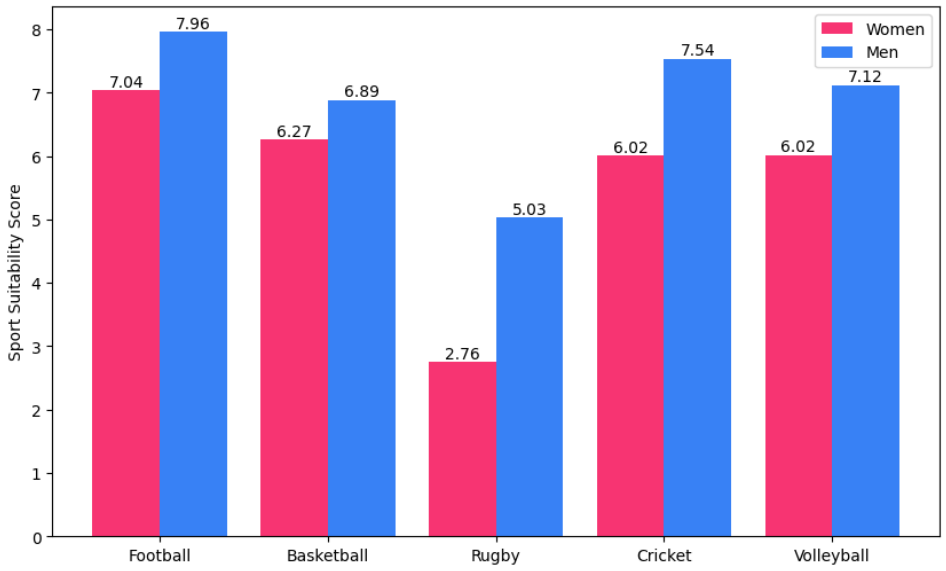


Figure 2: Sport Suitability Scores (SSS) for Men and Women in Different Sports

As can be observed both for male and female data, football scores the highest. Therefore it was chosen as the designated sport for the GSL. Furthermore, as can be observed in the tables from appendix A.1 and appendix A.2 there was a big difference between the number of fans for male and female sports, which reflects why the results for women are always lower.

3 Choosing clubs

3.1 ELO rating system

The second part of the first task was choosing the appropriate clubs. We decided to choose male clubs over female clubs as male football provided the highest score of all the tests sports for each gender. When choosing the appropriate clubs we created our own ELO system and utilized a dataset with match results for 207 world leagues as well as international competitions up till the beginning of 2024. ELO is a method of calculating the relative skill levels of clubs in zero-sum games and it is widely utilized in chess.[3] In general an ELO system uses the following formula to predict the probability of a team winning:

$$E_{T1} = \frac{1}{1 + 10^{\frac{R_{T2} - R_{T1}}{400}}} \tag{4}$$

Where:

- E_{T1} - probability of Team 1 winning
- R_{T1} - ELO ranking of Team 1
- R_{T2} - ELO ranking of Team 2

Since there is only a handful of international competitions, when applying the ELO in its pure form, teams that dominate leagues with lower skill will have the highest ELOs. Therefore, the relative strength of each league has to be taken into account. Moreover, the teams should not lose too many ranking points when participating in international competitions as, them getting in is a success itself. That is why in our solution we implemented a few changes to the system. We started by creating two additional factors: the league factor and the international factor.

The league factor takes into account the relative strength of the best 16 football leagues in the world.[4] In the source provided the league strengths are given with a relative score. For leagues that are not in the table in appendix B.1 the league factor is given value 1. Otherwise, it is calculated using the following formula:

$$\textbf{league factor } (lf) = \frac{s_{league}}{s_{benchmark}} \tag{5}$$

Where:

- s_{league} - league relative strength from table in appendix B.1
- $s_{benchmark}$ - strength score of the last team in the ranking, which in this case is 74.7

The international factor takes into account the importance and level of each international competitions that teams play in. The assumption is that the higher the prize money for a specific competition, the higher the rank and importance of the competition as well as the viewership, which suggests that better teams play in such competitions. Therefore we created a table with all the values of main prizes for the international competitions we use for obtaining the ELO. All these values can be

3. Chess.com, *Elo rating system - chess terms*, accessed April 3, 2025, https://www.chess.com/terms/elo-rating-chess.

4. Global Football Rankings, *Football League Rankings: Analysis of Top Football Leagues*, accessed April 3, 2025, https://globalfootballrankings.com/.

found in appendix B.2. The index has a value 0 for national legaue matches and in international competitions it is calculated using the following formula:

$$\textbf{international factor } (if) = \ln \frac{p_{competition}}{1000} \tag{6}$$

Where:

- $p_{competition}$ - main prize value of a chosen competition

Initially all teams are initiated with an ELO of 1000, with the exception of teams that play in any international competitions as their ELO is set to 1500. Then the matches from the last decade are analyzed chronologically, the point changes for each team are calculated using the following formulas:

$$\Delta_{winner} = lf \times K \times (1 - E_{winner}) + if \tag{7}$$

$$\Delta_{loser} = K \times (0 - E_{loser}) \tag{8}$$

Where:

- $K = 10$ - factor determining the amount of change in ELO rating points
- E_{winner} - win probability of the winning team
- E_{loser} - win probability of the losing team

The assumption in the used ELO rating model is that in case of a draw the elo rating points are not changed. Furthermore, to further gratify winning in international competitions, in such cases the rating points added to the winning team are given by the following formula:

$$\Delta_{winner\ international} = \Delta_{winner} - \Delta_{loser} \tag{9}$$

3.2 Determining 20 clubs

We created the python code for the method outlined above (appendix C.1) and we obtained a worldwide ranking of over 5000 football clubs. As we want the league to bring together fans from as many different countries and background as possible we decided that there can be a maximum of 1 team per country. Therefore, when using the ELO ranking we chose 2 teams best teams from different countries in each continent and then filled the rest of the 20 spots available with the best teams regardless of the continent in which they are. Using this method we obtained the following 20 teams: **Real Madrid, Bayern München, Manchester City, Dinamo Zagreb, Paris Saint-Germain, Benfica, SE Palmeiras, RB Salzburg, Juventus, Al Ahly, PSV Eindhoven, Qarabag FK, TP Mazembe, LDU Quito, Johor FC, Ulsan Hyundai FC, Auckland City FC, UANL Tigres, CD Olimpia, Melbourne Heart FC.** All the ELO ranking points as well as team continents can be found in appendix C.2. Furthermore, the geographical locations of the teams are visualized in Figure 3.

Figure 3: Chosen Clubs Locations

4 Scheduling

4.1 Scheduling Goals and Rules

4.1.1 What is the Goal?

We want a schedule that's efficient but also as fair as possible to every team. Just minimizing total travel can unfairly burden teams far from others (like those in Oceania). So, our target is a balance: we minimize a weighted mix of the overall travel distance ($TotalTravel$) and how unevenly that travel is distributed across all the teams(standard deviation, $StdDev(Travel)$):

$$\text{Minimize Cost} = W_{\text{Total}} \cdot TotalTravel + W_{\text{StdDev}} \cdot StdDev(Travel) \tag{10}$$

Here, $Travel_i$ is team i's total season travel, and $W_{\text{Total}}/W_{\text{StdDev}}$ let us tune the balance.

4.1.2 Constraints

Before optimizing, any schedule *must* follow these rules:

1. **Play Everyone Twice (Double Round Robin):** Each team has to play every other team exactly once at home and once away. This system is well-established and is considered to be one of the most fair reasonable ones.

2. **One Game Per Week:** Every team can play no more then one game each week. This ensures fatigue minimization.

3. **No Crazy Streaks:** Teams can't stay home or away forever. We limit consecutive home games to H_{max} and away games to A_{max}.

4.2 Distance Data

In order to effectively minimize travel and evaluate the fairness of its distribution, accurate distance information between all pairs of teams is essential. This data forms the basis for calculating the $TotalTravel$ and $StdDev(Travel)$ components in our objective function (equation 10).

4.2.1 Financial analysis

GSL, being one of the greatest, most entertaining, and value-providing leagues in history, must be maintained. To ensure its annual organization, IMMC needs to consider economic rationality. We must also include ecological damage in cost calculations to preserve the planet for future players.

This is especially important since the ecological impact of teams' fans is considerably larger compared to that of teams themselves. We assume that most of team's fans live in their home city, making the economic and ecological costs proportional to the distance traveled per team member (considering average fuel consumption during flights), plus constants. These constants account for the environmental impact of fans commuting to and from the stadium (while there are already in host city) as well as their consumption at the venue.

We do not consider any additional ecological impact from a team's stay in the hotels, as their consumption is assumed to remain consistent with their home environment. Hotel stay costs are included as a constant since teams spend five days in hotels in hots's city, regardless of circumstances. Taking that under consideration, we can derive a formula for total costs as a function of distance, as well as the team's specific costs (limited to hotel stays and flights):

$$\begin{aligned}\text{Team + Environment (total)} = 0.28 \times 77,000 \times (0.01919 + 0.11) \times d \\ + 5 \times 236 \times w + 7,753,000 \times n \times 0.19\,[\text{USD}]\end{aligned} \tag{11}$$

$$\text{Team + Environment (total)} = 2,785.3364 \times d + 1,473,070 \times n + 1,180 \times w\,[\text{USD}] \tag{12}$$

$$\text{Team (per member)} = 0.11d + w \times 1180\,[\text{USD}] \tag{13}$$

Where:

- Average stay at a 5-star hotel worldwide: 236 USD per day
- CO_2 emissions while burning jet fuel: 3.16 kg CO_2 / kg jet fuel
- Social Cost of Carbon (SCC) for 1 kg of CO_2 burned: 0.19 USD/kg CO_2
- 7,753 tons of CO_2 are released during a match (one flight "generates" one match to one match)
- Cost of a flight: 0.101 kg CO_2 burned per passenger per km (on average, as fans travel via various types of planes), equivalent to 0.01919 USD/km
- Median flight cost (for example Emirates): 0.11 USD/km
- Average football stadium accomodate 77,000 spectators
- 28% of spectators come from guest city
- n = number of matches played (constant in our solution)
- w = number of weeks (constant in our solution)

This model assures us that minimization of total distance is needed. The function of total cost for IMMC is sum of cost for teams + cost of organizing matches which doesn't depend on the schedule of gameplays. This means that it is linear with respect to the distance travelled by all of the teams. Optimizing this allows us to decrease both economic and environmental costs.

4.2.2 Data Acquisition and Calculation Method

We begin by sourcing the geographical coordinates (latitude ϕ and longitude λ) for the designated home location (e.g., city or primary stadium) of each of the $N = 20$ teams. Latitude and longitude are expressed in radians for the calculation.

To compute the travel distance between any two teams, i and j, located at (ϕ_i, λ_i) and (ϕ_j, λ_j) respectively, we employ the Haversine formula. This formula calculates the great-circle distance, which is the shortest distance between two points on the surface of a sphere. This provides a robust approximation for inter-city travel distances, particularly relevant for potential flight paths in a geographically diverse league.

The Haversine distance d_{ij} between team i and team j is calculated as follows:

$$\Delta\phi = \phi_j - \phi_i \tag{14}$$

$$\Delta\lambda = \lambda_j - \lambda_i \tag{15}$$

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_i)\cos(\phi_j)\sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{16}$$

$$c = 2 \cdot \operatorname{atan2}\left(\sqrt{a}, \sqrt{1-a}\right) \tag{17}$$

$$d_{ij} = R \cdot c \tag{18}$$

Where:

- $\Delta\phi$ and $\Delta\lambda$ - the differences in latitude and longitude (in radians).
- a - the square of half the chord length between the points.
- c - the angular distance in radians.
- R - the Earth's mean radius (approximately 6371 kilometers). We use this value to obtain distances in kilometers.
- $\operatorname{atan2}(y, x)$ - the arctangent function that handles quadrants correctly.

4.2.3 Distance Matrix Representation

Repeating this calculation for all pairs of teams yields a symmetric $N \times N$ matrix, which we denote as D. Each element $D[i][j]$ stores the computed distance d_{ij} (in kilometers) between team i and team j. By definition, the diagonal elements are zero ($D[i][i] = 0$) and the matrix is symmetric ($D[i][j] = D[j][i]$). This matrix is the core input for evaluating the travel cost of any proposed schedule.

4.2.4 Visualization

Figure 4 visually represents these pairwise distances. The map displays the home locations of all teams and connects each pair with a line representing the calculated great-circle distance. This visualization helps to intuitively understand the geographical scale of the league, identify potentially isolated teams (like those in Oceania, as mentioned earlier), and reinforces the need to balance total travel minimization with the fairness criterion ($StdDev(Travel)$) captured in our objective function.

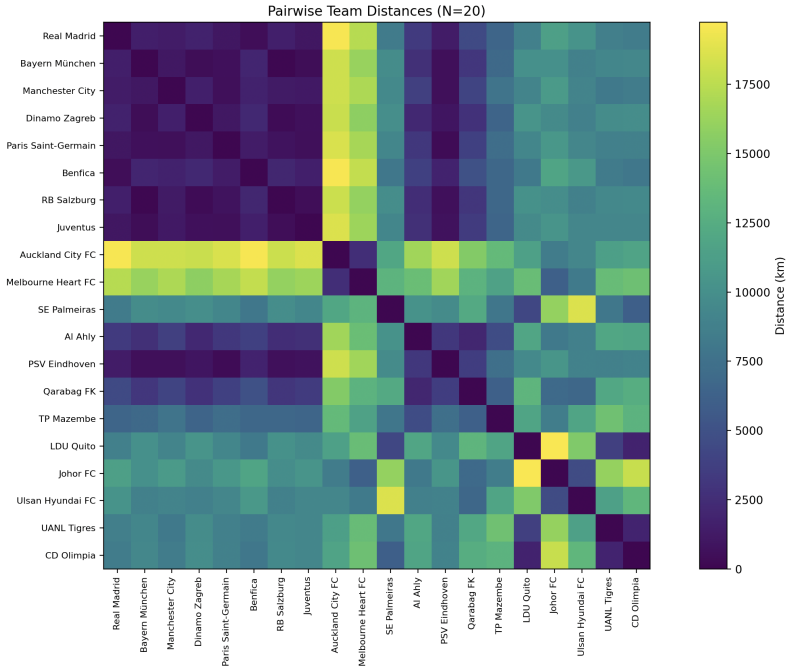


Figure 4: Visualization of Pairwise Team Distances.

4.3 Phase 1: Just Getting a Valid Schedule (CP-SAT)

Honestly, just finding *any* schedule that meets all those rules for 20 teams is tough! It's a classic combinatorial puzzle. To tackle this, we first use Google's OR-Tools CP-SAT solver. Its job is simple: find us *one* schedule that works, fast. We don't care about travel distance yet, just validity.

4.3.1 How CP-SAT Does It

We describe the problem to the solver using variables and constraints:

- **Variables:** For each team t and week w, we need to know its opponent (`opponent[t, w]`) and whether it's playing at home (`is_home[t, w]` $= 1$) or away (`is_home[t, w]` $= 0$).
- **Constraints:** We translate the rules:
  1. **Game Pairing:** If A plays B, B must play A, and one must be home, the other away. Mathematically:

$$\texttt{opponent[opponent[t, w], w]} = t \tag{19}$$

$$\texttt{is\_home[t, w]} + \texttt{is\_home[opponent[t, w], w]} = 1 \tag{20}$$

2. **DRR Met:** Ensure every team hosts every other team exactly once across the season. For distinct teams t, t':

$$\sum_{w \in \mathcal{W}} (\mathbb{I}(\texttt{opponent[t, w]} = t') \wedge \mathbb{I}(\texttt{is\_home[t, w]} = 1)) = 1 \tag{21}$$

($\mathbb{I}$ is 1 if true, 0 otherwise; $\mathcal{W}$ is the set of weeks).

3. **Streak Limits:** Enforce the max consecutive home (S_{home}) / away (S_{away}) games. For each team t and valid start week w:

$$\sum_{w'=w}^{w+S_{home}} \texttt{is\_home[t, w']} \leq S_{home} \tag{22}$$

$$\sum_{w'=w}^{w+S_{away}} (1 - \texttt{is\_home[t, w']}) \leq S_{away} \tag{23}$$

We set a time limit and use a handy callback function: as soon as CP-SAT finds the *first* valid schedule, we grab it and stop the search. This keeps Phase 1 quick. The list shows a look at the code that defines these rules for the solver.

4.4 Phase 2: Optimization with Simulated Annealing (SA)

Alright, Phase 1 gave us a schedule that follows the rules (S_{feas}). Now, we need to make it efficient according to our objective (equation (10)). For this, we turn to Simulated Annealing (SA), a smart heuristic good at exploring complex options and avoiding getting stuck with just "okay" solutions.

4.4.1 The SA Process: Shake, Check, Decide

SA starts with the valid schedule S_{feas} and tries to improve it iteratively:

1. **Shake it Up (Neighbor Generation):** Randomly tweak the current schedule slightly. We used moves like:
   - `SwapHomes`: Pick a week, find two home teams, and swap their opponents.
   - `SwapRounds`: Pick two different weeks and swap all their matchups.

   (Implemented in the Python code's main loop).

2. **Rule Check (Feasibility):** This is critical! After tweaking, check if the *new* schedule still follows all the DRR and streak rules using our `check_schedule_feasibility` function. If not, toss the tweak and try again. We only work with valid schedules.

3. **Cost it (Evaluation):** If the tweaked schedule is valid, calculate its cost using our weighted objective function (equation (10)). This involves calculating total travel for all teams ($TotalTravel$) and the standard deviation of those travels ($StdDev(Travel)$), using pre-computed distances ('distanceMatrix').

4. **Keep or Not? (Acceptance):** Compare the new cost to the current cost ($\Delta Cost = Cost_{new} - Cost_{current}$).
   - If $\Delta Cost < 0$ (it's better), definitely keep the tweaked schedule.
   - If $\Delta Cost \geq 0$ (it's worse), we *might* still keep it! The chance is $P = e^{-\Delta Cost/T}$, where T is the current "temperature". Early on (high T), we accept worse moves more often to explore; later (low T), we get pickier. This is the key to escaping local minima.

5. **Track the Best:** Always remember the best valid schedule found so far across all iterations.
6. **Cool Down:** Gradually decrease the temperature T (e.g., $T \leftarrow T \times \alpha$ using `SA_COOLING_RATE`, $\alpha < 1$).

We run this loop for a set number of iterations (`SA_MAX_ITERATION`). The best schedule found overall is our final result.

4.5 Generating Match Hours

Once we use the algorithms from section 4.3 and 4.4 we obtain a schedule which tells the match order as well as which team is playing at home. However, it does not determine the time of the matches. Therefore, a separate method has to be created in order to determine the suitable match hours. The timezones of our chosen clubs are varying and range from UTC-06:00 to UTC+12:00. The teams will be traveling all over the world to teams from different timezones. The human body can shift its biological clock by approximately 1 hour per day and we assume that a team gets to the city they play in 4 days before their match.[5] Therefore, naturally their biological clocks will be able to shift only by 4 hours, which means that they will not fully adapt. Therefore, a system has to be developed where the 2 teams playing themselves experience the same level of discomfort when it comes to the time of the match. We decided to represent the biological clock of a team by a cosine function. The higher its value the more ready a team is to play during a chosen hour:

$$\textbf{Club Match Time Suitability (CMTS)} = -\cos\left(\frac{2\pi}{24} \times (t - 4 - z)\right), \quad 0 \leq t < 24 \tag{24}$$

Where:

- t - match time
- z - club current timezon e in UTC

Equation (24) gives the Club Match Time Suitability (CMTS) graph of a team adjusted to UTC+00:00. The -4 stems from the fact that usually matches are played in the afternoon or evening, therefore the graph is always shifted to the right by 4 hours, changing the most optimal match time from 12 p.m. to 4 p.m.

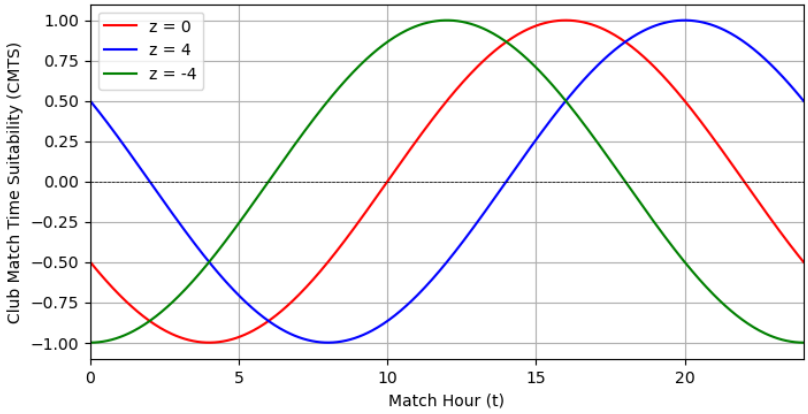


Figure 5: Club Match Time Suitability (CMTS) for varying z

When choosing match hours we simply take the current CMTS graphs for each team, that have been adjusted by a maximum of 4 hours in the direction of the timezone they will be playing in. To obtain

5. Charmane I. Eastman et al., "Advancing circadian rhythms before eastward flight: A strategy to prevent or reduce jet lag," *Sleep* 28, no. 1 (January 2005): 33–44, https://doi.org/10.1093/sleep/28.1.33.

the match time the positive solution of equating the graphs of two playing teams is taken and then this hour is adjusted for the timezone of the city in which the match will be played. We assume that a match usually lasts about 2 hours with the break. Hence, we subtract an 1 hour from the obtained values, so that the middle of the match will pass through the solution of the equation. Applying this method for the results from optimizing the week schedule yielded results from appendix D.2.

Furthermore, by taking the average CMTS value of each time throughout the entire schedule, the equality of the determined times can be compared. The lower a team's score the more fatigued on average it was when playing matches. These scores the determined schedule are in appendix D.3. As can be observed, the values range from 0.98 for some teams in Europe to 0.84 for teams in Oceania, which suggests that some teams were, on average, better rested than the teams in Oceania. However, such a result is inevitable as the teams in Oceania had to cover large distances to get to their opponents and they traveled across many different timezones.

A careful selection of match starting times is furthermore important not only for the teams, but also for their fans. It is worth noticing that the guest team brings with itself a prior distribution of their fans' attention. This highlights another advantage of our approach over simply scheduling based on home time - more fans will be able to watch their favorite clubs playing away. This will additionally contribute to viewership and potential revenue for the IMMC.

4.6 Results

4.6.1 Initial Feasible Schedule

Phase 1 (CP-SAT) successfully found an initial feasible schedule satisfying DRR and streak constraints ($H_{max} = 5, A_{max} = 5$) within the 60-second time limit. The travel characteristics of this initial schedule are summarized in Table 1.

Metric	Initial Feasible Schedule	Optimized Schedule
Total Travel Distance (km)	4,935,726	4,501,332
Std. Dev. of Team Travel (km)	61,532	13,455
Spread of Team Travel (km)	244,367	42,645
Min Team Travel (km)	165,372	209,740
Max Team Travel (km)	409,739	252,385
Combined Cost (Eq. 10)	70,613,073	17,803,059

Table 1: Comparison of Initial (CP-SAT) and Optimized (SA) Schedule Travel Metrics

4.6.2 Travel Path Visualization

Figure 6 visualizes the full-season travel itineraries for each of the 20 teams based on the final optimized schedule. This map helps illustrate the geographical scope of the league and the typical travel patterns required.

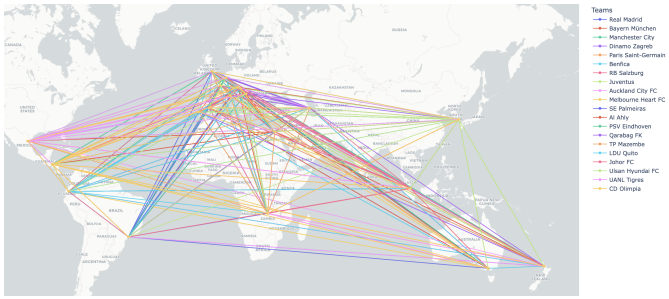


Figure 6: Visualization of Optimized GSL Travel Paths for all 20 Teams over 38 Weeks.

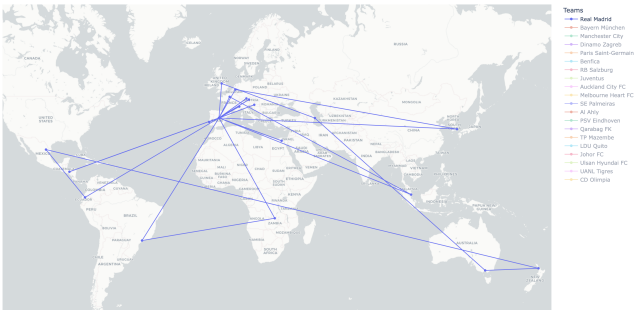


Figure 7: Visualization of Optimized GSL Travel Paths for one team (Real Madrid).

4.7 Probability

Participation in the GSL is a huge commitment for the teams, as it involves playing intense matches almost every week for the whole duration of 9 months. Therefore, it is crucial to ensure that the league format allows to find the best team. A good league schedule and scoring system should be characterized by their ability to quickly put the best team on top of the ladder. For this reason, we would like to know, how likely it is that the leader will be the winner.

Let's say that there are n players competing with each other. In a Double Round Robin format, each team plays against every other team twice—once at home and once away. To determine the total number of matches played, we first analyze how many matches would occur in a Single Round Robin. In such a system, each out of n teams competes against the remaining $n-1$ teams exactly once, leading to a total of $\frac{n(n-1)}{2}$ matches (as $n(n-1)$ calculates each match twice; when team A plays with team B and vice versa). However, in a Double Round Robin, each match is played twice, effectively doubling the number of matches back to $n(n-1)$.

Now, after x matches have been played, each team has competed in a fraction of their total matches. Since each pair of teams is expected to play $n-1$ double matches (i.e., $2(n-1)$ games in total), we assume that every pair has completed $\frac{x}{m}$ of their scheduled double matches, where $m = n(n-1)$ is the total number of matches. Thus, we assume that every pair of teams has already played $\frac{x}{m}$ of their Double Round Robin fixtures.

At this point, we can compute the expected value of the points each team will have at the end of the tournament. Let S_i be the current score of club i. The expected final score of the i-th team is given by:

$$E_i = S_i + q \sum_{j \neq i} P(i \text{ wins against } j) \tag{25}$$

where $P(i \text{ wins against } j)$ is the probability that player i wins against player j, and q represents the fraction of remaining matches to be played, defined as:

$$q = 1 - \frac{x}{m} \tag{26}$$

The variance of the final score is given by:

$$\sigma_i^2 = \sum_{j \neq i} qP(i \text{ wins against } j)q(1 - P(i \text{ wins against } j)) \tag{27}$$

$$\sigma_i^2 = q^2 \sum_{j \neq i} P(i \text{ wins against } j)(1 - P(i \text{ wins against } j)) \tag{28}$$

We now aim to compute the probability that no player will surpass the current leader. Let L be the final score of the leader and i the final score of $i-th$ team. Let's derive $P(i \text{ wins with } L) = P(i > L)$.

If we assume that $i \sim N(E_i, \sigma_i^2)$ and $L \sim N(E_L, \sigma_L^2)$, then the difference $i - L$ also follows a normal distribution:

$$i - L \sim N(E_i - E_L, \sigma_i^2 + \sigma_L^2) \tag{29}$$

Thus, we can express the probability that team i will surpass the leader as:

$$P(i > L) = P(i - L > 0) = 1 - \Phi\left(\frac{E_L - E_i}{\sqrt{\sigma_i^2 + \sigma_L^2}}\right) \tag{30}$$

where Φ represents the cumulative distribution function (CDF) of the standard normal distribution. Thus, the probability that the leader remains in first place is:

$$P(\text{leader wins}) = 1 - \prod_{i \neq L} P(i > L) \tag{31}$$

$$P(\text{leader wins}) = 1 - \prod_{i \neq L} P(i > L) \tag{32}$$

This formula allows us to calculate the probability that the current leader will win the entire tournament given the number of rounds played (x) and the current scores. We have implemented this model in Python (see appendix E.1), which computes the probability for given scores and rounds instead of matches, and generates probability distributions over different values of x. Our mathematical model assumes that for each game, a team can either gain 1 point or receive 0 points. However, in our implementation, we follow the common football scoring system, where a victory awards 3 points. Consequently, our variance formula adapts as follows:

$$\sigma_i^2 = 9q^2 \sum_{j \neq i} P(i \text{ wins against } j)(1 - P(i \text{ wins against } j)) \tag{33}$$

Similarly, the expected final score of player i is given by:

$$E_i = S_i + 3q \sum_{j \neq i} P(i \text{ wins against } j) \tag{34}$$

Assumptions:

- Each team plays with every other team twice (home and away).
- The probability of winning a match depends only on the Elo rating of the teams.
- A win earns 3 points and a loss earns 0 points for the team.
- The probability of a team's final score follows a normal distribution.
- The number of remaining matches is the same for each team.
- Elo of a team doesn't change over time.
- The expected scores of the leader E_L and other players E_i are independent random variables.

The same theoretical model allowed us to create a graph of probability that the winner will win as a function of total number of rounds. We did it by substituting zeros as initial scores, zero as initial number of rounds played. We only altered total numbers of games played what resulted in different expected values and variances results according to the formulas:

$$E_i = 3rq \sum_{j \neq i} P(i \text{ wins against } j) \tag{35}$$

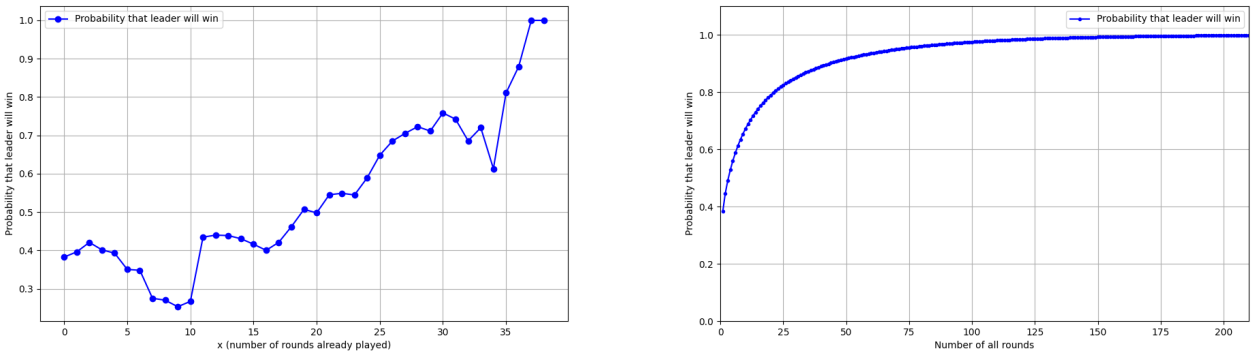


Figure 8: **LEFT**: Probability that Leader Will Win as a Function of X. **RIGHT**: Probability that Leader will Win as a Function of Number of All Rounds

$$\sigma_i^2 = 9rq^2 \sum_{j \neq i} P(i \text{ wins against } j)(1 - P(i \text{ wins against } j)) \tag{36}$$

Where r is the number of rounds. Code for this can be found in appendix: E.2.

Conclusions: As we can see on the Figure 8 **LEFT** (a graph for which teams' scores are averages of their scores from 10 000 simulations) the strongest club (which is most often the leader) until 35rd round do not have high certainty that they'll win, which means that it is possible that in the last moment another team will surpass them, what spices up the emotions in IMMC's GSL. However taking a look at Figure 8 **RIGHT** we can see that if the number of rounds was around 36, like in our case, probability at the beginning of the League that Real Madrid was going to win is almost 90%, informing us that under such assumption the winner will almost certainly emerge.

5 Extended GSL

5.1 Adding new teams

Great news for fans - the GSL has been extended to 24 teams! For the selection of them, we use the same procedure as in 3.1. The following teams were added: **Shakhtar Donetsk** (Ukraine, Europe), **Mamelodi Sundowns** (South Africa, Africa), **Celtic FC** (Scotland, Europe, **Esperance Tunis** (Tunisia, Africa).

5.2 The clustering algorithm

The addition of new teams increases the number of games, if the same procedure for determining schedule is used. In case of double robin, 46 matchdays (weeks) would be required for all teams to play against each other team twice - this would take more (almost 11 months) than the time desired by the IMMC (8-9 months).

Therefore, we approach the problem as a task to **create a league system where the choice of matchup allows for best determination of the winner, without everyone playing against each other** (twice). At the same time, our goal is to reduce travel costs and related consequences for the well-being of players. We also aim for an equitable and fair solution, both with in opponents selection and travel inconveniences.

One approach to assign matchups, borrowed from chess, would be to apply the Swiss system.[6] Here, matchups are made based on current leader board, so stronger teams would compete with stronger

6. Chess.com, *What is a Swiss tournament? — Chess.com Help Center*, accessed April 3, 2025, https://support.chess.com/en/articles/8558054-what-is-a-swiss-tournament.

ones, whereas weaker clubs would fight among themselves. However, the Swiss system is not fully feasible for our case:

- It is undesirable to play solely based on previous results - geographical proximity should also play a role (cutting costs).
- It is infeasible to reschedule the games every week. This would significantly complicate planning and could increase team costs.

The latter contradicts the IMMC goal of logistic and economic feasibility. We must, therefore find a balance between rescheduling and providing a reliable and consistent schedule. Therefore, we will employ a hybrid option - a balanced K-means algorithm for clustering teams. The teams will be clustered into 6 groups of 4 teams each. The choice of the groups will be based on both geographic proximity (flight reduction - economic and environmental sustainability) and teams' relative strength (competitiveness and equity of matchups) This will allow competitive games in all geographical regions, including the normally underrepresented in international football competitions.
Inside a group, the teams will play in the double robin system; points to the general ranking will be awarded for these games. The size of a group (4 teams) implies that clusters will be changed every sixth game, providing balance between adjusting to team strength and reducing uncertainty and posing logistical challenges to the teams.

5.2.1 Balanced K-means clustering

A standard K-means algorithm allows for grouping of The data is presented as a vectors in n-dimensional space. Values of specific components of the vector can represent its various characteristics. Centroids, points in the centers of clusters, are first chosen randomly. Each point is assigned to closest centroid, based on euclidean distance, in the considered . Centroids are then shifted to new mean locations, computed as average coordinates of the points (vectors) in each cluster. Then points are assigned to clusters with new centroids and the process is repeated until convergence. The use of K-means gives us a great advantage - we can group teams by both their ranking scores and their geographical proximity.
For the sake of this algorithm, we denote the information about the i-th team in a vector in 4D space

$$\vec{v_i} = (x_i, y_i, z_i, w \times s_i) \tag{37}$$

Where:

$$x_i = 6371 \cos\phi_i \cos\lambda_i, \quad y_i = 6371 \cos\phi_i \sin\lambda_i, \quad z_i = 6371 \sin\phi_i$$

are the spatial coordinates of a city in a reference frame with the origin in Earth core (radius of Earth is taken as 6371 km) and s_i is the $i-th$ team's number of ranking points (in the league) and w is the respective weight.
In this representation, the distance between two cities cannot be computed using the Pythagorean theorem. Therefore, we modify the formula for distance between points in our 4D space to

$$D(v_i, v_j) = \sqrt{(distance(x_i, y_i, z_i, x_j, y_j, z_j))^2 + w^2(s_i - s_j)^2} \tag{38}$$

Where

$$\text{distance}(x_i, y_i, z_i, x_j, y_j, z_j) = 6371 \arccos\left(\frac{x_i x_j + y_i y_j + z_i z_j}{6371}\right) \tag{39}$$

Is the distance on the sphere between the points, calculated as length of great circle arc with angle calculated as the angle between the spatial components between the vectors (obtained from the dot product formula).

The centroid in each round is calculated as the euclidean mean position of the points in the cluster. However, the spatial coordinates of the vector are normalized such the the centroid lays on the surface of Earth:

$$c_k = \text{normalised}(\sum_{i \in clusterk} x_i, \sum_{i \in clusterk} y_i, \sum_{i \in clusterk} z_i, \sum_{i \in clusterk} ws_i) \tag{40}$$

Where k indexes the centroid (cluster). The remaining score coordinate is not subject to normalization.

In our case of grouping, clusters are restricted to have the same size of 4. To ensure this, we slightly tweak the algorithm of cluster assignment. For each centroid, 4 closest points are found. The points are assigned to clusters in the following order

1. Points closest to only 1 cluster are assigned to the cluster, in whose 4 closest points they belong
2. The point that are assigned to only 1 cluster get assigned to their closest cluster (if not full yet)
3. Points assigned to more than 1 cluster
4. faraway points not assigned to any cluster

If any of the conditions does not occur for a point, the algorithm completes the assignment of points which satisfy that condition and then proceeds to the next group. In each group, points whose difference in distance between the second-closest and first-closest centroid is the greatest, are subject to assignment first. This order allows to first deal with highly certain points, then highly disputed between centroids and finish with points whose assignment to any centroid is not exceptionally good, ensuring proper clustering.

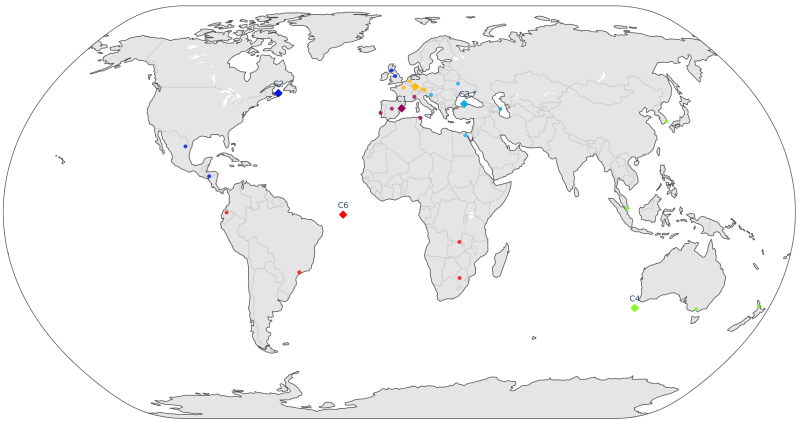


Figure 9: Teams clustered into 6 groups based only on geographic location, presented on a map. Centroids are labeled with larger diamond-shaped points. On the map, they may appear to not be in the group center, but this is largely due to projection error.

5.3 Algorithm performance

In Figure 9 we demonstrate the performance of the balanced K-means clustering algorithm with $w = 0$ (clustering by geography only). It is therefore proven that the algorithm does very well with spatial classification of the teams. The implementation code in Python is attached in appendix F.1.

5.4 Trade-off between goals

Some of the goals of the IMMC face trade-offs, which means they cannot be all achieved at the same time. For instance, larger geographic diversity comes at economic and environmental transportation

costs. A reliable winner determination system, as demonstrated in section 4.7, could require a larger (so more costly and more challenging for the players). Following the IMMC's goals, we now adjust the parameters of the clustering model to address them most effectively.

5.4.1 Weight adjustment

There remain 2 factors to be optimized: the weight w next to team's point scores and the length of the season. The meaning of w is it represents the relative importance of flight (cost and environmental damage) reduction and arrangement of competitive matchups. If the weight w is too low, teams will

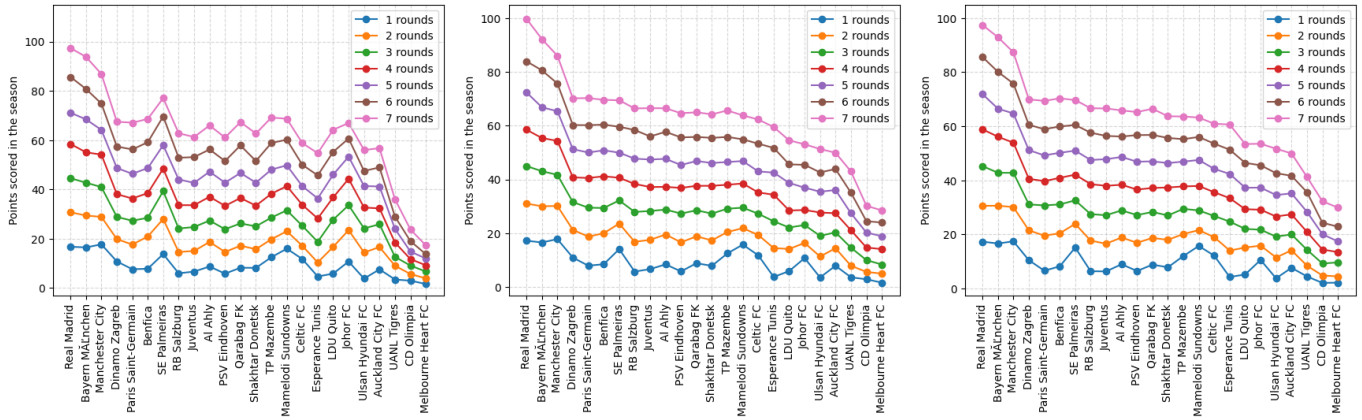


Figure 10: Monte Carlo points scored by teams for different lengths of the season **LEFT**: For $w = 500$ teams are clustered almost exclusively geographically. **CENTER**: For $w = 7000$. **RIGHT**: For $w = 15000$.

be clustered almost always by geography. As a result of that, consecutive matchups could be very similar and unequal (a strong team plays its neighbor just because of geographic proximity). This can observed in Figure 10 **LEFT** for SE Palmeiras. Their closest neighbors are in the Americas and in Africa. All these teams have lower ELO than SE Palmeiras, therefore the latter benefits from its location. On the **CENTER** and **RIGHT** image it is visible that there is with larger weights any advantages related to geographical position disappear within 4-5 rounds. In order to achieve comparable results to the double robin (similar number of games), and ensure the reliability of the clustering method, we choose to play 6 rounds per season. Each round consists of a double robin for 4 teams (6 matchdays), which results in $6 \cdot 6 = 36$ games per team.

In figure 11 **LEFT** the number of points scored by each team is presented for different weights w; teams are sorted by ELO. It can be observed that the league outcome is relatively similar for any positive weight (above 3000). For $w > 6000$ or unexpected results are virtually nonexistent. On the other hand, on the **RIGHT** it can be observed that with increasing w, the average travel distance also increases. This is a sensible result - for higher dependency on score, geographic clustering decreases travel distance by a smaller amount. For this reason it is advisable not to overestimate w. Therefore, in schedule creation we advise the IMMC to use weight $w = 9000$ for 6 rounds of clustering. It is worth noting that $w = 9000$ is in the order of magnitude of Earth's radius, implying that both travel distance reduction and score accuracy optimization (matchup competitiveness) are equally important.

5.5 Comparison of scheduling performance

The advantage of balanced K-means is that it takes into account **both** the geographic distribution of teams and their relative strength (scored points). This means that the leader more often plays with stronger opponents, whereas the less competitive teams are allowed to play competitive games within their group. Despite this, stronger teams still have more points (won games) than the weaker

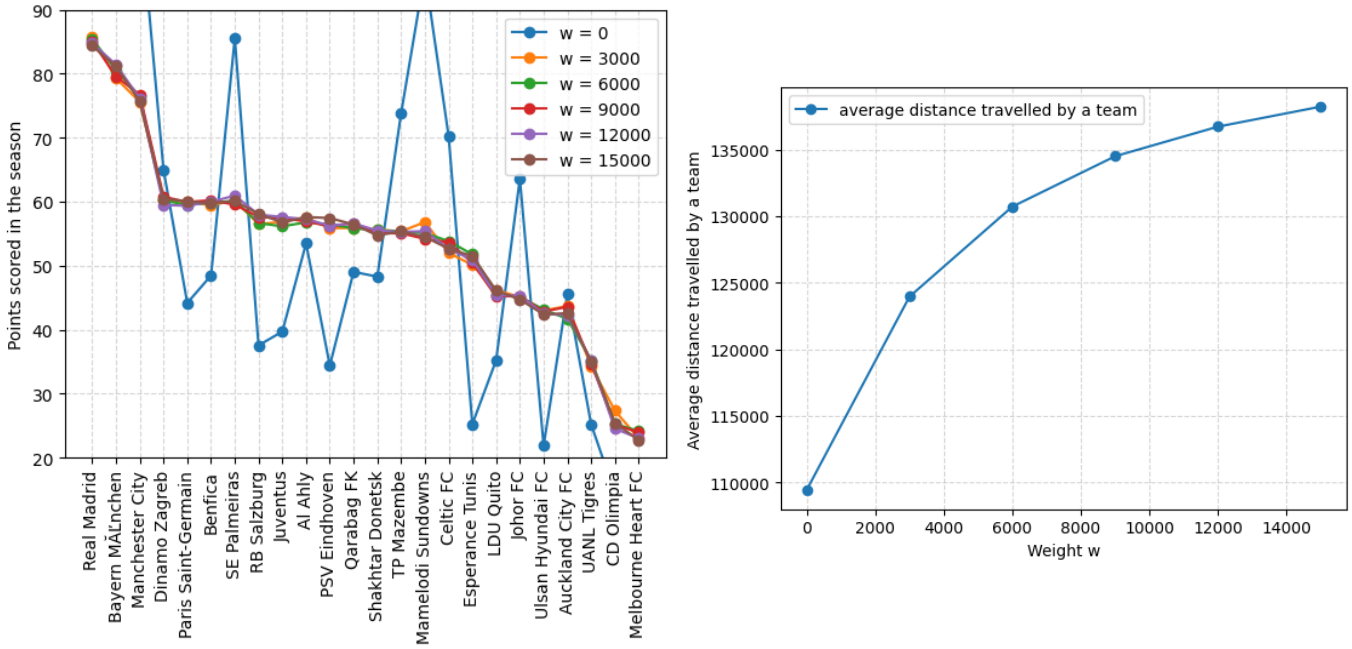


Figure 11: **LEFT**: Monte Carlo number of points scored by different teams for 6 rounds for different weights. **RIGHT**: Monet Carlo average total distance traveled by a team during a 6-round season.

ones (Figure 10). Therefore, winning under clustered groups schedule involves proving victorious in more balanced and competitive games. The competitiveness of matchups can be expressed as the average difference in ELO of team playing against each other. A comparison of relevant parameters with the **20 teams** double robin is presented in 2. We can observe that the Balanced K-means schedule manages to significantly decrease the cost of the GSL and increase its competitiveness (half the mean ELO difference of the Double Robin).

Metric	Optimized Double Robin Schedule	Balanced K-means schedule
Total Travel Distance (km)	4,501,332	3,376,193
Distance per team (km)	225,067	140,674
Mean ELO difference	380	192
Number of match days	38	36
Number of teams (km)	20	24
Estimated total cost ($)	65 mln	70 mln.

Table 2: Comparison relevant metrics in the double robin and balanced K-means clustering

The IMMC would also like to be handed the full schedule generated at the beginning. In order to provide it, we simulate an average clustering season. In each clustering round, we run 1000 Monte Carlo simulations to obtain average point scores for each team. Anytime, the match result is simulated using the formula (4). Then we establish a ranking and cluster for the next round. The process is repeated until all round are simulated and full schedule is obtained. The schedule, with match hours calculated as in section 4.5 is presented in the appendix F.2.

6 Generalization of the Model

We have successfully built a model for obtaining an optimal schedule for the Global Football League (GFL), following IMMC goals. A strong side of the approach we present in this paper, is the fact that it is easily generalizable to any other desired sports. So far, it has boiled down to a few key steps:

1. Figure out how good the teams are (ELO system).
2. Know where everyone is and how far they have to travel (the distance matrix and costs).
3. Set the ground rules: how long the season is, how many games a week, no crazy home or away streaks.
4. Decide what it takes to get a good schedule: balancing the total travel cost, while making sure no team gets unfairly long trips (our optimization goal).
5. Use smart tools to first find any schedule that works (CP-SAT) and then polish it to meet our "good schedule" criteria (Simulated Annealing to the rescue).
6. And finally, for the extended case of 24 teams, use clustering (balanced K-means grouping) to group teams smartly, as playing everyone twice takes too much time and money.

None of these are team-, geographic distribution- or football-specific. They can work for almost all other team sports (as long as those which fit the GSL requirements for the number of active players on the field/court).

6.1 Applying our model to different sports

The GSL could be decided to accommodate different disciplines. As defined in section 2.1, some sports' characteristics can be quantified in FVI and MFI, scores which give us relevant information about what to optimize. For instance if a discipline is desired, but has low MFI (high variation between teams' strengths) we could consider rising the minimum number of teams from a continent to 3.
The **ELO engine** should be adjusted accordingly:

- *Sensitivity (K-Factor):* A lower K-factor might be more reasonable for a fast-paced sports like **T20 cricket** where upsets are more frequent, in order to avoid dramatic changes.
- *League Strength:* For various sports different league are dominant, so different league factors would be assigned to different countries.

6.1.1 The impact of a sport's rules

Various disciplines are contained within their sets of rules. The most significant of these are game times and hence different frequencies of games the players are reasonably able to physically endure. This is captured in the FVI, so sports with high frequency of games are likely to be included in the GSL. The capability to play more matches is an advantage that can be used to more accurately determine the winner. However, as demonstrated in 4.7, a relatively small number of rounds is necessary. Therefore we propose that break in the season are introduced for high frequency sports. They will allow the players to regenerate or enable the teams to participate in other, parallel championships.
Some sports may have extraordinary rules that need special accommodations. For example, cricket test matches can last up to 5 days). This could significantly decrease the frequency of games and make double robin inapplicable for the league. Here, the balanced K-means clustering algorithm can be successfully employed.

6.2 Team dynamics

Depending on the IMMC's attitude, a second class league could be selected in a similar manner to create a promotion-relegation system. Since the season schedule is computed at the start of each season, therefore it should be relatively simple to find a new calendar. We proposed that teams are selected based on an ELO ranking, accounting for geographical representation.

6.2.1 Further league expansion

Following the first year's success, the GSL could be expanded to include more teams. The balanced K-means algorithm has been shown to handle larger leagues well, so it would be applied there. Further expansion could create further logistical challenges such as shared infrastructure between the teams (e.g. Inter Milan and AC Milan). Our model does, however provide a framework to solve this issue - if the teams happen to be clustered together, a constraint to the double robin CP-SAT algorithm can prevent date collisions.

6.2.2 Women's sports

In section 2.1 we compared the Sport Suitability Score for different disciplines, for both genders. Men scored higher in each sport, however the difference was mostly marginal and largely due to . Women had better MFIs, which means that women's sports are more competitive cross-country. It would be therefore extremely interesting, given enough time, to investigate how the GSL would evolve for women's sports.

7 Evaluation

7.1 Strengths

Overall, the model we have created produces optimal results in a relatively short amount of time, especially when considering that the problem is NP-hard and using methods like integer linear programming stops being feasible with the number of clubs as low as 8. In both models, for 20 and 24 teams, we are able to significantly decrease the average distance traveled when compared to a random schedule that meets the constraints.

Furthermore, the implemented balanced K-Means clustering method ensures competitive matchups along with geographical consideration and financial as well as environmental costs. Grouping teams by their so-far performance allows for quick determination of the winner, allowing the league to accurately determine the best team in a short number of match days. Our probabilistic model confirms that if matchups are competitive, the winner should emerge quickly.

One of the biggest strengths of our program is that we take into account fairness and equity when choosing match hours. This means that a team will not be disadvantaged by being tired due to jet lag or staying away from home for extensive periods of time. Furthermore, our system ensures that the matches are at reasonable hours even for the fans of the away teams.

7.2 Limitations

Obiviously, each model has its underlying assumptions that deem it unsuitable for creating leagues in certain sports. The following are some of our model's limitations:

- **Sports with speicialized equipment:** In disciplines like sailing there is a lot of equipment that needs transporting between events, making it much harder logistically to hold such events regularly. In such cases limiting the travel distance is crucial and usually the constraint that requires getting back home has to be omitted.

- **Atmospheric conditions:** There are sports like cricket or baseball that are played in the open and yet require good atmospheric conditions. This means that scenarios where matches have to be postponed could occur, making the scheduling more complicated. Taking such cases into account in our model would be a hard to implement improvement.
- **Time Quantization**: We divided timespan of the whole season into week-long spaces. While this simplifies the solution and optimization, it is a considerable limitation for generalization to sports with more frequent games.
- **Limited Viewership**: Our model optimizes game times to ensure that teams are equally tired when playing each other. This combined with the fact that matches are played on one single day means that, when many matches are played in a single timezone, there is a high probability that they will be scheduled on one hour. This limits the possible viewership as fans can watch only one match at time.
- **Draws**: The model we have created does not take into account draws, which are a crucial aspect of many sports. Implementing an upgrade to the ELO system, where if two clubs draw and have completely different rankings, the team with the higher ranking loses points, while the other one gains them. This improvement is not required for some sports like volleyball, where after each match there has to be a clear winner.
- **History of games**: In the balanced K-means model some teams may play the same opponent in 2 consecutive rounds of clustering. This may be viewed as both an advantage and a disadvantage. This ensures competitive and exciting games between the top teams, but may lead to lower diversity of fixtures.

7.3 Conclusion

Our GSL scheduling framework is robust and ready for action beyond football. To get it working for another sport the following are required:

1. The right datasets for that sport (teams, historical games, locations, rules).
2. Tweaking the scoring rules for the ELO system based on the rules.
3. Tuning the parameters (K, lf, if, game frequency, streak limits, objective weights, clustering weight w).
4. Ensuring the sport fits the basic assumptions the model expects.

If the aforementioned points are applied, our model can produce fair, feasible and sustainable leagues combining clubs from all over the world in various disciplines.

Bibliography

Chess.com. *Elo rating system - chess terms*. Accessed April 3, 2025. https://www.chess.com/terms/elo-rating-chess.

———. *What is a Swiss tournament? — Chess.com Help Center*. Accessed April 3, 2025. https://support.chess.com/en/articles/8558054-what-is-a-swiss-tournament.

Dream Travel Points, May 2018. Accessed April 3, 2025. https://dreamtravelonpoints.com/the-cheapest-airline-in-the-world-per-km-is/.

Eastman, Charmane I., Clifford J Gazda, Helen J. Burgess, Stephanie J. Crowley, and Louis F. Fogg. "Advancing circadian rhythms before eastward flight: A strategy to prevent or reduce jet lag." *Sleep* 28, no. 1 (January 2005): 33–44. https://doi.org/10.1093/sleep/28.1.33.

August 2024. Accessed April 3, 2025. https://gauteng.news/2024/08/20/caf-boosts-prize-money-for-champions-league-and-confederation-cup/.

Global Football Rankings. *Football League Rankings: Analysis of Top Football Leagues*. Accessed April 3, 2025. https://globalfootballrankings.com/.

IATA. https://www.iata.org/contentassets/922ebc4cbcd24c4d9fd55933e7070947/icop_faq_general-for-airline-participants.pdf.

Independent, Carbon. Accessed April 3, 2025. https://www.carbonindependent.org/22.html.

International Olympic Committee. *Olympic agenda 2020 - strategic roadmap for the Olympic Movement*. Accessed April 3, 2025. https://www.olympics.com/ioc/olympic-agenda-2020.

Intracen. Accessed April 3, 2025. https://intracen.org/news-and-events/news/kick-for-trade-uefa-foundation-and-the-international-trade-centre-team-up-to.

Jaehne, Undine. *European Football Championship: This Is How Much co soccer fans produce*, February 2025. Accessed April 3, 2025. https://en.air-q.com/blog/fussball-em-2024-so-viel-co2-produzieren-fussballfans-im-stadion.

Johnson, Harry, Written by Harry Johnson, and Harry Johnson. *USA officially has most expensive five-star hotels in the world*, August 2021. Accessed April 3, 2025. https://eturbonews.com/usa-officially-has-most-expensive-five-star-hotels-in-the-world/?utm.

Sharma, Gulshan, By, and Gulshan Sharma. *AFC Champions League prize money 2025: How much the winner get?*, September 2024. Accessed April 3, 2025. https://sillyseason.com/football/afc-champions-league-prize-money-207867/?utm.

Singh, Aryan. *The most popular sports in the world*, February 2025. Accessed April 3, 2025. https://www.worldatlas.com/articles/what-are-the-most-popular-sports-in-the-world.html.

Sport Plus, August 2024. Accessed April 3, 2025. https://sportplus.co.za/caf-increases-prize-money-for-champions-league/?utm.

Watkins, Latham. *EPA releases updated, elevated estimates for the Social Cost of greenhouse gases*, December 2024. Accessed April 3, 2025. https://www.globalelr.com/2024/01/epa-releases-updated-elevated-estimates-for-the-social-cost-of-greenhouse-gases/.

Wikipedia, March 2025. Accessed April 3, 2025. https://en.wikipedia.org/wiki/List_of_association_football_stadiums_by_capacity.

———, January 2025. Accessed April 3, 2025. https://en.wikipedia.org/wiki/2006_FIFA_World_Cup?utm_source.

Wikipedia, March 2025. Accessed April 3, 2025.
https://en.wikipedia.org/wiki/2023%E2%80%9324_AFC_Cup?utm.

———, March 2025. Accessed April 3, 2025.
https://en.wikipedia.org/wiki/2024_Copa_Sudamericana.

———, February 2025. Accessed April 3, 2025.
https://en.wikipedia.org/wiki/2022_Caribbean_Club_Championship?utm.

Wright, Joe. *Is the Club World Cup 2025 prize money a record? how much winners and teams get, and "Solidarity Investment" explained*, March 2025. Accessed April 3, 2025.
https://www.sportingnews.com/us/soccer/news/club-world-cup-2025-prize-money-how-much-teams-win-fifa/f5cb6ecb2155e2a3d538e3aa.

8 Appendices

A Sport Suitability Scores

A.1 Sport variables for men

Sport	Number of fans (n) [$\times 10^6$]	Weekly frequency (f)	Standard deviation ($\sigma_{continents}$)	Number of different world champions (v)	FVI	MFI	SSS
Football	3500	1.5	7.96	5	5250	1.59	7.96
Basketball	2400	3	7.36	1	7200	7.36	6.89
Rugby	450	1	5.88	2	450	2.94	5.03
Cricket	2500	2.5	6.67	2	6250	3.33	7.54
Volleyball	900	3	8.71	4	2700	2.18	7.12

A.2 Sport variables for women

Sport	Number of fans (n) [$\times 10^6$]	Weekly frequency (f)	Standard deviation ($\sigma_{continents}$)	Number of different world champions (v)	FVI	MFI	SSS
Football	1750	1.5	9.15	4	2625	2.29	7.04
Basketball	1200	3	6.82	1	3600	6.82	6.27
Rugby	50	1	6.34	2	50	3.17	2.76
Cricket	500	2.5	6.10	2	1250	3.05	6.02
Volleyball	300	3	8.75	4	900	2.19	6.02

A.3 Code

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from bs4 import BeautifulSoup
import time
import csv
import numpy as np
import math
import matplotlib.pyplot as plt

# Navigate to the FIFA World Ranking page
```

```
driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
driver.get("https://inside.fifa.com/fifa-world-ranking/women")
time.sleep(5)

# Scroll through webpage
last_height = driver.execute_script("return document.body.scrollHeight")
while True:
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
    time.sleep(2)

    new_height = driver.execute_script("return document.body.scrollHeight")
    if new_height == last_height:
        break
    last_height = new_height

soup = BeautifulSoup(driver.page_source, 'html.parser')

#Locate the ranking table
ranking_table = soup.find('table', class_='table-module_table__76hUL fcds-table')

#Extract country rankings
rankings = []
for row in ranking_table.find_all('tr')[1:]:
    columns = row.find_all('td')
    if len(columns) >= 2:
        country = columns[1].text.strip()
        country = country[:-3]

        rankings.append((country))

#Quit the browser
driver.quit()

# Delete unranked countries
del rankings[-8:]

# Define a function to load information about countries and continents
def load_countries_continents(filename):
    countries_continents = {}
    with open(filename, mode='r') as file:
        reader = csv.DictReader(file)
        for row in reader:
            countries_continents[row['Country']] = row['Continent']
    return countries_continents

# Load the CSV data into the dictionary countries_continents
countries_continents = load_countries_continents('countries_continents.csv')

#Match countries to continents
updated_rankings = []
for country in rankings:
    continent = countries_continents.get(country, "Unknown") #Find the
   corresponding continent
    updated_rankings.append((country, continent))

# Assign new ranks to the clubs
for i in range(len(updated_rankings)):
    updated_rankings[i] = (i+1,) + updated_rankings[i]

# Create rankings for other sports
football_ranking = updated_rankings
basket_ranking = [[1, 'USA', 'North America'], [2, 'Australia', 'Oceania'], [3, '
   France', 'Europe'], [4, 'China', 'Asia'], [5, 'Spain', 'Europe'], [6, 'Belgium
```

```
    ', 'Europe'], [7, 'Canada', 'North America'], [8, 'Serbia', 'Europe'], [9, '
    Japan', 'Asia'], [10, 'Brazil', 'South America'], [11, 'Nigeria', 'Africa'],
    [12, 'Puerto Rico', 'North America'], [13, 'Germany', 'Europe'], [14, 'Korea',
     'Asia'], [15, 'Hungary', 'Europe'], [16, 'Italy', 'Europe'], [17, 'T  rkiye
    ', 'Asia'], [18, 'Czechia', 'Europe'], [19, 'Montenegro', 'Europe'], [20, '
    Great Britain', 'Europe'], [21, 'Greece', 'Europe'], [22, 'Slovenia', 'Europe'
    ], [23, 'Bosnia and Herzegovina', 'Europe'], [24, 'Mali', 'Africa'], [25, '
    Sweden', 'Europe'], [26, 'New Zealand', 'Oceania'], [27, 'Senegal', 'Africa'],
     [28, 'Slovakia', 'Europe'], [29, 'Latvia', 'Europe'], [30, 'Colombia', 'South
     America'], [31, 'Argentina', 'South America'], [32, 'Mozambique', 'Africa'],
    [33, 'Croatia', 'Europe'], [34, 'Dominican Republic', 'North America'], [35, '
    Ukraine', 'Europe'], [36, 'Venezuela', 'South America'], [37, 'Egypt', 'Africa
    '], [38, 'Mexico', 'North America'], [39, 'Chinese Taipei', 'Asia'], [40, '
    Portugal', 'Europe'], [41, 'Cuba', 'North America'], [42, 'Cameroon', 'Africa'
    ], [43, 'Poland', 'Europe'], [44, 'Philippines', 'Asia'], [45, 'Lithuania', '
    Europe'], [46, 'Angola', 'Africa'], [47, 'Israel', 'Asia'], [48, 'Virgin
    Islands', 'North America'], [49, 'Switzerland', 'Europe'], [50, 'Luxembourg',
    'Europe'], [51, 'Netherlands', 'Europe'], [52, 'Uganda', 'Africa'], [53, "
     C   te d'Ivoire", 'Africa'], [54, 'Lebanon', 'Asia'], [55, 'Denmark', 'Europe
    '], [56, 'El Salvador', 'North America'], [57, 'Indonesia', 'Asia'], [58, '
    Bulgaria', 'Europe'], [59, 'Iran', 'Asia'], [60, 'Romania', 'Europe'], [61, '
    Finland', 'Europe'], [62, 'Iceland', 'Europe'], [63, 'Paraguay', 'South
    America'], [64, 'Jordan', 'Asia'], [65, 'Rwanda', 'Africa'], [66, 'Thailand',
    'Asia'], [67, 'Norway', 'Europe'], [68, 'Kenya', 'Africa'], [69, 'Estonia', '
    Europe'], [70, 'Chile', 'South America'], [71, 'Costa Rica', 'North America'],
     [72, 'Guatemala', 'North America'], [73, 'Malaysia', 'Asia'], [74, 'North
    Macedonia', 'Europe'], [75, 'Congo DR', 'Africa'], [76, 'Malta', 'Europe'],
    [77, 'Kazakhstan', 'Asia'], [78, 'Cyprus', 'Europe'], [79, 'India', 'Asia'],
    [80, 'Austria', 'Europe'], [81, 'Ireland', 'Europe'], [82, 'Kosovo', 'Europe'
    ], [83, 'Bahamas', 'North America'], [84, 'Syria', 'Asia'], [85, 'Ecuador', '
    South America'], [86, 'South Sudan', 'Africa'], [87, 'Armenia', 'Europe'],
    [88, 'Albania', 'Europe'], [89, 'Bolivia', 'South America'], [90, 'Mongolia',
    'Asia'], [91, 'Guinea', 'Africa'], [92, 'Barbados', 'North America'], [93, '
    Cape Verde', 'Africa'], [94, 'Uzbekistan', 'Asia'], [95, 'Singapore', 'Asia'],
     [96, 'Uruguay', 'South America'], [97, 'Panama', 'North America'], [98, '
    Zimbabwe', 'Africa'], [99, 'Tunisia', 'Africa'], [100, 'Honduras', 'North
    America'], [101, 'Jamaica', 'North America'], [102, 'St.Vincent and the
    Grenadines', 'South America'], [103, 'Azerbaijan', 'Europe'], [104, 'Andorra',
     'Europe'], [105, 'Korea DPR', 'Asia'], [106, 'Moldova', 'Europe'], [107, '
    Suriname', 'South America'], [108, 'Nicaragua', 'North America'], [109, '
    Central African Rep.', 'Africa'], [110, 'Fiji', 'Oceania'], [111, 'Gibraltar',
     'Europe'], [112, 'Zambia', 'Africa'], [113, 'Benin', 'Africa'], [114, 'Sri
    Lanka', 'Asia'], [115, 'Burundi', 'Africa'], [116, 'Gabon', 'Africa'], [117, '
    Guyana', 'South America'], [118, 'Georgia', 'Europe']]
rugby_ranking = [[1, 'England', 'Europe'], [2, 'Canada', 'North America'], [3, '
    New Zealand', 'Oceania'], [4, 'France', 'Europe'], [5, 'Ireland', 'Europe'],
    [6, 'Australia', 'Oceania'], [7, 'Scotland', 'Europe'], [8, 'USA', 'North
    America'], [9, 'Italy', 'Europe'], [10, 'Wales', 'Europe'], [11, 'Japan', '
    Asia'], [12, 'South Africa', 'Africa'], [13, 'Spain', 'Europe'], [14, 'Samoa',
     'Oceania'], [15, 'Netherlands', 'Europe'], [16, 'Fiji', 'Oceania'], [17, '
    Hong Kong China', 'Asia'], [18, 'Kazakhstan', 'Asia'], [19, 'Russia', 'Europe'
    ], [20, 'Sweden', 'Europe'], [21, 'Germany', 'Europe'], [22, 'Kenya', 'Africa'
    ], [23, 'Belgium', 'Europe'], [24, 'Madagascar', 'Africa'], [25, 'Portugal', '
    Europe'], [26, 'Tonga', 'Oceania'], [27, 'China', 'Asia'], [28, 'Trinidad &
    Tobago', 'South America'], [29, 'Colombia', 'South America'], [30, 'Mexico', '
    North America'], [31, 'Finland', 'Europe'], [32, 'Tunisia', 'Africa'], [33, '
    Croatia', 'Europe'], [34, 'Brazil', 'South America'], [35, 'Andorra', 'Europe'
    ], [36, 'Cameroon', 'Africa'], [37, 'Romania', 'Europe'], [38, 'Jamaica', '
    North America'], [39, 'Uganda', 'Africa'], [40, 'Czechia', 'Europe'], [41, '
    Latvia', 'Europe'], [42, 'Norway', 'Europe'], [43, 'Bulgaria', 'Europe'], [44,
     'Papua New Guinea', 'Oceania'], [45, 'Denmark', 'Europe'], [46, 'Singapore',
    'Asia'], [47, 'Zambia', 'Africa'], [48, 'Guyana', 'South America'], [49, '
    Senegal', 'Africa'], [50, "Cote D'Ivoire", 'Africa'], [51, 'Bosnia &
```

```
    Herzegovina', 'Europe'], [52, 'India', 'Asia'], [53, 'Zimbabwe', 'Africa'],
    [54, 'Thailand', 'Asia'], [55, 'Switzerland', 'Europe'], [56, 'Uzbekistan', '
    Asia'], [57, 'Burkina Faso', 'Africa'], [58, 'Cayman Islands', 'North America'
    ], [59, 'Botswana', 'Africa'], [60, 'Philippines', 'Asia'], [61, 'Luxembourg',
     'Europe'], [62, 'Namibia', 'Africa'], [63, 'Serbia', 'Europe'], [64, '
    Barbados', 'North America'], [65, 'St Vincent and the Grenadines', 'North
    America'], [66, 'Bahamas', 'North America']]
74 cricket_ranking = [[1, 'Australia', 'Oceania'], [2, 'England', 'Europe'], [3, '
    India', 'Asia'], [4, 'New Zealand', 'Oceania'], [5, 'South Africa', 'Africa'],
     [6, 'West Indies', 'North America'], [7, 'Sri Lanka', 'Asia'], [8, 'Pakistan'
    , 'Asia'], [9, 'Bangladesh', 'Asia'], [10, 'Ireland', 'Europe'], [11, '
    Scotland', 'Europe'], [12, 'PNG', 'Asia'], [13, 'Thailand', 'Asia'], [14, '
    Zimbabwe', 'Africa'], [15, 'UAE', 'Asia'], [16, 'Netherlands', 'Europe'], [17,
     'Namibia', 'Africa'], [18, 'Uganda', 'Africa'], [19, 'Tanzania', 'Africa'],
    [20, 'Indonesia', 'Asia'], [21, 'Nepal', 'Asia'], [22, 'Hong Kong', 'Asia'],
    [23, 'Italy', 'Europe'], [24, 'Spain', 'Europe'], [25, 'Nigeria', 'Africa'],
    [26, 'USA', 'North America'], [27, 'Rwanda', 'Africa'], [28, 'Malaysia', 'Asia
    '], [29, 'Kenya', 'Africa'], [30, 'Cyprus', 'Europe'], [31, 'Jersey', 'Europe'
    ], [32, 'Vanuatu', 'Oceania'], [33, 'Canada', 'North America'], [34, '
    Gibraltar', 'Europe'], [35, 'Isle of Man', 'Europe'], [36, 'Myanmar', 'Asia'],
     [37, 'Brazil', 'South America'], [38, 'Croatia', 'Europe'], [39, 'Sweden', '
    Europe'], [40, 'Greece', 'Europe'], [41, 'Denmark', 'Europe'], [42, 'Guernsey'
    , 'Europe'], [43, 'Germany', 'Europe'], [44, 'Samoa', 'Oceania'], [45, 'Japan'
    , 'Asia'], [46, 'Botswana', 'Africa'], [47, 'Sierra Leone', 'Africa'], [48, '
    China', 'Asia'], [49, 'Kuwait', 'Asia'], [50, 'France', 'Europe'], [51, '
    Mozambique', 'Africa'], [52, 'Mexico', 'North America'], [53, 'Bhutan', 'Asia'
    ], [54, 'Qatar', 'Asia'], [55, 'Romania', 'Europe'], [56, 'Serbia', 'Europe'],
     [57, 'Argentina', 'South America'], [58, 'Malawi', 'Africa'], [59, 'Austria',
     'Europe'], [60, 'Cook Islands', 'Oceania'], [61, 'Fiji', 'Oceania'], [62, '
    Luxembourg', 'Europe'], [63, 'Norway', 'Europe'], [64, 'Philippines', 'Asia'],
     [65, 'Singapore', 'Asia'], [66, 'Cambodia', 'Asia'], [67, 'Malta', 'Europe'],
     [68, 'Oman', 'Asia'], [69, 'Mongolia', 'Asia'], [70, 'Eswatini', 'Africa'],
    [71, 'Lesotho', 'Africa'], [72, 'Ghana', 'Africa'], [73, 'Estonia', 'Europe'],
     [74, 'Cameroon', 'Africa'], [75, 'Bulgaria', 'Europe'], [76, 'Bahrain', 'Asia
    '], [77, 'Czechia', 'Europe']]
75 volleyball_ranking = [[1, 'Italy', 'Europe'], [2, 'Brazil', 'South America'], [3,
     'USA', 'North America'], [4, 'T  rkiye', 'Asia'], [5, 'China', 'Asia'], [6,
     'Poland', 'Europe'], [7, 'Japan', 'Asia'], [8, 'Canada', 'North America'],
    [9, 'Netherlands', 'Europe'], [10, 'Serbia', 'Europe'], [11, 'Dominican
    Republic', 'North America'], [12, 'Germany', 'Europe'], [13, 'Thailand', 'Asia
    '], [14, 'Belgium', 'Europe'], [15, 'Czechia', 'Europe'], [16, 'Puerto Rico',
    'North America'], [17, 'Argentina', 'South America'], [18, 'Ukraine', 'Europe'
    ], [19, 'France', 'Europe'], [20, 'Bulgaria', 'Europe'], [21, 'Colombia', '
    South America'], [22, 'Kenya', 'Africa'], [23, 'Cuba', 'North America'], [24,
    'Sweden', 'Europe'], [25, 'Mexico', 'North America'], [26, 'Slovenia', 'Europe
    '], [27, 'Slovakia', 'Europe'], [28, 'Spain', 'Europe'], [29, 'Greece', '
    Europe'], [30, 'Romania', 'Europe'], [31, 'Kazakhstan', 'Asia'], [32, 'Vietnam
    ', 'Asia'], [33, 'Croatia', 'Europe'], [34, 'Azerbaijan', 'Europe'], [35, '
    Korea', 'Asia'], [36, 'Hungary', 'Europe'], [37, 'Portugal', 'Europe'], [38, '
    Austria', 'Europe'], [39, 'Bosnia and Herzegovina', 'Europe'], [40, 'Peru', '
    South America'], [41, 'Finland', 'Europe'], [42, 'Montenegro', 'Europe'], [43,
     'Switzerland', 'Europe'], [44, 'Cameroon', 'Africa'], [45, 'Chinese Taipei',
    'Asia'], [46, 'Latvia', 'Europe'], [47, 'Estonia', 'Europe'], [48, 'Egypt', '
    Africa'], [49, 'Costa Rica', 'North America'], [50, 'Suriname', 'South America
    '], [51, 'Chile', 'South America'], [52, 'Philippines', 'Asia'], [53, 'India',
     'Asia'], [54, 'Indonesia', 'Asia'], [55, 'Australia', 'Oceania'], [56, '
    Israel', 'Asia'], [57, 'Iran', 'Asia'], [58, 'Singapore', 'Asia'], [59, '
    Georgia', 'Europe'], [60, 'Hong Kong, China', 'Asia'], [61, 'Kosovo', 'Europe'
    ], [62, 'North Macedonia', 'Europe'], [63, 'Luxembourg', 'Europe'], [64, '
    Algeria', 'Africa'], [65, 'Faroe Islands', 'Europe'], [66, 'Iceland', 'Europe'
    ], [67, 'Denmark', 'Europe'], [68, 'New Zealand', 'Oceania']]
76
77 def choosing(rankings, fanbase, frequency, win_variance):
```

```
    chosen = []

    # Select the top 50 teams
    for i in rankings:
        if len(chosen) != 50 and i not in chosen:
            chosen.append(i)

    chosen = sorted(chosen, key=lambda x: x[0])
    print(chosen)

    #Create the rankings list
    rankings = []
    for i in chosen:
        rankings.append(i[0])

    # Count number of teams from each continent
    frequency_table = [0, 0, 0, 0, 0, 0]
    for i in chosen:
        if i[2] == "Europe":
            frequency_table[0] += 1
        if i[2] == "Asia":
            frequency_table[1] += 1
        if i[2] == "North America":
            frequency_table[2] += 1
        if i[2] == "South America":
            frequency_table[3] += 1
        if i[2] == "Oceania":
            frequency_table[4] += 1
        if i[2] == "Africa":
            frequency_table[5] += 1

    # Calculate standard deviation
    std = np.std(rankings)
    std_continent = np.std(frequency_table)

    print(std_continent)

    # Calculate final indexes
    fvi_index = fanbase * frequency
    mfi_index =  std_continent / win_variance
    score = fvi_index/mfi_index
    log_score = math.log(fvi_index/mfi_index)

    return fvi_index, mfi_index, score, log_score

#Run the function for different sports
print('Football')
fvi_football, mfi_football, score_football, log_score_football = choosing(
    football_ranking, 1750, 1.5, 4)
print(fvi_football, mfi_football, score_football)
print('Basketball')
fvi_basket, mfi_basket, score_basket, log_score_basket = choosing(basket_ranking,
    1200, 3, 1)
print(fvi_basket, mfi_basket, score_basket)
print('Rugby')
fvi_rugby, mfi_rugby, score_rugby, log_score_rugby = choosing(rugby_ranking, 50,
    1, 2)
print(fvi_rugby, mfi_rugby, score_rugby)
print('Cricket')
fvi_cricket, mfi_cricket, score_cricket, log_score_cricket = choosing(
    cricket_ranking, 500, 2.5, 2)
print(fvi_cricket, mfi_cricket, score_cricket)
print('Volleyball')
```

```
fvi_volleyball, mfi_volleyball, score_volleyball, log_score_volleyball = choosing(volleyball_ranking, 300, 3, 4)
print(fvi_volleyball, mfi_volleyball, score_volleyball)

#Plot the scores on a bar chart
import matplotlib.pyplot as plt
scores = [float(score_football), float(score_basket), float(score_rugby), float(score_cricket), float(score_volleyball)]
log_scores = [log_score_football, log_score_basket, log_score_rugby, log_score_cricket, log_score_volleyball]
x = np.arange(5)
width = 0.25
fig, ax = plt.subplots()
bars = ax.bar(x + width, scores, width, label='Score')

# Add labels to the bars
def add_value_labels(bars):
    for bar in bars:
        height = bar.get_height()
        ax.annotate('{}'.format(round(height, 2)),
                    xy=(bar.get_x() + bar.get_width() / 2, height),
                    xytext=(0, 3),
                    textcoords="offset points",
                    ha='center', va='bottom')
add_value_labels(bars)
plt.tight_layout()
plt.show()

# Print final scores
print(scores)
print(log_scores)
log_scores_women = log_scores

#Navigate to the FIFA World Ranking page
driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
driver.get("https://inside.fifa.com/fifa-world-ranking/women")
time.sleep(5)

#Parsing the page source
soup = BeautifulSoup(driver.page_source, 'html.parser')

print(soup.prettify())

driver.quit()

# Delete unranked country
del rankings[-1]

# Define a function to load information about countries and continents
def load_countries_continents(filename):
    countries_continents = {}
    with open(filename, mode='r') as file:
        reader = csv.DictReader(file)
        for row in reader:
            countries_continents[row['Country']] = row['Continent']
    return countries_continents

# Load the CSV data into the dictionary countries_continents
countries_continents = load_countries_continents('countries_continents.csv')

#Match countries to continents
updated_rankings = []
for country in rankings:
```

```
    continent = countries_continents.get(country, "Unknown") #Find the corresponding continent
    updated_rankings.append((country, continent))


# Assign new ranks to the clubs
for i in range(len(updated_rankings)):
    updated_rankings[i] = (i+1,) + updated_rankings[i]

# Create rankings for other sports
football_ranking = updated_rankings
basket_ranking = [[1, 'USA', 'North America'], [2, 'Serbia', 'Europe'], [3, 'Germany', 'Europe'], [4, 'France', 'Europe'], [5, 'Spain', 'Europe'], [6, 'Canada', 'North America'], [7, 'Australia', 'Oceania'], [8, 'Argentina', 'South America'], [9, 'Latvia', 'Europe'], [10, 'Lithuania', 'Europe'], [11, 'Slovenia', 'Europe'], [12, 'Brazil', 'South America'], [13, 'Greece', 'Europe'], [14, 'Italy', 'Europe'], [15, 'Puerto Rico', 'North America'], [16, 'Montenegro', 'Europe'], [17, 'Poland', 'Europe'], [18, 'Dominican Republic', 'North America'], [19, 'Czechia', 'Europe'], [20, 'Finland', 'Europe'], [21, 'Japan', 'Asia'], [22, 'New Zealand', 'Oceania'], [23, 'South Sudan', 'Africa'], [24, 'Georgia', 'Europe'], [25, 'Venezuela', 'South America'], [26, 'Mexico', 'North America'], [27, 'T  rkiye', 'Asia'], [28, 'Iran', 'Asia'], [29, 'Lebanon', 'Asia'], [30, 'China', 'Asia'], [31, "C  te d'Ivoire", 'Africa'], [32, 'Croatia', 'Europe'], [33, 'Angola', 'Africa'], [34, 'Philippines', 'Asia'], [35, 'Jordan', 'Asia'], [36, 'Tunisia', 'Africa'], [37, 'Ukraine', 'Europe'], [38, 'Egypt', 'Africa'], [39, 'Israel', 'Asia'], [40, 'Belgium', 'Europe'], [41, 'Bosnia and Herzegovina', 'Europe'], [42, 'Nigeria', 'Africa'], [43, 'Estonia', 'Europe'], [44, 'Hungary', 'Europe'], [45, 'Cape Verde', 'Africa'], [46, 'Bulgaria', 'Europe'], [47, 'Senegal', 'Africa'], [48, 'Great Britain', 'Europe'], [49, 'Sweden', 'Europe'], [50, 'Iceland', 'Europe'], [51, 'Uruguay', 'South America'], [52, 'Bahamas', 'North America'], [53, 'Korea', 'Asia'], [54, 'Netherlands', 'Europe'], [55, 'Colombia', 'South America'], [56, 'Portugal', 'Europe'], [57, 'Panama', 'North America'], [58, 'North Macedonia', 'Europe'], [59, 'Denmark', 'Europe'], [60, 'Slovakia', 'Europe'], [61, 'Chile', 'South America'], [62, 'Switzerland', 'Europe'], [63, 'Romania', 'Europe'], [64, 'Cameroon', 'Africa'], [65, 'Saudi Arabia', 'Asia'], [66, 'Bahrain', 'Asia'], [67, 'Austria', 'Europe'], [68, 'Cuba', 'North America'], [69, 'Virgin Islands', 'North America'], [70, 'Kazakhstan', 'Asia'], [71, 'Syria', 'Asia'], [72, 'Congo DR', 'Africa'], [73, 'Chinese Taipei', 'Asia'], [74, 'Costa Rica', 'North America'], [75, 'Guinea', 'Africa'], [76, 'India', 'Asia'], [77, 'Kosovo', 'Europe'], [78, 'Norway', 'Europe'], [79, 'Indonesia', 'Asia'], [80, 'Nicaragua', 'North America'], [81, 'Ireland', 'Europe'], [82, 'Paraguay', 'South America'], [83, 'Mali', 'Africa'], [84, 'Cyprus', 'Europe'], [85, 'Uganda', 'Africa'], [86, 'Luxembourg', 'Europe'], [87, 'Qatar', 'Asia'], [88, 'Guam', 'Oceania'], [89, 'Central African Rep.', 'Africa'], [90, 'Thailand', 'Asia'], [91, 'Armenia', 'Europe'], [92, 'Iraq', 'Asia'], [93, 'Rwanda', 'Africa'], [94, 'Libya', 'Africa'], [95, 'Albania', 'Europe'], [96, 'Andorra', 'Europe'], [97, 'Kenya', 'Africa'], [98, 'Palestine', 'Asia'], [99, 'Malta', 'Europe'], [100, 'Morocco', 'Africa'], [101, 'Mongolia', 'Asia'], [102, 'Ecuador', 'South America'], [103, 'UAE', 'Asia'], [104, 'El Salvador', 'North America'], [105, 'Madagascar', 'Africa'], [106, 'Bolivia', 'South America'], [107, 'Jamaica', 'North America'], [108, 'Malaysia', 'Asia'], [109, 'Barbados', 'North America'], [110, 'Azerbaijan', 'Europe'], [111, 'Hong Kong, China', 'Asia'], [112, 'Kuwait', 'Asia'], [113, 'San Marino', 'Europe'], [114, 'Burundi', 'Africa'], [115, 'Mozambique', 'Africa'], [116, 'Honduras', 'North America'], [117, 'Chad', 'Africa'], [118, 'St.Vincent and the Grenadines', 'South America'], [119, 'Guyana', 'South America'], [120, 'Antigua & Barbuda', 'South America'], [121, 'Algeria', 'Africa'], [122, 'Belize', 'North America'], [123, 'Grenada', 'North America'], [124, 'Gibraltar', 'Europe'], [125, 'Samoa', 'Oceania'], [126, 'Guatemala', 'North America'], [127, 'South Africa', 'Africa'], [128, 'Suriname', 'South America'], [129, 'Singapore', 'Asia'], [130, 'Bermuda', 'North America'], [131, 'Gabon', 'Africa'], [132, 'Sri Lanka', 'Asia'], [133, 'Montserrat', 'North America'], [134, 'Maldives', 'Asia'],
```

```
    [135, 'Vietnam', 'Asia'], [136, 'Somalia', 'Africa'], [137, 'Tahiti', 'Oceania
    '], [138, 'Equatorial Guinea', 'Africa'], [139, 'Tonga', 'Oceania'], [140, '
    Bangladesh', 'Asia'], [141, 'Zambia', 'Africa'], [142, 'American Samoa', '
    Oceania'], [143, 'Zimbabwe', 'Africa'], [144, 'Bhutan', 'Asia'], [145, 'Brunei
     Darussalam', 'Asia'], [146, 'Tanzania', 'Africa'], [147, 'Oman', 'Asia'],
    [148, 'Macau', 'Asia'], [149, 'St. Lucia', 'South America'], [150, 'Nepal', '
    Asia'], [151, 'Moldova', 'Europe'], [152, 'Fiji', 'Oceania'], [153, 'Cambodia'
    , 'Asia'], [154, 'Myanmar', 'Asia'], [155, 'Eritrea', 'Africa'], [156, 'Cook
    Islands', 'Oceania'], [157, 'Haiti', 'North America']]
208 rugby_ranking = [[1, 'South Africa', 'Africa'], [2, 'New Zealand', 'Oceania'],
    [3, 'Ireland', 'Europe'], [4, 'France', 'Europe'], [5, 'Argentina', 'South
    America'], [6, 'England', 'Europe'], [7, 'Scotland', 'Europe'], [8, 'Australia
    ', 'Oceania'], [9, 'Fiji', 'Oceania'], [10, 'Italy', 'Europe'], [11, 'Georgia'
    , 'Europe'], [12, 'Wales', 'Europe'], [13, 'Japan', 'Asia'], [14, 'Samoa', '
    Oceania'], [15, 'USA', 'North America'], [16, 'Spain', 'Europe'], [17, '
    Uruguay', 'South America'], [18, 'Portugal', 'Europe'], [19, 'Tonga', 'Oceania
    '], [20, 'Romania', 'Europe'], [21, 'Chile', 'South America'], [22, 'Belgium',
     'Europe'], [23, 'Hong Kong China', 'Asia'], [24, 'Canada', 'North America'],
    [25, 'Namibia', 'Africa'], [26, 'Zimbabwe', 'Africa'], [27, 'Netherlands', '
    Europe'], [28, 'Brazil', 'South America'], [29, 'Switzerland', 'Europe'], [30,
     'Sweden', 'Europe'], [31, 'Russia', 'Europe'], [32, 'Poland', 'Europe'], [33,
     'Kenya', 'Africa'], [34, 'Czechia', 'Europe'], [35, 'Korea', 'Asia'], [36, '
    Germany', 'Europe'], [37, 'Paraguay', 'South America'], [38, 'Morocco', '
    Africa'], [39, 'Mexico', 'North America'], [40, 'Sri Lanka', 'Asia'], [41, '
    Uganda', 'Africa'], [42, 'Malta', 'Europe'], [43, 'Croatia', 'Europe'], [44, '
    Colombia', 'South America'], [45, 'Trinidad & Tobago', 'South America'], [46,
    'Madagascar', 'Africa'], [47, 'Tunisia', 'Africa'], [48, 'United Arab Emirates
    ', 'Asia'], [49, 'Malaysia', 'Asia'], [50, 'Ukraine', 'Europe'], [51, 'Cayman
    Islands', 'North America'], [52, 'Senegal', 'Africa'], [53, 'Moldova', 'Europe
    '], [54, 'Lithuania', 'Europe'], [55, 'Luxembourg', 'Europe'], [56, 'Latvia',
    'Europe'], [57, 'Algeria', 'Africa'], [58, 'Guyana', 'South America'], [59, '
    Singapore', 'Asia'], [60, 'Philippines', 'Asia'], [61, "Cote D'Ivoire", '
    Africa'], [62, 'Kazakhstan', 'Asia'], [63, 'Chinese Taipei', 'Asia'], [64, '
    Jamaica', 'North America'], [65, 'Israel', 'Asia'], [66, 'Zambia', 'Africa'],
    [67, 'Cook Islands', 'Oceania'], [68, 'Bulgaria', 'Europe'], [69, 'Austria', '
    Europe'], [70, 'Bermuda', 'North America'], [71, 'Denmark', 'Europe'], [72, '
    Barbados', 'North America'], [73, 'Thailand', 'Asia'], [74, 'Nigeria', 'Africa
    '], [75, 'Venezuela', 'South America'], [76, 'Botswana', 'Africa'], [77, '
    Finland', 'Europe'], [78, 'Hungary', 'Europe'], [79, 'Peru', 'South America'],
     [80, 'Slovenia', 'Europe'], [81, 'Turkey', 'Europe'], [82, 'Ghana', 'Africa'
    ], [83, 'Andorra', 'Europe'], [84, 'St Vincent and the Grenadines', 'North
    America'], [85, 'Serbia', 'Europe'], [86, 'India', 'Asia'], [87, 'Qatar', '
    Asia'], [88, 'Burkina Faso', 'Africa'], [89, 'Guam', 'Oceania'], [90, '
    Pakistan', 'Asia'], [91, 'Iran', 'Asia'], [92, 'Costa Rica', 'North America'],
     [93, 'St Lucia', 'North America'], [94, 'China', 'Asia'], [95, 'Bosnia &
    Herzegovina', 'Europe'], [96, 'Papua New Guinea', 'Oceania'], [97, 'Bahamas',
    'North America'], [98, 'Eswatini', 'Africa'], [99, 'Uzbekistan', 'Asia'],
    [100, 'Norway', 'Europe'], [101, 'Mauritius', 'Africa'], [102, 'Laos', 'Asia'
    ], [103, 'Nepal', 'Asia'], [104, 'Rwanda', 'Africa'], [105, 'Cameroon', '
    Africa'], [106, 'Niue Island', 'Oceania'], [107, 'Burundi', 'Africa'], [108, '
    Solomon Islands', 'Oceania'], [109, 'Indonesia', 'Asia'], [110, 'Monaco', '
    Europe'], [111, 'Greece', 'Europe'], [112, 'Vanuatu', 'Oceania'], [113, '
    American Samoa', 'Oceania']]
209 cricket_ranking = [[1, 'India', 'Asia'], [2, 'Australia', 'Oceania'], [3, '
    England', 'Europe'], [4, 'New Zealand', 'Oceania'], [5, 'West Indies', 'North
    America'], [6, 'South Africa', 'Africa'], [7, 'Pakistan', 'Asia'], [8, 'Sri
    Lanka', 'Asia'], [9, 'Bangladesh', 'Asia'], [10, 'Afghanistan', 'Asia'], [11,
    'Zimbabwe', 'Africa'], [12, 'Ireland', 'Europe'], [13, 'Scotland', 'Europe'],
    [14, 'Netherlands', 'Europe'], [15, 'Namibia', 'Africa'], [16, 'United Arab
    Emirates', 'Asia'], [17, 'Nepal', 'Asia'], [18, 'United States of America', '
    North Amercia'], [19, 'Oman', 'Asia'], [20, 'Papua New Guinea', 'Oceania'],
    [21, 'Canada', 'North America'], [22, 'Uganda', 'Africa'], [23, 'Hong Kong', '
    Asia'], [24, 'Kuwait', 'Asia'], [25, 'Bahrain', 'Asia'], [26, 'Malaysia', '
```

```
    Asia'], [27, 'Jersey', 'Europe'], [28, 'Qatar', 'Asia'], [29, 'Bermuda', '
    North America'], [30, 'Spain', 'Europe'], [31, 'Italy', 'Europe'], [32, 'Saudi
     Arabia', 'Asia'], [33, 'Kenya', 'Africa'], [34, 'Tanzania', 'Africa'], [35, '
    Germany', 'Europe'], [36, 'Nigeria', 'Africa'], [37, 'Guernsey', 'Europe'],
    [38, 'Singapore', 'Asia'], [39, 'Norway', 'Europe'], [40, 'Cayman Islands', '
    North America'], [41, 'Denmark', 'Europe'], [42, 'Portugal', 'Europe'], [43, '
    Cambodia', 'Asia'], [44, 'Isle of Man', 'Europe'], [45, 'Japan', 'Asia'], [46,
     'Belgium', 'Europe'], [47, 'Austria', 'Europe'], [48, 'France', 'Europe'],
    [49, 'Switzerland', 'Europe'], [50, 'Botswana', 'Africa'], [51, 'Argentina', '
    South America'], [52, 'Romania', 'Europe'], [53, 'Cook Islands', 'Oceania'],
    [54, 'Malawi', 'Africa'], [55, 'Thailand', 'Asia'], [56, 'Sweden', 'Europe'],
    [57, 'Finland', 'Europe'], [58, 'Bahamas', 'North America'], [59, 'Vanuatu', '
    Oceania'], [60, 'Czech Republic', 'Europe'], [61, 'Philippines', 'Asia'], [62,
     'Mozambique', 'Africa'], [63, 'Rwanda', 'Africa'], [64, 'Indonesia', 'Asia'],
     [65, 'Ghana', 'Africa'], [66, 'Estonia', 'Europe'], [67, 'Samoa', 'Oceania'],
     [68, 'Fiji', 'Oceania'], [69, 'Hungary', 'Europe'], [70, 'Sierra Leone', '
    Africa'], [71, 'Luxembourg', 'Europe'], [72, 'Costa Rica', 'North America'],
    [73, 'Israel', 'Asia'], [74, 'Malta', 'Europe'], [75, 'Belize', 'North America
    '], [76, 'Gibraltar', 'Europe'], [77, 'Slovenia', 'Europe'], [78, 'Cyprus', '
    Europe'], [79, 'Suriname', 'South America'], [80, 'Brazil', 'South America'],
    [81, 'Bhutan', 'Asia'], [82, 'Panama', 'North America'], [83, 'Mexico', 'North
     America'], [84, 'Croatia', 'Europe'], [85, 'Eswatini', 'Africa'], [86, '
    Maldives', 'Asia'], [87, 'St Helena', 'Africa'], [88, 'Cameroon', 'Africa'],
    [89, 'China', 'Asia'], [90, 'Bulgaria', 'Europe'], [91, 'Serbia', 'Europe'],
    [92, 'Lesotho', 'Africa'], [93, 'Mongolia', 'Asia'], [94, 'Turkey', 'Europe'],
     [95, 'South Korea', 'Asia'], [96, 'Seychelles', 'Africa'], [97, 'Myanmar', '
    Asia'], [98, 'Mali', 'Africa'], [99, 'Greece', 'Europe'], [100, 'Gambia', '
    Africa']]
volleyball_ranking = [[1, 'Poland', 'Europe'], [2, 'France', 'Europe'], [3, 'USA'
    , 'North America'], [4, 'Slovenia', 'Europe'], [5, 'Italy', 'Europe'], [6, '
    Japan', 'Asia'], [7, 'Brazil', 'South America'], [8, 'Germany', 'Europe'], [9,
     'Argentina', 'South America'], [10, 'Serbia', 'Europe'], [11, 'Canada', '
    North America'], [12, 'Cuba', 'North America'], [13, 'Netherlands', 'Europe'],
     [14, 'Ukraine', 'Europe'], [15, 'Iran', 'Asia'], [16, 'Belgium', 'Europe'],
    [17, 'T  rkiye', 'Asia'], [18, 'Czechia', 'Europe'], [19, 'Bulgaria', '
    Europe'], [20, 'Egypt', 'Africa'], [21, 'Qatar', 'Asia'], [22, 'Portugal', '
    Europe'], [23, 'Finland', 'Europe'], [24, 'China', 'Asia'], [25, 'Romania', '
    Europe'], [26, 'Chile', 'South America'], [27, 'Korea', 'Asia'], [28, 'Croatia
    ', 'Europe'], [29, 'Puerto Rico', 'North America'], [30, 'Dominican Republic',
     'North America'], [31, 'Greece', 'Europe'], [32, 'Spain', 'Europe'], [33, '
    Australia', 'Oceania'], [34, 'Mexico', 'North America'], [35, 'Latvia', '
    Europe'], [36, 'Estonia', 'Europe'], [37, 'Slovakia', 'Europe'], [38, '
    Switzerland', 'Europe'], [39, 'North Macedonia', 'Europe'], [40, 'Tunisia', '
    Africa'], [41, 'Denmark', 'Europe'], [42, 'Montenegro', 'Europe'], [43, '
    Colombia', 'South America'], [44, 'Austria', 'Europe'], [45, 'Chinese Taipei',
     'Asia'], [46, 'Pakistan', 'Asia'], [47, 'Cameroon', 'Africa'], [48, 'Hungary'
    , 'Europe'], [49, 'Norway', 'Europe'], [50, 'Israel', 'Asia'], [51, 'Indonesia
    ', 'Asia'], [52, 'Vietnam', 'Asia'], [53, 'Kazakhstan', 'Asia'], [54, 'Sweden'
    , 'Europe'], [55, 'Kosovo', 'Europe'], [56, 'Philippines', 'Asia'], [57, '
    Thailand', 'Asia'], [58, 'Bahrain', 'Asia'], [59, 'Libya', 'Africa'], [60, '
    Peru', 'South America'], [61, 'Georgia', 'Europe'], [62, 'Faroe Islands', '
    Europe'], [63, 'Luxembourg', 'Europe'], [64, 'Iceland', 'Europe'], [65, '
    Algeria', 'Africa'], [66, 'Guatemala', 'North America'], [67, 'Azerbaijan', '
    Europe'], [68, 'Morocco', 'Africa'], [69, 'New Zealand', 'Oceania']]

#Run the function for different sports
print('Football')
fvi_football, mfi_football, score_football, log_score_football = choosing(
    football_ranking, 3500, 1.5, 5)
print(fvi_football, mfi_football, score_football)
print('Basketball')
fvi_basket, mfi_basket, score_basket, log_score_basket = choosing(basket_ranking,
     2400, 3, 1)
```

```
print(fvi_basket, mfi_basket, score_basket)
print('Rugby')
fvi_rugby, mfi_rugby, score_rugby, log_score_rugby = choosing(rugby_ranking, 450, 1, 2)
print(fvi_rugby, mfi_rugby, score_rugby)
print('Cricket')
fvi_cricket, mfi_cricket, score_cricket, log_score_cricket = choosing(cricket_ranking, 2500, 2.5, 2)
print(fvi_cricket, mfi_cricket, score_cricket)
print('Volleyball')
fvi_volleyball, mfi_volleyball, score_volleyball, log_score_volleyball = choosing(volleyball_ranking, 900, 3, 4)
print(fvi_volleyball, mfi_volleyball, score_volleyball)

#Plot the scores on a bar chart
import matplotlib.pyplot as plt
scores = [float(score_football), float(score_basket), float(score_rugby), float(score_cricket), float(score_volleyball)]
log_scores = [log_score_football, log_score_basket, log_score_rugby, log_score_cricket, log_score_volleyball]
x = np.arange(5)
width = 0.25
fig, ax = plt.subplots()
bars = ax.bar(x + width, log_scores, width, label='Score')

# Add labels to the bars
def add_value_labels(bars):
    for bar in bars:
        height = bar.get_height()
        ax.annotate('{}'.format(round(height, 2)),
                    xy=(bar.get_x() + bar.get_width() / 2, height),
                    xytext=(0, 3),
                    textcoords="offset points",
                    ha='center', va='bottom')
add_value_labels(bars)
plt.tight_layout()
plt.show()

# Print final scores
print(scores)
print(log_scores)
log_scores_men = log_scores

sports = ["Football", "Basketball", "Rugby", "Cricket", "Volleyball"]

# plot the graphs
x = np.arange(len(sports))

width = 0.4

fig, ax = plt.subplots(figsize=(10,6))

bars1 = ax.bar(x - width/2, log_scores_women, width, color='#f73472', label='Women')
bars2 = ax.bar(x + width/2, log_scores_men, width, color='#3881f5', label='Men')

ax.set_xticks(x)
ax.set_xticklabels(sports)
ax.set_ylabel("Sport Suitability Score")
ax.legend()

# Adding values on top of bars
for bar in bars1:
```

```
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, height, f'{height:.2f}', ha='center'
    , va='bottom')

for bar in bars2:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, height, f'{height:.2f}', ha='center'
    , va='bottom')

plt.show()
```

B Choosing sports

B.1 League factor

League country of origin	**league relative strength** (s)
England	85.9
Italy	84.2
Germany	84.0
Spain	83.5
France	83.1
Brazil	80.1
Portugal	78.6
Belgium	77.3
United States	77.0
Argentina	76.8
Netherlands	76.2
Denmark	76.0
Mexico	75.7
Czech Republic	75.2
Poland	75.1
Turkey	74.8

B.2 International factor

League name	Winner prize (p) [$\times 10^6$]
AFC CL	42.000
Copa Lib	225.900
AFC Pokal	14.800
CAF CL	19.200
CAF CC	11.500
UEFA CL	2912.400
CAF SC	0.750
UEFA EL	502.200
CONCACAF CC	7.000
CFU CC	0.115
OFC CC	0.325
AFC Pres. Cup	14.800
Recopa	2.500
UEFA SC	9.720
Copa Sud	80.180
Fifa Club	1000.000
CONCACAF L	0.500
UEFA CONF L	7.000

C Choosing clubs

C.1 Code

```
import pandas as pd
import matplotlib as mpl
import math

# Read the file containing data
file_path = 'games.csv'
df = pd.read_csv(file_path, header=None)
# Assign names to each column in the DataFrame
df.columns = ['home_club', 'away_club', 'date', 'gh', 'ga', 'full_time', '
    competition', 'home_indent', 'away_indent', 'home_country', 'away_country', '
    home_code', 'away_code', 'home_continent', 'away_continent', 'continent', '
    level']
```

```
10
11 # Format the column headers to be uppercase
12 df.style \
13   .format_index(str.upper, axis=1)
14
15 # Remove specified columns from the DataFrame
16 df.drop(['home_indent', 'away_indent', 'full_time','home_code', 'away_code'],
       axis=1, inplace=True)
17
18 # Generate an array of unique competition names
19 international_competitions = df.loc[df['level'] == 'international', 'competition'
       ].unique()
20
21 all_teams = pd.concat([df['home_club'], df['away_club']]).unique()
22
23 # Filter teams playing at an international level
24 international_teams = set(
25     df.loc[df['level'] == 'international', 'home_club']
26 ).union(
27     df.loc[df['level'] == 'international', 'away_club']
28 )
29
30 elo_ratings = {}
31
32 # Teams Playing at  start with 1500 Elo, while others start with 1000 Elo
33 for team in all_teams:
34     if team in international_teams:
35         elo_ratings[team] = 1000 + 500
36     else:
37         elo_ratings[team] = 1000
38
39 # Function calculating the expected probability of a team winning based on Elo
       ratings
40 def expected_score(rating_a, rating_b):
41     return 1 / (1 + 10 ** ((rating_b - rating_a) / 400))
42
43 # Function
44 def update_elo(winner, loser, elo_ratings, level, competition):
45     expected_winner = expected_score(elo_ratings[winner], elo_ratings[loser])
46     expected_loser = expected_score(elo_ratings[loser], elo_ratings[winner])
47
48     k_factor = 10
49     sigma_factor = 1
50
51     # Adjust Elo changes based on country strength in sigma_dictionary
52     sigma_dictionary = {
53         'england': 85.9,
54         'italy': 84.2,
55         'germany': 84.0,
56         'spain': 83.5,
57         'france': 83.1,
58         'brazil': 80.1,
59         'portugal': 78.6,
60         'belgium': 77.3,
61         'united-states': 77.0,
62         'argentina': 76.8,
63         'netherlands': 76.2,
64         'denmark': 76.0,
65         'mexico': 75.5,
66         'czech-republic': 75.2,
67         'poland': 75.1,
68         'turkey': 74.8,
69     }
```

```
70
71      alpha_factor = 0
72
73      #Adjust Elo based on competition prize pool in alpha_dictionary
74      alpha_dictionary = {
75          'AFC CL': 42000,
76          'Copa Lib': 225900,
77          'AFC Pokal': 14800,
78          'CAF CL': 19200,
79          'CAF CC': 11500,
80          'UEFA CL': 2912400,
81          'CAF SC': 750,
82          'UEFA EL': 502200,
83          'CONCACAF CC': 7000,
84          'CFU CC': 115,
85          'OFC CC': 325,
86          'AFC Pres. Cup': 14800,
87          'Recopa': 2500,
88          'UEFA SC': 9720,
89          'Copa Sud': 80180,
90          'Fifa Club': 1000000,
91          'CONCACAF L': 500,
92          'UEFA CONF L': 7000
93      }
94
95      # Adjust sigma_factor if the competition is in the sigma_dictionary:
96      if competition in sigma_dictionary.keys():
97          calc = sigma_dictionary[competition]
98          sigma_factor = calc/74.7
99
100     # Adjust sigma_factor if the competition is in the alpha_dictionary:
101     if competition in alpha_dictionary.keys():
102         calc = alpha_dictionary[competition]
103         alpha_factor = math.log(calc)
104
105     # Compute Elo change
106     normal_winner_gain = sigma_factor*k_factor * (1 - expected_winner) +
    alpha_factor
107     normal_loser_change = k_factor * (0 - expected_loser)
108
109     # Increase ratings of winners of international matches
110     if level == 'international':
111         elo_ratings[winner] += normal_winner_gain - normal_loser_change
112     else:
113         elo_ratings[winner] += normal_winner_gain
114         elo_ratings[loser]  += normal_loser_change
115
116 # Iterate over the matches and update Elo ratings
117 for index, row in df.iterrows():
118     home_team = row['home_club']
119     away_team = row['away_club']
120     home_score = row['gh']
121     away_score = row['ga']
122     level = row['level']
123     competition = row['competition']
124
125     if home_score > away_score:
126         update_elo(home_team, away_team, elo_ratings, level, competition)
127     elif away_score > home_score:
128         update_elo(away_team, home_team, elo_ratings, level, competition)
129
130
131 # Convert elo_ratings into a dataFrame with columns "team" and "elo_rating"
```

```
elo_df = pd.DataFrame({
    "team": list(elo_ratings.keys()),
    "elo_rating": list(elo_ratings.values()),
})

# Map teams to continents and countries
team_continent_map = {}

for index, row in df.iterrows():
    team_continent_map[row['home_club']] = row['home_continent']
    team_continent_map[row['away_club']] = row['away_continent']

team_country_map = {}

for index, row in df.iterrows():
    team_country_map[row['home_club']] = row['home_country']
    team_country_map[row['away_club']] = row['away_country']

# Merge the mappings into the Elo DataFrame
elo_df['continent'] = elo_df['team'].map(team_continent_map)
elo_df['country'] = elo_df['team'].map(team_country_map)

# Sort Teams by Elo Rating
elo_df.sort_values(by='elo_rating', ascending=False, inplace=True)

# Identify the top club in each country.
top_teams_per_country = elo_df.groupby(['country', 'continent']).apply(lambda x:
   x.nlargest(1, 'elo_rating')).reset_index(drop=True)

# Rank these teams globally
top_teams_per_country.sort_values(by='elo_rating', ascending=False, inplace=True)
top_teams_per_country

# Initial setup
selected_teams = []

selected_continents={
    'Europe': 0,
    'Asia': 0,
    'Africa': 0,
    'North America': 0,
    'South America': 0,
    'Oceania': 0
}

#Select teams into dataframe
for index, row in top_teams_per_country.iterrows():
    continent = row['continent']

    if selected_continents[continent] < 2:
        selected_teams.append(row)
        selected_continents[continent] += 1
selected_teams_df = pd.DataFrame(selected_teams)

# Identify already selected countries
selected_countries = selected_teams_df['country'].unique()

# Filter out selected teams and those from their countries in the Elo DataFrame
remaining_teams = elo_df[~elo_df['team'].isin(selected_teams_df['team'])]
remaining_teams = remaining_teams[~remaining_teams['country'].isin(
   selected_countries)]

# Calculate how many more teams are needed to complete the 24-team selection
```

```
193 additional_teams_needed = 24 - len(selected_teams_df)
194
195 #Select additional teams based on Elo rating
196 additional_teams = []
197 for index, row in remaining_teams.nlargest(len(remaining_teams), 'elo_rating').
    iterrows():
198     if len(additional_teams) < additional_teams_needed:
199         if row['country'] not in selected_countries:
200             additional_teams.append(row)
201             selected_countries = list(selected_countries) + [row['country']]  #
    Update selected countries
202
203 additional_teams_df = pd.DataFrame(additional_teams)
204
205 #Final selection
206 final_selection = pd.concat([selected_teams_df, additional_teams_df]).reset_index
    (drop=True)
207
208 print("Final selection of teams:")
209 #Sort final selection by Elo rating
210 final_selection.sort_values(by='elo_rating', ascending=False, inplace=True)
211 final_selection
```

C.2 20 Clubs

Club name	Club ELO rating	Club country	Club continent
Real Madrid	2846	Spain	Europe
Bayern München	2809	Germany	Europe
Manchester City	2765	England	Europe
Dinamo Zagreb	2408	Croatia	Europe
Paris Saint-Germain	2401	France	Europe
Benfica	2401	Portgual	Europe
SE Palmeiras	2387	Brazil	South America
RB Salzburg	2362	Austria	Europe
Juventus	2350	Italy,	Europe
Al Ahly	2348	Egypt	Africa
PSV Eindhoven	2344	Netherlands	Europe
Qarabak FK	2326	Azerbaijan	Asia
TP Mazembe	2306	DR Congo	Africa
LDU Quito	2070	Ecuador	South America
Johor FC	2055	Indonesia	Asia
Ulsan Hyundai FC	1996	Korean Republic	Asia
Auckland City FC	1944	New Zealand	Oceania
UANL Tigres	1774	Mexico	North America
CD Olimpia	1690	Honduras	North America
Melbourne Heart FC	1679	Australia	Oceania

D Optimizing the schedule

D.1 Code

```
import math
import itertools
import random
import time
import copy
import json
```

```python
import statistics
import sys # For exiting cleanly
import pandas as pd
import plotly.graph_objects as go
from ortools.sat.python import cp_model

# ============================================================================
# 1. Configuration & Constants
# ============================================================================

# --- Input Data ---
TEAMS_FILE_PATH = "teams.json"
# Load only the first N teams if the file is larger. Set to None to load all.
# Useful for testing with smaller instances.
MAX_TEAMS_TO_LOAD = 20 # <--- Adjust number of teams to load here

# --- Scheduling Constraints ---
# Maximum number of consecutive home or away games allowed for any team.
# Set high (e.g., >= num_weeks) to effectively disable the constraint.
MAX_CONSECUTIVE_HOME_GAMES = 20 # Example: limit to 3 or 4 for realistic schedules
MAX_CONSECUTIVE_AWAY_GAMES = 20

# --- Solver Parameters ---
# Time limit in seconds for CP-SAT to find the *first* feasible solution.
CP_FEASIBILITY_TIME_LIMIT_SECONDS = 60.0

# --- Simulated Annealing (SA) Parameters ---
# These parameters significantly impact SA performance and require tuning based
# on the problem size (number of teams) and the scale of travel distances.
SA_INITIAL_TEMPERATURE = 10000.0 # Initial heat. Higher allows more "bad" moves early.
SA_COOLING_RATE = 0.995          # Multiplicative factor for temperature reduction (e.g., 0.99 to 0.999). Closer to 1 means slower cooling.
SA_MAX_ITERATIONS = 30000        # Total number of iterations for the SA search. Increase for larger problems or more thorough search.

# --- Fairness Weights for SA Cost Function ---
# These weights determine the trade-off between minimizing total travel distance
# and ensuring fairness (similar travel distances for all teams).
# W_STD_DEV needs careful tuning relative to typical travel distances.
# If W_STD_DEV is too low, fairness is ignored. If too high, total travel might increase significantly.
W_TOTAL_TRAVEL = 1.0  # Weight for minimizing the sum of all teams' travel distances.
W_STD_DEV = 10.0     # Weight for minimizing the standard deviation of travel distances across teams.

# --- Geographic Constants ---
EARTH_RADIUS_KM = 6371.0

# ============================================================================
# 2. Data Loading and Preparation
# ============================================================================

def load_team_data(filepath, max_teams=None):
    """Loads team data from a JSON file."""
    try:
        with open(filepath, "r") as f:
            all_teams_data = json.load(f)
        if max_teams is not None and max_teams > 0:
            loaded_data = all_teams_data[:max_teams]
            print(f"Loaded the first {len(loaded_data)} teams from '{filepath}'."
```

```
    )
        else:
            loaded_data = all_teams_data
            print(f"Loaded all {len(loaded_data)} teams from '{filepath}'.")
        return loaded_data
    except FileNotFoundError:
        print(f"Error: Team data file '{filepath}' not found.")
        sys.exit(1)
    except json.JSONDecodeError:
        print(f"Error: '{filepath}' does not contain valid JSON.")
        sys.exit(1)
    except Exception as e:
        print(f"Error loading '{filepath}': {e}")
        sys.exit(1)

def prepare_league_data(teams_data):
    """Processes loaded team data to set up league parameters and structures."""
    num_teams = len(teams_data)
    if num_teams == 0:
        print("Error: No teams loaded. Cannot create a schedule.")
        sys.exit(1)
    if num_teams % 2 != 0:
        print(f"Error: Number of teams ({num_teams}) must be even for a double round-robin.")
        sys.exit(1)

    num_weeks = (num_teams - 1) * 2 # Required weeks for a double round-robin.

    team_names = [team['name'] for team in teams_data]
    team_indices = list(range(num_teams))
    team_locations = {i: (team['latitude'], team['longitude']) for i, team in enumerate(teams_data)}
    week_indices_0 = list(range(num_weeks)) # 0 to N-1
    week_indices_1 = list(range(1, num_weeks + 1)) # 1 to N

    print(f"League Setup: {num_teams} teams, {num_weeks} weeks.")
    print(f"Max consecutive home/away games: {MAX_CONSECUTIVE_HOME_GAMES}/{MAX_CONSECUTIVE_AWAY_GAMES}")
    print(f"SA Cost Weights: W_TotalTravel={W_TOTAL_TRAVEL}, W_StdDev={W_STD_DEV}")

    return num_teams, num_weeks, team_names, team_indices, team_locations, week_indices_0, week_indices_1

# ==============================================================================
# 3. Distance Calculation
# ==============================================================================

def calculate_haversine_distance(lat1, lon1, lat2, lon2):
    """Calculates the great-circle distance between two points on Earth."""
    if (lat1, lon1) == (lat2, lon2):
        return 0.0
    lat1_rad, lon1_rad = math.radians(lat1), math.radians(lon1)
    lat2_rad, lon2_rad = math.radians(lat2), math.radians(lon2)
    dlon, dlat = lon2_rad - lon1_rad, lat2_rad - lat1_rad
    a = math.sin(dlat / 2)**2 + math.cos(lat1_rad) * math.cos(lat2_rad) * math.sin(dlon / 2)**2
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
    return EARTH_RADIUS_KM * c

def calculate_distance_matrix(team_locations, team_indices):
    """Creates a matrix of distances between all pairs of team locations."""
    num_teams = len(team_indices)
```

```
    dist_matrix = [[0.0] * num_teams for _ in range(num_teams)]
    for i in team_indices:
        for j in team_indices:
            if i != j:
                loc_i = team_locations[i]
                loc_j = team_locations[j]
                dist_matrix[i][j] = calculate_haversine_distance(loc_i[0], loc_i[1], loc_j[0], loc_j[1])
    return dist_matrix

# ==============================================================================
# 4. Schedule Representation and Helper Functions
# ==============================================================================

# Schedule Format:
# A dictionary where keys are week numbers (1-based) and values are lists of tuples.
# Each tuple represents a game: (home_team_index, away_team_index).
# Example: { 1: [(0, 1), (2, 3)], 2: [(1, 0), (3, 2)], ... }

def get_team_location_for_week(schedule, team_idx, week_idx_1):
    """
    Finds the location index (which is the home team's index) where a team plays
    in a given week (1-based index). Returns None if the game isn't found.
    """
    if week_idx_1 not in schedule or not schedule[week_idx_1]:
        return None # Week data missing or empty
    for home_team, away_team in schedule[week_idx_1]:
        if home_team == team_idx:
            return home_team # Team is home, location is its own index
        if away_team == team_idx:
            return home_team # Team is away, location is the opponent's index
    return None # Team not found playing in this week (shouldn't happen in valid DRR)

def calculate_team_travels(schedule, dist_matrix, num_teams, num_weeks):
    """
    Calculates the total travel distance for each team over the season.
    Returns a dictionary {team_idx: total_distance} or None if an error occurs
    (e.g., team location not found for a week, indicating schedule inconsistency).
    """
    if not schedule: return None
    team_travels = {k: 0.0 for k in range(num_teams)}
    week_indices_1 = list(range(1, num_weeks + 1))

    for team_idx in range(num_teams):
        current_location_idx = team_idx # Start at home base before week 1

        try:
            # Travel from home to week 1 game location
            week1_loc_idx = get_team_location_for_week(schedule, team_idx, 1)
            if week1_loc_idx is None: raise ValueError(f"Location missing for Team {team_idx} in Week 1")
            team_travels[team_idx] += dist_matrix[current_location_idx][week1_loc_idx]
            current_location_idx = week1_loc_idx

            # Travel between consecutive weeks (from week i location to week i+1 location)
            for w in range(1, num_weeks): # Iterate from week 1 to week num_weeks-1
                next_week_loc_idx = get_team_location_for_week(schedule, team_idx
```

```
    , w + 1)
174                 if next_week_loc_idx is None: raise ValueError(f"Location missing
     for Team {team_idx} in Week {w+1}")
175                 team_travels[team_idx] += dist_matrix[current_location_idx][
    next_week_loc_idx]
176                 current_location_idx = next_week_loc_idx # Update location for
    the next iteration
177 
178             # Travel from last game location back to home base
179             team_travels[team_idx] += dist_matrix[current_location_idx][team_idx]
180 
181         except ValueError as e:
182             # print(f"Travel Calculation Error: {e}. Returning None.") # Optional
     debug print
183             return None # Indicate an error due to missing location data
184 
185     return team_travels
186 
187 def calculate_schedule_costs(schedule, dist_matrix, num_teams, num_weeks,
    weight_total, weight_std_dev):
188     """
189     Calculates the total travel, standard deviation, and combined weighted cost
190     for a given schedule. Returns (None, None, None) if travel calculation fails.
191     """
192     team_travels = calculate_team_travels(schedule, dist_matrix, num_teams,
    num_weeks)
193     if team_travels is None:
194         return None, None, None # Indicate calculation failure
195 
196     travel_distances = list(team_travels.values())
197     total_travel = sum(travel_distances)
198 
199     std_dev = 0.0
200     if len(travel_distances) > 1:
201         try:
202             std_dev = statistics.stdev(travel_distances)
203         except statistics.StatisticsError:
204             # This happens if all travel distances are identical (variance is
    zero)
205             std_dev = 0.0
206 
207     combined_cost = (weight_total * total_travel) + (weight_std_dev * std_dev)
208     return total_travel, std_dev, combined_cost
209 
210 def check_schedule_feasibility(schedule, num_teams, num_weeks, max_home_streak,
    max_away_streak, verbose=True):
211     """
212     Checks if a given schedule meets the Double Round-Robin (DRR) requirements
213     and the consecutive home/away game streak limits.
214 
215     Args:
216         schedule (dict): The schedule to check (1-based weeks).
217         num_teams (int): Total number of teams.
218         num_weeks (int): Total number of weeks.
219         max_home_streak (int): Maximum allowed consecutive home games.
220         max_away_streak (int): Maximum allowed consecutive away games.
221         verbose (bool): If True, prints detailed error messages.
222 
223     Returns:
224         bool: True if the schedule is feasible, False otherwise.
225     """
226     errors = []
227     team_indices = list(range(num_teams))
```

```
    week_indices_1 = list(range(1, num_weeks + 1))
    games_per_week = num_teams // 2

    # 1. Basic Structure Check (Correct number of games per week)
    for w in week_indices_1:
        if w not in schedule or not isinstance(schedule.get(w), list) or len(schedule.get(w, [])) != games_per_week:
            errors.append(f"Structure Error: Week {w} has {len(schedule.get(w,[]))} games, expected {games_per_week}.")
            # Early exit if structure is fundamentally wrong
            if verbose: print("Feasibility Check Failed:"); [print(f"  - {e}") for e in errors]
            return False

    # 2. DRR Check (Each team plays every other team exactly once home and once away)
    required_matchups = set((i, j) for i in team_indices for j in team_indices if i != j)
    played_matchups = set()
    teams_playing_per_week = {w: set() for w in week_indices_1}

    for w in week_indices_1:
        for home_team, away_team in schedule[w]:
            # Check for self-play
            if home_team == away_team:
                errors.append(f"DRR Error: Team {home_team} plays itself in Week {w}.")
            # Check for repeated matchups
            if (home_team, away_team) in played_matchups:
                errors.append(f"DRR Error: Matchup ({home_team} vs {away_team}) is repeated.")
            played_matchups.add((home_team, away_team))
            # Check if a team plays multiple games in the same week
            if home_team in teams_playing_per_week[w]:
                errors.append(f"DRR Error: Team {home_team} involved in multiple games in Week {w}.")
            if away_team in teams_playing_per_week[w]:
                errors.append(f"DRR Error: Team {away_team} involved in multiple games in Week {w}.")
            teams_playing_per_week[w].add(home_team)
            teams_playing_per_week[w].add(away_team)

    # Check if all required matchups were played exactly once
    if played_matchups != required_matchups:
        missing = required_matchups - played_matchups
        extra = played_matchups - required_matchups
        errors.append(f"DRR Error: Matchup set mismatch. Missing {len(missing)}, Extra {len(extra)}.")
        # Optionally print missing/extra matchups if needed for debugging
        # if missing: errors.append(f"  Missing: {missing}")
        # if extra: errors.append(f"  Extra: {extra}")

    # Check if the correct number of teams play each week
    for w in week_indices_1:
        if len(teams_playing_per_week[w]) != num_teams:
            errors.append(f"DRR Error: Week {w} involves {len(teams_playing_per_week[w])} teams, expected {num_teams}.")

    # 3. Streak Check (Consecutive home/away games)
    for team_idx in team_indices:
        consecutive_home = 0
        consecutive_away = 0
        for w in week_indices_1:
```

```
            is_home = None
            for home_team, away_team in schedule[w]:
                if home_team == team_idx:
                    is_home = True
                    break
                if away_team == team_idx:
                    is_home = False
                    break

            if is_home is None:
                # This should have been caught by DRR checks, but defensively check anyway.
                errors.append(f"Streak Error: Team {team_idx} not found playing in Week {w}.")
                continue # Skip streak logic for this week if game missing

            if is_home:
                consecutive_home += 1
                consecutive_away = 0
            else:
                consecutive_away += 1
                consecutive_home = 0

            if consecutive_home > max_home_streak:
                errors.append(f"Streak Error: Team {team_idx} exceeds max home streak ({max_home_streak}) ending Week {w}.")
            if consecutive_away > max_away_streak:
                errors.append(f"Streak Error: Team {team_idx} exceeds max away streak ({max_away_streak}) ending Week {w}.")

    # Final Result
    if errors:
        if verbose:
            print("Feasibility Check Failed:")
            for e in errors:
                print(f"  - {e}")
        return False
    else:
        # Only print success if verbose was also requested for failures
        # if verbose: print("Feasibility Check Passed.")
        return True


# ==============================================================================
# 5. Phase 1: Find Initial Feasible Schedule (CP-SAT)
# ==============================================================================

class FeasibleSolutionFinderCallback(cp_model.CpSolverSolutionCallback):
    """CP-SAT callback to stop search after finding the first feasible solution."""
    def __init__(self, opponent_vars, is_home_vars, num_teams, num_weeks):
        cp_model.CpSolverSolutionCallback.__init__(self)
        self._opponent_vars = opponent_vars
        self._is_home_vars = is_home_vars
        self._num_teams = num_teams
        self._num_weeks = num_weeks
        self.solution_schedule = None
        self._solution_count = 0

    def OnSolutionCallback(self):
        """Called by the solver when a feasible solution is found."""
        self._solution_count += 1
        print(f"Feasible solution #{self._solution_count} found by CP-SAT.
```

```
    Stopping search.")

        # --- Reconstruct the schedule from variable values ---
        schedule = {w + 1: [] for w in range(self._num_weeks)} # 1-based weeks
        # Keep track of games already added per week to avoid duplicates (H-A and A-H)
        processed_games_in_week = {w: set() for w in range(self._num_weeks)}

        for w_0based in range(self._num_weeks):
            week_1based = w_0based + 1
            for team_k in range(self._num_teams):
                # Check if this team's game for the week has already been added
                # by processing its opponent.
                game_processed = False
                for home, away in processed_games_in_week[w_0based]:
                    if team_k == home or team_k == away:
                        game_processed = True
                        break
                if game_processed:
                    continue

                # If not processed, figure out the game and add it
                opponent_idx = self.Value(self._opponent_vars[team_k, w_0based])
                is_team_k_home = self.Value(self._is_home_vars[team_k, w_0based])

                if is_team_k_home:
                    home_team, away_team = team_k, opponent_idx
                else:
                    home_team, away_team = opponent_idx, team_k

                schedule[week_1based].append((home_team, away_team))
                # Mark this game pair as processed for this week
                processed_games_in_week[w_0based].add(tuple(sorted((home_team, away_team))))

        self.solution_schedule = schedule
        self.StopSearch() # Stop CP-SAT search immediately

    def get_schedule(self):
        """Returns the found schedule (or None if none was found)."""
        return self.solution_schedule

def find_feasible_schedule_cpsat(num_teams, num_weeks, max_home, max_away, time_limit):
    """
    Uses Google OR-Tools CP-SAT solver to find an initial feasible schedule.

    Returns:
        dict: A feasible schedule in the standard format, or None if no solution
              is found within the time limit or if an error occurs.
    """
    print("\n--- Phase 1: Finding First Feasible Schedule (CP-SAT) ---")
    start_time = time.time()

    model = cp_model.CpModel()
    team_indices = list(range(num_teams))
    week_indices_0 = list(range(num_weeks))

    # --- CP-SAT Variables ---
    # opponent_vars[k, w]: opponent of team k in week w (0-based index)
    opponent_vars = {}
    for k in team_indices:
        for w in week_indices_0:
```

```
            # The opponent cannot be the team itself.
            opponent_vars[k, w] = model.NewIntVar(0, num_teams - 1, f"opp_{k}_w{w}")
            model.Add(opponent_vars[k, w] != k)

    # is_home_vars[k, w]: boolean, True if team k is home in week w
    is_home_vars = {}
    for k in team_indices:
        for w in week_indices_0:
            is_home_vars[k, w] = model.NewBoolVar(f"home_{k}_w{w}")

    # --- CP-SAT Constraints ---
    # 1. Symmetry/Pairing Constraint: If k plays o, then o must play k.
    #    Also, one must be home, the other away.
    all_opponent_vars_list = [opponent_vars[j, w] for j in team_indices for w in week_indices_0] # Flat list for AddElement
    all_is_home_vars_list = [is_home_vars[j,w] for j in team_indices for w in week_indices_0] # Flat list for AddElement

    for w in week_indices_0:
        for k in team_indices:
            opponent_of_k = opponent_vars[k, w] # This is a variable representing the index of k's opponent

            # Constraint: opponent_vars[opponent_of_k, w] == k
            # We need AddElement because opponent_of_k is a variable, not a fixed index.
            # It retrieves the value from the opponent_vars list at the index specified by opponent_of_k.
            opp_list_for_week_w = [opponent_vars[j, w] for j in team_indices]
            model.AddElement(opponent_of_k, opp_list_for_week_w, k)

            # Constraint: is_home_vars[k, w] != is_home_vars[opponent_of_k, w]
            # One team must be home, the other away.
            home_list_for_week_w = [is_home_vars[j, w] for j in team_indices]
            is_home_opponent = model.NewBoolVar(f"is_home_opp_{k}_w{w}")
            model.AddElement(opponent_of_k, home_list_for_week_w, is_home_opponent)
            model.Add(is_home_vars[k, w] != is_home_opponent)


    # 2. Double Round-Robin Constraint: Each team i plays team j exactly once at home.
    for i in team_indices:
        for j in team_indices:
            if i == j: continue

            # Create a list of boolean variables, one for each week.
            # plays_home_ij_w[w] is True if i plays j at home in week w.
            plays_home_ij_w_vars = []
            for w in week_indices_0:
                # Intermediate bool var: lit_is_opponent_j == (opponent_vars[i, w] == j)
                lit_is_opponent_j = model.NewBoolVar(f'lit_opp_{i}_{j}_w{w}')
                model.Add(opponent_vars[i, w] == j).OnlyEnforceIf(lit_is_opponent_j)
                model.Add(opponent_vars[i, w] != j).OnlyEnforceIf(lit_is_opponent_j.Not())

                # Target bool var: target_plays_home == (lit_is_opponent_j AND is_home_vars[i, w])
                target_plays_home = model.NewBoolVar(f"plays_home_{i}_{j}_w{w}")
                model.AddBoolAnd([lit_is_opponent_j, is_home_vars[i, w]]).
```

```
    OnlyEnforceIf(target_plays_home)
448                 # Equivalent to: If target_plays_home is false, then either
    opponent is not j OR i is not home.
449                 model.AddBoolOr([lit_is_opponent_j.Not(), is_home_vars[i, w].Not
    ()]).OnlyEnforceIf(target_plays_home.Not())
450
451                 plays_home_ij_w_vars.append(target_plays_home)
452
453             # Constraint: The sum of these boolean variables across all weeks
    must be exactly 1.
454             # This means team i plays team j at home exactly once during the
    season.
455             model.Add(sum(plays_home_ij_w_vars) == 1)
456
457
458     # 3. Streak Constraints: Limit consecutive home/away games.
459     #    A sliding window sum ensures no more than max_N games of the same type
    occur consecutively.
460     for k in team_indices:
461         # Home streak constraint
462         if max_home < num_weeks: # Only add if constraint is meaningful
463             for w_start in range(num_weeks - max_home):
464                 window_vars = [is_home_vars[k, w] for w in range(w_start, w_start
     + max_home + 1)]
465                 model.Add(sum(window_vars) <= max_home) # Max 'max_home' true
    values in any window of size max_home + 1
466
467         # Away streak constraint (using Not() for away games)
468         if max_away < num_weeks: # Only add if constraint is meaningful
469              for w_start in range(num_weeks - max_away):
470                 # We sum the negation (is_away) variables.
471                 window_vars = [is_home_vars[k, w].Not() for w in range(w_start,
    w_start + max_away + 1)]
472                 model.Add(sum(window_vars) <= max_away) # Max 'max_away' true (
    away) values in any window of size max_away + 1
473
474     # --- Solve ---
475     solver = cp_model.CpSolver()
476     solver.parameters.max_time_in_seconds = time_limit
477     # solver.parameters.log_search_progress = True # Uncomment for detailed CP-
    SAT logs
478
479     solution_callback = FeasibleSolutionFinderCallback(opponent_vars,
    is_home_vars, num_teams, num_weeks)
480     status = solver.Solve(model, solution_callback)
481     end_time = time.time()
482
483     print(f"Phase 1 Time (CP-SAT): {end_time - start_time:.2f} seconds")
484     print(f"CP-SAT Status: {solver.StatusName(status)}")
485
486     feasible_schedule = solution_callback.get_schedule()
487
488     if feasible_schedule:
489         print("Verifying Phase 1 schedule feasibility...")
490         # Use verbose=True here for detailed errors if verification fails
491         is_feasible = check_schedule_feasibility(feasible_schedule, num_teams,
    num_weeks, max_home, max_away, verbose=True)
492         if is_feasible:
493             print("Phase 1 schedule (from CP-SAT) is FEASIBLE.")
494             return feasible_schedule
495         else:
496             print("Error: CP-SAT returned a schedule that failed verification!")
497             return None
```

```
    else:
        print(f"CP-SAT failed to find a feasible solution within the time limit (Status: {solver.StatusName(status)}).")
        return None

# ==============================================================================
# 6. Phase 2: Optimize Schedule (Simulated Annealing)
# ==============================================================================

# --- SA Neighbor Generation Moves ---
# These functions take a schedule and modify it slightly to create a "neighbor".
# They should ensure the move *potentially* preserves feasibility (DRR might be broken
# temporarily, but streak constraints should ideally be considered).
# The main SA loop will re-check feasibility after the move.

def _swap_homes_in_week(schedule, week_idx_1, num_teams):
    """
    SA Move: In a randomly chosen week, randomly select two games
             and swap their home/away assignments.
             Example: (A vs B) and (C vs D) becomes (A vs D) and (C vs B).
             This preserves which teams play in that week but changes opponents.
             *Warning*: This move breaks the DRR constraint (matchups change).
                        It needs careful use or combination with other moves.
                        Currently commented out in the main SA loop due to DRR violation.
    """
    if week_idx_1 not in schedule or len(schedule[week_idx_1]) < 2:
        return False # Cannot swap if fewer than 2 games

    games_in_week = schedule[week_idx_1]
    idx1, idx2 = random.sample(range(len(games_in_week)), 2) # Choose 2 distinct game indices

    game1 = games_in_week[idx1] #(h1, a1)
    game2 = games_in_week[idx2] #(h2, a2)
    h1, a1 = game1
    h2, a2 = game2

    # Create the new swapped games
    new_game1 = (h1, a2)
    new_game2 = (h2, a1)

    # Update the schedule for that week
    schedule[week_idx_1][idx1] = new_game1
    schedule[week_idx_1][idx2] = new_game2
    return True # Move applied

def _swap_rounds(schedule, week_indices_1):
    """
    SA Move: Randomly select two different weeks and swap all the games
             between them. This preserves the overall DRR matchup set
             but changes the timing, affecting travel and streaks.
    """
    if len(week_indices_1) < 2:
        return False # Cannot swap if fewer than 2 weeks

    w1, w2 = random.sample(week_indices_1, 2) # Choose 2 distinct week indices

    # Swap the lists of games for the chosen weeks
    schedule[w1], schedule[w2] = schedule[w2], schedule[w1]
    return True # Move applied

```

```python
def _swap_opponents_in_week(schedule, week_idx_1):
    """
    SA Move: In a randomly chosen week, randomly select two games (H1 vs A1) and (H2 vs A2).
             Swap the away teams: (H1 vs A2) and (H2 vs A1).
             This preserves home teams for the week but changes matchups.
             *Warning*: This move breaks the DRR constraint.
    """
    if week_idx_1 not in schedule or len(schedule[week_idx_1]) < 2:
        return False

    games = schedule[week_idx_1]
    idx1, idx2 = random.sample(range(len(games)), 2)

    h1, a1 = games[idx1]
    h2, a2 = games[idx2]

    # Swap away teams
    schedule[week_idx_1][idx1] = (h1, a2)
    schedule[week_idx_1][idx2] = (h2, a1)
    return True

def _swap_home_away_in_game(schedule, week_idx_1):
    """
    SA Move: Randomly pick a week and a game within that week,
             and swap the home and away teams for that single game.
             *Warning*: This move breaks the DRR constraint (changes who is home).
    """
    if week_idx_1 not in schedule or not schedule[week_idx_1]:
        return False

    games = schedule[week_idx_1]
    game_idx = random.randrange(len(games))
    home_team, away_team = games[game_idx]

    # Swap home and away
    schedule[week_idx_1][game_idx] = (away_team, home_team)
    return True

# --- Main Simulated Annealing Function ---

def optimize_schedule_sa(initial_schedule, dist_matrix, num_teams, num_weeks,
                         max_home, max_away, week_indices_1, sa_config, cost_weights):
    """
    Improves an initial feasible schedule using Simulated Annealing to minimize
    a weighted combination of total travel and travel fairness (std dev).

    Args:
        initial_schedule (dict): A feasible starting schedule.
        dist_matrix (list[list]): Precomputed distances between teams.
        num_teams, num_weeks (int): League parameters.
        max_home, max_away (int): Streak limits.
        week_indices_1 (list): List of week numbers [1, ..., num_weeks].
        sa_config (dict): Contains SA parameters ('initial_temp', 'cooling_rate', 'max_iterations').
        cost_weights (dict): Contains cost function weights ('w_total', 'w_std_dev').

    Returns:
        tuple: (best_schedule_found, best_total_travel, best_std_dev, best_combined_cost)
```

```
            Returns the initial schedule's details if SA fails to find improvements or errors occur.
    """
    print("\n--- Phase 2: Improving Schedule (Simulated Annealing) ---")
    start_time = time.time()

    # Initial state
    current_schedule = copy.deepcopy(initial_schedule)
    initial_total, initial_stddev, initial_combined = calculate_schedule_costs(
        current_schedule, dist_matrix, num_teams, num_weeks,
        cost_weights['w_total'], cost_weights['w_std_dev']
    )
    if initial_combined is None:
        print("Error: Could not calculate cost for the initial schedule. Aborting SA.")
        return initial_schedule, 0, 0, 0 # Return dummy values

    current_cost = initial_combined
    best_schedule = copy.deepcopy(current_schedule)
    best_cost = current_cost
    best_total_travel = initial_total
    best_std_dev = initial_stddev

    print(f"Initial SA State: Total Travel={initial_total:,.1f} km, Std Dev={initial_stddev:,.1f} km, Combined Cost={current_cost:,.2f}")

    T = sa_config['initial_temp']
    alpha = sa_config['cooling_rate']
    max_iterations = sa_config['max_iterations']

    accepted_moves = 0
    infeasible_neighbors_skipped = 0
    cost_calc_errors = 0

    # --- Moves available for SA ---
    # Select moves that are likely to maintain or easily restore feasibility.
    # 'SwapRounds' is generally safe for DRR but affects streaks.
    # Moves like 'SwapHomesInWeek', 'SwapOpponentsInWeek', 'SwapHomeAwayInGame'
    # break DRR constraints and are harder to manage unless combined with repair mechanisms.
    # We will primarily use 'SwapRounds' for this example.
    available_moves = ["SwapRounds"]
    # Add other moves here if desired, but be aware of feasibility implications.
    # available_moves = ["SwapRounds", "SwapHomeAwayInGame"] # Example with a DRR-breaking move

    for i in range(max_iterations):
        # 1. Create Neighbor by Applying a Random Move
        neighbor_schedule = copy.deepcopy(current_schedule)
        move_type = random.choice(available_moves)
        move_successful = False

        if move_type == "SwapRounds" and num_weeks >= 2:
            move_successful = _swap_rounds(neighbor_schedule, week_indices_1)
        # --- Example of including a DRR-breaking move ---
        # elif move_type == "SwapHomeAwayInGame" and num_weeks >= 1:
        #     w = random.choice(week_indices_1)
        #     move_successful = _swap_home_away_in_game(neighbor_schedule, w)
        # Add other move types here...

        if not move_successful:
            continue # Try another iteration if move failed (e.g., not enough weeks/games)
```

```
671
672         # 2. Check Feasibility of Neighbor
673         # IMPORTANT: If using moves that break DRR, this check will likely fail often.
674         # A more advanced SA might include "repair" steps or allow temporary infeasibility.
675         if not check_schedule_feasibility(neighbor_schedule, num_teams, num_weeks, max_home, max_away, verbose=False):
676             infeasible_neighbors_skipped += 1
677             continue # Reject infeasible neighbors outright
678
679         # 3. Calculate Cost of Neighbor
680         neighbor_total, neighbor_stddev, neighbor_cost = calculate_schedule_costs(
681             neighbor_schedule, dist_matrix, num_teams, num_weeks,
682             cost_weights['w_total'], cost_weights['w_std_dev']
683         )
684
685         if neighbor_cost is None:
686             cost_calc_errors += 1
687             continue # Skip if cost calculation failed (e.g., schedule became inconsistent)
688
689         # 4. Acceptance Criterion (Metropolis)
690         delta_cost = neighbor_cost - current_cost
691
692         # Accept if neighbor is better (delta_cost < 0)
693         # Accept "worse" neighbors with a probability decreasing with temperature T
694         if delta_cost < 0 or (T > 1e-9 and random.random() < math.exp(-delta_cost / T)):
695             accepted_moves += 1
696             current_schedule = neighbor_schedule # Keep the neighbor (it's already a copy)
697             current_cost = neighbor_cost
698
699             # Update best solution found so far
700             if current_cost < best_cost:
701                 best_cost = current_cost
702                 best_schedule = copy.deepcopy(current_schedule) # Store the best schedule state
703                 best_total_travel = neighbor_total
704                 best_std_dev = neighbor_stddev
705                 # Optional: Print improvement
706                 # print(f"Iter {i+1}: New best! Cost={best_cost:,.2f} (T={best_total_travel:,.1f}, S={best_std_dev:,.1f})")
707
708         # 5. Cool Down
709         T *= alpha
710
711         # Optional: Periodic progress update
712         if (i + 1) % (max_iterations // 10) == 0:
713            progress = (i + 1) * 100 // max_iterations
714            print(f"  SA Progress: {progress}% (Iter {i+1}/{max_iterations}), Temp={T:.2f}, "
715                  f"CurrentCost={current_cost:,.2f}, BestCost={best_cost:,.2f}")
716
717
718     end_time = time.time()
719     print(f"Phase 2 Time (Simulated Annealing): {end_time - start_time:.2f} seconds")
720     print(f"SA Summary: Accepted {accepted_moves} moves, Skipped {infeasible_neighbors_skipped} infeasible, "
```

```
          f"{cost_calc_errors} cost errors.")
    print(f"Final Best Schedule Found by SA:")
    print(f"  - Combined Cost (Objective): {best_cost:,.2f}")
    print(f"  - Total Travel Distance:     {best_total_travel:,.2f} km")
    print(f"  - Travel Standard Deviation: {best_std_dev:,.2f} km")

    return best_schedule, best_total_travel, best_std_dev, best_cost

# ==============================================================================
# 7. Output and Visualization
# ==============================================================================

def print_schedule_details(schedule, team_names, week_indices_1):
    """Prints the final schedule week by week."""
    print("\n" + "=" * 30)
    print("Final Optimized Schedule Details:")
    print("=" * 30)
    if not schedule:
        print("No schedule data to display.")
        return

    for w in week_indices_1:
        print(f"\n--- Week {w} ---")
        if w in schedule and schedule[w]:
            for home_idx, away_idx in schedule[w]:
                # Basic check for valid indices
                if 0 <= home_idx < len(team_names) and 0 <= away_idx < len(team_names):
                    print(f"  {team_names[home_idx]} vs {team_names[away_idx]}")
                else:
                    print(f"  Invalid Game Data: Home={home_idx}, Away={away_idx}")
        else:
            print("  (No games scheduled for this week - data missing)")

def print_travel_summary(final_schedule, dist_matrix, team_names, num_teams, num_weeks, best_total_from_sa):
    """Calculates and prints the final travel summary and performs verification."""
    print("\n" + "=" * 30)
    print("Final Travel Summary & Verification:")
    print("=" * 30)

    # Recalculate final travels directly from the best schedule for verification
    final_team_travels = calculate_team_travels(final_schedule, dist_matrix, num_teams, num_weeks)

    if final_team_travels is None:
        print("  Error: Could not recalculate final team travels for verification.")
        print(f"  Best Total Travel reported by SA: {best_total_from_sa:,.2f} km (cannot verify)")
        return # Cannot proceed with summary

    # Print individual team travels
    print("Final Individual Team Travels (km):")
    max_name_len = max(len(name) for name in team_names) if team_names else 10
    for i, name in enumerate(team_names):
        print(f"  {name:<{max_name_len}}: {final_team_travels[i]:,.1f}")

    # Verify total travel
    recalculated_total = sum(final_team_travels.values())
    print("\nVerification:")
```

```
    print(f"  Recalculated Total Travel (Sum): {recalculated_total:,.2f} km")
    print(f"  Best Total Travel from SA run:   {best_total_from_sa:,.2f} km")

    discrepancy = abs(recalculated_total - best_total_from_sa)
    if discrepancy < 0.1: # Allow small floating point tolerance
        print("  Verification Successful: Recalculated total matches SA result.")
    else:
        print(f"  Warning: Discrepancy of {discrepancy:.2f} km found in total travel verification.")

    # Print final standard deviation
    travel_values = list(final_team_travels.values())
    final_std_dev = 0.0
    if len(travel_values) > 1:
        try:
            final_std_dev = statistics.stdev(travel_values)
        except statistics.StatisticsError:
            final_std_dev = 0.0
    print(f"  Recalculated Travel Std Dev:     {final_std_dev:,.2f} km")


def visualize_travel_paths(schedule, team_locations, team_names, num_teams, num_weeks):
    """Generates and displays an interactive 2D map of team travel paths."""
    print("\n" + "=" * 30)
    print("Generating Travel Path Visualization (Mercator Projection Map)...")
    print("=" * 30)

    if not schedule:
        print("Visualization skipped: Final schedule data is missing.")
        return
    if not team_locations:
        print("Visualization skipped: Team location data is missing.")
        return
    if not team_names:
        print("Visualization skipped: Team name data is missing.")
        return

    try:
        import pandas as pd
        import plotly.graph_objects as go
    except ImportError:
        print("Visualization skipped: Plotly and/or Pandas library not found.")
        print("Please install them: pip install pandas plotly")
        return

    all_paths_data = []
    week_indices_1 = list(range(1, num_weeks + 1))
    team_indices = list(range(num_teams))
    visualization_possible = True

    # --- Prepare data points for each team's path ---
    for k in team_indices:
        team_name = team_names[k]
        path_points = []
        current_loc_idx = k # Start at home base

        # 1. Add starting home location
        home_lat, home_lon = team_locations[k]
        path_points.append({
            'lat': home_lat, 'lon': home_lon,
            'text': f'{team_name}: Start/End Home'
        })
```

```
        # 2. Add locations for each week's game
        try:
            for w in week_indices_1:
                game_loc_idx = get_team_location_for_week(schedule, k, w)
                if game_loc_idx is None:
                    print(f"Visualization Error: Missing location for Team {team_name} (idx {k}) in Week {w}. Skipping team.")
                    path_points = [] # Clear points for this team
                    visualization_possible = False # Mark that at least one team failed
                    break # Stop processing this team

                game_loc_lat, game_loc_lon = team_locations[game_loc_idx]
                opponent_name = team_names[game_loc_idx] if game_loc_idx != k else "Home" # Should always be opponent if away
                path_points.append({
                    'lat': game_loc_lat, 'lon': game_loc_lon,
                    'text': f'{team_name} Wk {w} @ {opponent_name}'
                })

            if not path_points: continue # Skip if loop was broken

            # 3. Add final return trip to home
            path_points.append({
                'lat': home_lat, 'lon': home_lon,
                'text': f'{team_name}: Return Home'
            })

            # 4. Format for DataFrame
            for i, p in enumerate(path_points):
                all_paths_data.append({
                    'Team': team_name,
                    'Step': i, # Order of travel
                    'Lat': p['lat'],
                    'Lon': p['lon'],
                    'Text': p['text'] # Hover text
                })

        except Exception as e:
            print(f"Unexpected error processing path for Team {team_name}: {e}")
            visualization_possible = False # Mark failure


    if not all_paths_data:
        print("Visualization skipped: No valid path data could be generated.")
        if not visualization_possible: print("(Errors occurred during path data preparation)")
        return

    # --- Create Plotly Figure ---
    df_paths = pd.DataFrame(all_paths_data)
    fig = go.Figure()

    # Calculate map center (average coordinates)
    avg_lat = df_paths['Lat'].mean()
    avg_lon = df_paths['Lon'].mean()

    # Add a trace (line + markers) for each team
    teams_to_plot = df_paths['Team'].unique()
    colors = go.Figure().layout.template.layout.colorway # Get default plotly colors
    num_colors = len(colors)
```

```
    for i, team in enumerate(teams_to_plot):
        df_team = df_paths[df_paths['Team'] == team].sort_values(by='Step')
        fig.add_trace(go.Scattermapbox(
            mode="markers+lines",
            lon=df_team['Lon'],
            lat=df_team['Lat'],
            marker={'size': 7, 'color': colors[i % num_colors]}, # Cycle through colors
            line={'width': 1.5},
            name=team,
            text=df_team['Text'],
            hoverinfo='text' # Show only our custom text on hover
        ))

    # Configure map layout
    fig.update_layout(
        title=f'Team Travel Paths (N={num_teams}, {num_weeks} Weeks) - Mercator Projection',
        hovermode='closest',
        legend_title_text='Teams',
        mapbox=dict(
            style="carto-positron", # Light, clean map style
            bearing=0,
            center=dict(lat=avg_lat, lon=avg_lon),
            pitch=0, # Force 2D view
            zoom=2.5  # Adjust zoom based on geographical spread of teams
                      # Lower for world view, higher for regional. Needs tuning.
        ),
        margin={"r":0,"t":50,"l":0,"b":0} # Minimize margins
    )

    print(f"Displaying interactive map for {len(teams_to_plot)} teams.")
    fig.show()
    # To save the map as an HTML file:
    # html_filename = f"gsl_travel_paths_{num_teams}teams.html"
    # fig.write_html(html_filename)
    # print(f"Map saved as '{html_filename}'")


# ==============================================================================
# 8. Main Execution Flow
# ==============================================================================


"""Main function to run the GSL scheduler."""
start_total_time = time.time()

# --- 1. Load and Prepare Data ---
teams_data = load_team_data(TEAMS_FILE_PATH, MAX_TEAMS_TO_LOAD)
num_teams, num_weeks, team_names, team_indices, team_locations, week_indices_0, week_indices_1 = prepare_league_data(teams_data)
distance_matrix = calculate_distance_matrix(team_locations, team_indices)

# --- 2. Phase 1: Find Feasible Schedule ---
initial_feasible_schedule = find_feasible_schedule_cpsat(
    num_teams, num_weeks,
    MAX_CONSECUTIVE_HOME_GAMES, MAX_CONSECUTIVE_AWAY_GAMES,
    CP_FEASIBILITY_TIME_LIMIT_SECONDS
)

if initial_feasible_schedule is None:
    print("\nFailed to find an initial feasible schedule. Exiting.")
```

```
    sys.exit(1)

# --- 3. Phase 2: Optimize Schedule ---
sa_parameters = {
    'initial_temp': SA_INITIAL_TEMPERATURE,
    'cooling_rate': SA_COOLING_RATE,
    'max_iterations': SA_MAX_ITERATIONS
}
cost_function_weights = {
    'w_total': W_TOTAL_TRAVEL,
    'w_std_dev': W_STD_DEV
}

final_schedule, final_total_travel, final_std_dev, final_combined_cost =
    optimize_schedule_sa(
    initial_schedule=initial_feasible_schedule,
    dist_matrix=distance_matrix,
    num_teams=num_teams, num_weeks=num_weeks,
    max_home=MAX_CONSECUTIVE_HOME_GAMES, max_away=MAX_CONSECUTIVE_AWAY_GAMES,
    week_indices_1=week_indices_1,
    sa_config=sa_parameters,
    cost_weights=cost_function_weights
)

# --- 4. Display Results ---
print_schedule_details(final_schedule, team_names, week_indices_1)
print_travel_summary(final_schedule, distance_matrix, team_names, num_teams,
    num_weeks, final_total_travel)

# --- 5. Visualize ---
visualize_travel_paths(final_schedule, team_locations, team_names, num_teams,
    num_weeks)

end_total_time = time.time()
print("\n" + "=" * 30)
print(f"Total Execution Time: {end_total_time - start_total_time:.2f} seconds")
print("=" * 30)


final_schedule_teams = []
for key in final_schedule:
    final_schedule_teams.append([])
    for match in final_schedule[key]:
        final_schedule_teams[key-1].append((TEAMS[match[0]], TEAMS[match[1]]))


clubs_time_zones = {
    "Real Madrid": 1,
    "Bayern M nchen": 1,
    "Manchester City": 0,
    "Dinamo Zagreb": 1,
    "Paris Saint-Germain": 1,
    "Benfica": 0,
    "SE Palmeiras": -3,
    "RB Salzburg": 1,
    "Juventus": 1,
    "Al Ahly": 2,
    "PSV Eindhoven": 1,
    "Qarabag FK": 4,
    "TP Mazembe": 2,
    "LDU Quito": -5,
    "Johor FC": 8,
    "Ulsan Hyundai FC": 9,
```

```
    "Auckland City FC": 12,
    "UANL Tigres": -6,
    "CD Olimpia": -6,
    "Melbourne Heart FC": 11
}


#Calculating match hours
def match_hours(final_schedule_teams, clubs_time_zones):
    next_timezone = clubs_time_zones.copy()
    team_timezone = clubs_time_zones.copy()

    def calc_next_timezone(next_timezone, curr_week, final_schedule_teams, clubs_time_zones):
        for matchup in final_schedule_teams[curr_week]:
            next_timezone[matchup[0]] = clubs_time_zones[matchup[0]]
            next_timezone[matchup[1]] = clubs_time_zones[matchup[0]]
        return next_timezone

    def adjust_team_timezone(next_timezone, team_timezone):
        for team in team_timezone:
            next = next_timezone[team]
            if abs(next - team_timezone[team]) > 4:
                if next - team_timezone[team] > 0:
                    team_timezone[team] += 4
                else:
                    team_timezone[team] -= 4
            else:
                team_timezone[team] = next
        return team_timezone

    from math import cos, pi

    def match_time(matchup, timezone_shift, clubs_time_zones):
        club_1 = matchup[0]
        club_2 = matchup[1]

        def calc_time(timezone_shift, club_1, club_2, clubs_time_zones):
            a = timezone_shift[club_1]
            b = timezone_shift[club_2]

            def trig(x, a):
                return -cos(2*pi/24 * (x-4+a))

            if a == b:
                x = 16 - a
                y = 1
                return (x, y)

            else:
                solutions = []
                solutions.append((4 - (a+b)/2)%24)
                solutions.append((16 - (a+b)/2)%24)
                solutions[0] = (solutions[0], trig(solutions[0], a))
                solutions[1] = (solutions[1], trig(solutions[1], a))
                if abs(a-b) == 12:
                    time_1 = (solutions[0][0] + clubs_time_zones[club_1])%24
                    time_2 = (solutions[1][0] + clubs_time_zones[club_1])%24
                    if abs(time_1 - 16) < abs(time_2 - 16):
                        return solutions[0]
                    elif abs(time_1 - 16) > abs(time_2 - 16):
                        return solutions[1]
                    elif time_1 > time_2:
```

```
                        return solutions[0]
                    else:
                        return solutions[1]


                else:
                    if solutions[0][1] > solutions[1][1]:
                        return solutions[0]
                    elif solutions[0][1] < solutions[1][1]:
                        return solutions[1]

        time, probability = calc_time(timezone_shift, club_1, club_2, clubs_time_zones)
        final_time = (time + clubs_time_zones[club_1])%24

        return (matchup, final_time, probability)


    matches_w_hours = []
    counter = 1
    for week in range(len(final_schedule_teams)):
        #print('Week ' + str(counter))
        next_timezone = calc_next_timezone(next_timezone, week, final_schedule_teams, clubs_time_zones)
        team_timezone = adjust_team_timezone(next_timezone, team_timezone)
        for matchup in final_schedule_teams[week]:
            time = match_time(matchup, team_timezone, clubs_time_zones)
            matches_w_hours.append(time)
            #print(time)
        #print('-' * 50)
        counter += 1

    return matches_w_hours



#Printing the results
matches_w_hours = match_hours(final_schedule_teams, clubs_time_zones)
equality = {k: [] for k in clubs_time_zones.keys()}

import statistics as st

for matches in matches_w_hours:
    for team in matches[0]:
        equality[team].append(matches[2])

for team in equality:
    equality[team] = st.mean(equality[team])

print(equality)
print(matches_w_hours)
```

D.2 Match Times

Week	Home Team	Away Team	Hour	CMTS
1	TP Mazembe	Real Madrid	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Bayern München	Qarabag FK	16.0	1.000
	Manchester City	Dinamo Zagreb	16.0	1.000
	Ulsan Hyundai FC	Paris Saint-Germain	18.0	0.866
	Juventus	Benfica	16.0	1.000
	SE Palmeiras	PSV Eindhoven	16.0	1.000
	UANL Tigres	RB Salzburg	14.5	0.924
	Johor FC	Al Ahly	17.0	0.966
	CD Olimpia	LDU Quito	16.0	1.000
	Auckland City FC	Melbourne Heart FC	16.0	1.000
2	Real Madrid	Manchester City	16.0	1.000
	Ulsan Hyundai FC	Bayern München	18.0	0.866
	Paris Saint-Germain	Dinamo Zagreb	16.0	1.000
	Benfica	Juventus	16.0	1.000
	SE Palmeiras	Qarabag FK	16.0	1.000
	Melbourne Heart FC	RB Salzburg	21.0	0.259
	Al Ahly	TP Mazembe	16.0	1.000
	LDU Quito	PSV Eindhoven	16.0	1.000
	Johor FC	Auckland City FC	16.0	1.000
	UANL Tigres	CD Olimpia	16.0	1.000
3	Dinamo Zagreb	Real Madrid	16.0	1.000
	CD Olimpia	Bayern München	12.5	0.609
	Manchester City	TP Mazembe	16.0	1.000
	Paris Saint-Germain	PSV Eindhoven	17.0	0.966
	Benfica	Qarabag FK	16.0	1.000
	LDU Quito	SE Palmeiras	16.0	1.000
	Ulsan Hyundai FC	RB Salzburg	18.0	0.866
	Juventus	Melbourne Heart FC	13.0	0.707
	Auckland City FC	Al Ahly	19.0	0.707

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	UANL Tigres	Johor FC	11.0	0.259
4	Melbourne Heart FC	Real Madrid	19.0	0.707
	LDU Quito	Bayern München	15.0	0.966
	Manchester City	UANL Tigres	17.0	0.966
	Benfica	Dinamo Zagreb	16.0	1.000
	Paris Saint-Germain	Qarabag FK	16.0	1.000
	CD Olimpia	SE Palmeiras	16.0	1.000
	RB Salzburg	TP Mazembe	16.0	1.000
	Auckland City FC	Juventus	19.5	0.609
	Al Ahly	Ulsan Hyundai FC	14.5	0.924
	PSV Eindhoven	Johor FC	16.0	1.000
5	Auckland City FC	Real Madrid	17.5	0.924
	PSV Eindhoven	Bayern München	16.0	1.000
	Manchester City	RB Salzburg	16.0	1.000
	Dinamo Zagreb	Qarabag FK	16.0	1.000
	Paris Saint-Germain	Benfica	16.0	1.000
	SE Palmeiras	TP Mazembe	16.0	1.000
	Johor FC	Juventus	17.5	0.924
	CD Olimpia	Al Ahly	14.0	0.866
	Melbourne Heart FC	LDU Quito	22.0	0.0
	UANL Tigres	Ulsan Hyundai FC	12.5	0.609
6	Real Madrid	PSV Eindhoven	14.0	0.866
	Bayern München	UANL Tigres	17.5	0.924
	Manchester City	Benfica	16.0	1.000
	Dinamo Zagreb	RB Salzburg	16.0	1.000
	TP Mazembe	Paris Saint-Germain	16.5	1.0
	SE Palmeiras	CD Olimpia	16.0	1.000
	Juventus	Al Ahly	14.5	0.924

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Qarabag FK	Melbourne Heart FC	14.5	0.924
	Auckland City FC	LDU Quito	20.5	0.4
	Johor FC	Ulsan Hyundai FC	17.5	0.924
7	Paris Saint-Germain	Real Madrid	16.0	1.000
	Bayern München	Benfica	16.0	1.000
	Dinamo Zagreb	Manchester City	16.0	1.000
	SE Palmeiras	Ulsan Hyundai FC	14.0	0.866
	Juventus	RB Salzburg	16.0	1.000
	Al Ahly	Melbourne Heart FC	15.5	1.0
	PSV Eindhoven	UANL Tigres	16.0	1.000
	Qarabag FK	TP Mazembe	16.0	1.000
	Johor FC	LDU Quito	16.5	1.000
	CD Olimpia	Auckland City FC	21.0	0.259
8	Real Madrid	Juventus	16.0	1.000
	Bayern München	CD Olimpia	17.5	0.924
	Ulsan Hyundai FC	Manchester City	20.0	1.000
	Dinamo Zagreb	TP Mazembe	16.0	1.000
	Paris Saint-Germain	LDU Quito	15.0	1.0
	SE Palmeiras	Benfica	16.0	1.000
	RB Salzburg	Johor FC	14.5	0.924
	PSV Eindhoven	Al Ahly	16.0	1.000
	Qarabag FK	Auckland City FC	16.0	1.000
	UANL Tigres	Melbourne Heart FC	12.0	1.000
9	Real Madrid	Bayern München	16.0	1.000
	TP Mazembe	Manchester City	16.0	1.000
	PSV Eindhoven	Dinamo Zagreb	16.0	1.000
	Auckland City FC	Paris Saint-Germain	21.5	0.924
	Al Ahly	Benfica	16.5	1.0

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	SE Palmeiras	Melbourne Heart FC	16.0	1.000
	RB Salzburg	LDU Quito	16.0	1.000
	UANL Tigres	Juventus	14.5	0.924
	Qarabag FK	Ulsan Hyundai FC	16.0	1.000
	CD Olimpia	Johor FC	13.0	0.707
10	SE Palmeiras	Real Madrid	16.0	1.000
	TP Mazembe	Bayern München	16.0	1.000
	CD Olimpia	Manchester City	14.0	0.866
	Dinamo Zagreb	Johor FC	16.0	1.000
	Paris Saint-Germain	Melbourne Heart FC	16.0	1.000
	Benfica	LDU Quito	16.0	1.000
	RB Salzburg	PSV Eindhoven	16.0	1.000
	Juventus	UANL Tigres	17.5	0.924
	Al Ahly	Qarabag FK	16.0	1.000
	Auckland City FC	Ulsan Hyundai FC	18.0	1.000
11	Real Madrid	LDU Quito	16.0	1.000
	Bayern München	PSV Eindhoven	16.0	1.000
	Manchester City	Qarabag FK	16.0	1.000
	Dinamo Zagreb	Melbourne Heart FC	16.0	1.000
	Benfica	Paris Saint-Germain	16.0	1.000
	Al Ahly	SE Palmeiras	16.0	1.000
	RB Salzburg	Juventus	16.0	1.000
	TP Mazembe	UANL Tigres	13.0	0.707
	Auckland City FC	Johor FC	16.0	1.000
	Ulsan Hyundai FC	CD Olimpia	16.0	1.000
12	RB Salzburg	Real Madrid	16.0	1.000
	Bayern München	Ulsan Hyundai FC	14.0	0.866
	SE Palmeiras	Manchester City	15.5	0.991

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Dinamo Zagreb	Juventus	16.0	1.000
	Paris Saint-Germain	Auckland City FC	12.5	0.609
	Benfica	TP Mazembe	16.0	1.000
	UANL Tigres	Al Ahly	9.0	0.707
	PSV Eindhoven	Melbourne Heart FC	16.0	1.000
	Qarabag FK	CD Olimpia	15.5	0.991
	LDU Quito	Johor FC	8.5	0.131
13	Bayern München	Real Madrid	16.0	1.000
	Manchester City	CD Olimpia	15.5	0.991
	Dinamo Zagreb	Benfica	16.0	1.000
	SE Palmeiras	Paris Saint-Germain	16.0	1.000
	RB Salzburg	Qarabag FK	16.0	1.000
	Juventus	TP Mazembe	16.0	1.000
	Al Ahly	Auckland City FC	15.0	0.966
	PSV Eindhoven	Ulsan Hyundai FC	16.0	1.000
	LDU Quito	UANL Tigres	13.5	0.793
	Johor FC	Melbourne Heart FC	17.5	0.924
14	Juventus	Real Madrid	16.0	1.000
	Auckland City FC	Bayern München	21.5	0.924
	Paris Saint-Germain	Manchester City	16.0	1.000
	Johor FC	Dinamo Zagreb	17.5	0.924
	Benfica	Ulsan Hyundai FC	16.0	1.000
	UANL Tigres	SE Palmeiras	15.0	0.966
	CD Olimpia	RB Salzburg	13.0	1.000
	LDU Quito	Al Ahly	14.5	0.924
	TP Mazembe	PSV Eindhoven	16.0	1.000
	Melbourne Heart FC	Qarabag FK	20.0	0.866
15	Real Madrid	Paris Saint-Germain	14.5	0.924

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Johor FC	Bayern München	16.0	1.000
	Manchester City	Al Ahly	16.0	1.000
	Qarabag FK	Paris Saint-Germain	16.0	1.000
	CD Olimpia	Benfica	15.0	0.966
	Ulsan Hyundai FC	SE Palmeiras	16.5	0.991
	RB Salzburg	Auckland City FC	16.0	1.000
	Juventus	PSV Eindhoven	16.0	1.000
	Melbourne Heart FC	TP Mazembe	21.5	0.609
	UANL Tigres	LDU Quito	14.0	0.866
16	Real Madrid	LDU Quito	16.0	1.000
	Bayern München	PSV Eindhoven	16.0	1.000
	Manchester City	Qarabag FK	16.0	1.000
	Dinamo Zagreb	Melbourne Heart FC	16.0	1.000
	Benfica	Paris Saint-Germain	16.0	1.000
	Al Ahly	SE Palmeiras	16.0	1.000
	RB Salzburg	Juventus	16.0	1.000
	TP Mazembe	UANL Tigres	13.0	0.707
	Auckland City FC	Johor FC	16.0	1.000
	Ulsan Hyundai FC	CD Olimpia	16.0	1.000
17	RB Salzburg	Real Madrid	16.0	1.000
	Bayern München	Ulsan Hyundai FC	14.0	0.866
	SE Palmeiras	Manchester City	15.5	0.991
	Dinamo Zagreb	Juventus	16.0	1.000
	Paris Saint-Germain	Auckland City FC	12.5	0.609
	Benfica	TP Mazembe	16.0	1.000
	UANL Tigres	Al Ahly	9.0	0.707
	PSV Eindhoven	Melbourne Heart FC	16.0	1.000
	Qarabag FK	CD Olimpia	15.5	0.991

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	LDU Quito	Johor FC	8.5	0.131
18	Bayern München	Real Madrid	16.0	1.000
	Manchester City	CD Olimpia	15.5	0.991
	Dinamo Zagreb	Benfica	16.0	1.000
	SE Palmeiras	Paris Saint-Germain	16.0	1.000
	RB Salzburg	Qarabag FK	16.0	1.000
	Juventus	TP Mazembe	16.0	1.000
	Al Ahly	Auckland City FC	15.0	0.966
	PSV Eindhoven	Ulsan Hyundai FC	16.0	1.000
	LDU Quito	UANL Tigres	13.5	0.793
	Johor FC	Melbourne Heart FC	17.5	0.924
19	Juventus	Real Madrid	16.0	1.000
	Auckland City FC	Bayern München	21.5	0.924
	Paris Saint-Germain	Manchester City	16.0	1.000
	Johor FC	Dinamo Zagreb	17.5	0.924
	Benfica	Ulsan Hyundai FC	16.0	1.000
	UANL Tigres	SE Palmeiras	15.0	0.966
	CD Olimpia	RB Salzburg	13.0	1.000
	LDU Quito	Al Ahly	14.5	0.924
	TP Mazembe	PSV Eindhoven	16.0	1.000
	Melbourne Heart FC	Qarabag FK	20.0	0.866
20	Real Madrid	Auckland City FC	14.5	0.924
	Johor FC	Bayern München	16.0	1.000
	Manchester City	Melbourne Heart FC	13.5	0.793
	Dinamo Zagreb	Paris Saint-Germain	16.0	1.000
	Benfica	CD Olimpia	16.0	1.000
	SE Palmeiras	Al Ahly	16.0	1.000
	LDU Quito	RB Salzburg	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	PSV Eindhoven	Juventus	16.0	1.000
	UANL Tigres	Qarabag FK	12.5	0.609
	TP Mazembe	Ulsan Hyundai FC	16.0	1.000
21	Real Madrid	Melbourne Heart FC	16.0	1.000
	Benfica	Bayern München	14.0	0.866
	Manchester City	PSV Eindhoven	16.0	1.000
	TP Mazembe	Dinamo Zagreb	16.0	1.000
	UANL Tigres	Paris Saint-Germain	14.5	0.924
	RB Salzburg	SE Palmeiras	17.0	0.965
	Juventus	Auckland City FC	16.0	1.000
	Al Ahly	Johor FC	15.5	0.924
	LDU Quito	Qarabag FK	15.0	0.966
	Benfica	Real Madrid	15.5	0.991
22	Bayern München	Auckland City FC	16.0	1.000
	Manchester City	Paris Saint-Germain	16.0	1.000
	LDU Quito	Dinamo Zagreb	15.0	0.966
	SE Palmeiras	UANL Tigres	16.0	1.000
	Al Ahly	RB Salzburg	16.0	1.000
	Ulsan Hyundai FC	Juventus	18.0	0.866
	Qarabag FK	PSV Eindhoven	16.0	1.000
	Johor FC	TP Mazembe	17.0	0.965
	CD Olimpia	Melbourne Heart FC	12.5	0.609
	Real Madrid	CD Olimpia	17.5	0.924
23	Qarabag FK	Bayern München	16.0	1.000
	RB Salzburg	Manchester City	16.0	1.000
	Al Ahly	Dinamo Zagreb	16.5	1.000
	Paris Saint-Germain	Juventus	16.0	1.000
	Benfica	Auckland City FC	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	TP Mazembe	SE Palmeiras	16.5	1.000
	Ulsan Hyundai FC	PSV Eindhoven	16.5	1.000
	LDU Quito	Melbourne Heart FC	15.0	0.966
	Johor FC	UANL Tigres	19.5	0.609
	Real Madrid	UANL Tigres	16.0	1.000
24	RB Salzburg	Bayern München	16.0	1.000
	Melbourne Heart FC	Manchester City	0.0	0.866
	Dinamo Zagreb	Auckland City FC	16.0	1.000
	Al Ahly	Paris Saint-Germain	16.0	1.000
	Benfica	SE Palmeiras	16.0	1.000
	LDU Quito	Juventus	15.0	0.966
	CD Olimpia	PSV Eindhoven	11.0	0.259
	Ulsan Hyundai FC	Qarabag FK	16.5	1.000
	TP Mazembe	Johor FC	15.0	0.966
	Real Madrid	Johor FC	16.0	1.000
25	Bayern München	Dinamo Zagreb	16.0	1.000
	Manchester City	Al Ahly	16.0	1.000
	Qarabag FK	Paris Saint-Germain	16.0	1.000
	CD Olimpia	Benfica	15.0	0.966
	Ulsan Hyundai FC	SE Palmeiras	16.5	1.000
	RB Salzburg	Auckland City FC	16.0	1.000
	Juventus	PSV Eindhoven	16.0	1.000
	Melbourne Heart FC	TP Mazembe	21.5	0.609
	UANL Tigres	LDU Quito	14.0	0.866
	Real Madrid	LDU Quito	16.0	1.000
26	Bayern München	PSV Eindhoven	16.0	1.000
	Manchester City	Qarabag FK	16.0	1.000
	Dinamo Zagreb	Melbourne Heart FC	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Benfica	Paris Saint-Germain	16.0	1.000
	Al Ahly	SE Palmeiras	16.0	1.000
	RB Salzburg	Juventus	16.0	1.000
	TP Mazembe	UANL Tigres	16.0	1.000
	Auckland City FC	Johor FC	16.0	1.000
	Ulsan Hyundai FC	CD Olimpia	16.0	1.000
	RB Salzburg	Real Madrid	16.0	1.000
27	Bayern München	Ulsan Hyundai FC	14.0	0.866
	SE Palmeiras	Manchester City	15.5	0.991
	Dinamo Zagreb	Juventus	16.0	1.000
	Paris Saint-Germain	Auckland City FC	12.5	0.609
	Benfica	TP Mazembe	16.0	1.000
	UANL Tigres	Al Ahly	9.0	0.707
	PSV Eindhoven	Melbourne Heart FC	16.0	1.000
	Qarabag FK	CD Olimpia	15.5	0.991
	LDU Quito	Johor FC	8.5	0.130
	Bayern München	Real Madrid	16.0	1.000
28	Manchester City	CD Olimpia	15.5	0.991
	Dinamo Zagreb	Benfica	16.0	1.000
	SE Palmeiras	Paris Saint-Germain	16.0	1.000
	RB Salzburg	Qarabag FK	16.0	1.000
	Juventus	TP Mazembe	16.0	1.000
	Al Ahly	Auckland City FC	15.0	0.966
	PSV Eindhoven	Ulsan Hyundai FC	16.0	1.000
	LDU Quito	UANL Tigres	13.5	0.793
	Johor FC	Melbourne Heart FC	17.5	0.924
	Juventus	Real Madrid	16.0	1.000
29	Auckland City FC	Bayern München	21.5	0.924

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Paris Saint-Germain	Manchester City	16.0	1.000
	Johor FC	Dinamo Zagreb	17.5	0.924
	Benfica	Ulsan Hyundai FC	16.0	1.000
	UANL Tigres	SE Palmeiras	15.0	0.966
	CD Olimpia	RB Salzburg	13.0	1.000
	LDU Quito	Al Ahly	14.5	0.924
	TP Mazembe	PSV Eindhoven	16.0	1.000
	Melbourne Heart FC	Qarabag FK	20.0	0.866
	Real Madrid	Auckland City FC	14.5	0.924
30	Johor FC	Bayern München	16.0	1.000
	Manchester City	Melbourne Heart FC	13.5	0.793
	Dinamo Zagreb	Paris Saint-Germain	16.0	1.000
	Benfica	CD Olimpia	16.0	1.000
	SE Palmeiras	Al Ahly	16.0	1.000
	LDU Quito	RB Salzburg	16.0	1.000
	PSV Eindhoven	Juventus	16.0	1.000
	UANL Tigres	Qarabag FK	12.5	0.609
	TP Mazembe	Ulsan Hyundai FC	16.0	1.000
	RB Salzburg	Real Madrid	16.0	1.000
31	Bayern München	Ulsan Hyundai FC	14.0	0.866
	SE Palmeiras	Manchester City	15.5	0.991
	Dinamo Zagreb	Juventus	16.0	1.000
	Paris Saint-Germain	Auckland City FC	12.5	0.609
	Benfica	TP Mazembe	16.0	1.000
	UANL Tigres	Al Ahly	9.0	0.707
	PSV Eindhoven	Melbourne Heart FC	16.0	1.000
	Qarabag FK	CD Olimpia	15.5	0.991
	LDU Quito	Johor FC	8.5	0.130

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Bayern München	Real Madrid	16.0	1.000
32	Manchester City	CD Olimpia	15.5	0.991
	Dinamo Zagreb	Benfica	16.0	1.000
	SE Palmeiras	Paris Saint-Germain	16.0	1.000
	RB Salzburg	Qarabag FK	16.0	1.000
	Juventus	TP Mazembe	16.0	1.000
	Al Ahly	Auckland City FC	15.0	0.966
	PSV Eindhoven	Ulsan Hyundai FC	16.0	1.000
	LDU Quito	UANL Tigres	13.5	0.793
	Johor FC	Melbourne Heart FC	17.5	0.924
	Juventus	Real Madrid	16.0	1.000
33	Auckland City FC	Bayern München	21.5	0.924
	Paris Saint-Germain	Manchester City	16.0	1.000
	Johor FC	Dinamo Zagreb	17.5	0.924
	Benfica	Ulsan Hyundai FC	16.0	1.000
	UANL Tigres	SE Palmeiras	15.0	0.966
	CD Olimpia	RB Salzburg	13.0	1.000
	LDU Quito	Al Ahly	14.5	0.924
	TP Mazembe	PSV Eindhoven	16.0	1.000
	Melbourne Heart FC	Qarabag FK	20.0	0.866
	Real Madrid	Auckland City FC	14.5	0.924
34	Johor FC	Bayern München	16.0	1.000
	Manchester City	Melbourne Heart FC	13.5	0.793
	Dinamo Zagreb	Paris Saint-Germain	16.0	1.000
	Benfica	CD Olimpia	16.0	1.000
	SE Palmeiras	Al Ahly	16.0	1.000
	LDU Quito	RB Salzburg	16.0	1.000
	PSV Eindhoven	Juventus	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	UANL Tigres	Qarabag FK	12.5	0.609
	TP Mazembe	Ulsan Hyundai FC	16.0	1.000
	RB Salzburg	Real Madrid	16.0	1.000
35	Bayern München	Ulsan Hyundai FC	14.0	0.866
	SE Palmeiras	Manchester City	15.5	0.991
	Dinamo Zagreb	Juventus	16.0	1.000
	Paris Saint-Germain	Auckland City FC	12.5	0.609
	Benfica	TP Mazembe	16.0	1.000
	UANL Tigres	Al Ahly	9.0	0.707
	PSV Eindhoven	Melbourne Heart FC	16.0	1.000
	Qarabag FK	CD Olimpia	15.5	0.991
	LDU Quito	Johor FC	8.5	0.130
	Bayern München	Real Madrid	16.0	1.000
36	Manchester City	CD Olimpia	15.5	0.991
	Dinamo Zagreb	Benfica	16.0	1.000
	SE Palmeiras	Paris Saint-Germain	16.0	1.000
	RB Salzburg	Qarabag FK	16.0	1.000
	Juventus	TP Mazembe	16.0	1.000
	Al Ahly	Auckland City FC	15.0	0.966
	PSV Eindhoven	Ulsan Hyundai FC	16.0	1.000
	LDU Quito	UANL Tigres	13.5	0.793
	Johor FC	Melbourne Heart FC	17.5	0.924
	Juventus	Real Madrid	16.0	1.000
37	Auckland City FC	Bayern München	21.5	0.924
	Paris Saint-Germain	Manchester City	16.0	1.000
	Johor FC	Dinamo Zagreb	17.5	0.924
	Benfica	Ulsan Hyundai FC	16.0	1.000
	UANL Tigres	SE Palmeiras	15.0	0.966

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	CD Olimpia	RB Salzburg	13.0	1.000
	LDU Quito	Al Ahly	14.5	0.924
	TP Mazembe	PSV Eindhoven	16.0	1.000
	Melbourne Heart FC	Qarabag FK	20.0	0.866
	Real Madrid	Auckland City FC	14.5	0.924
38	Johor FC	Bayern München	16.0	1.000
	Manchester City	Melbourne Heart FC	13.5	0.793
	Dinamo Zagreb	Paris Saint-Germain	16.0	1.000
	Benfica	CD Olimpia	16.0	1.000
	SE Palmeiras	Al Ahly	16.0	1.000
	LDU Quito	RB Salzburg	16.0	1.000
	PSV Eindhoven	Juventus	16.0	1.000
	UANL Tigres	Qarabag FK	12.5	0.609
	TP Mazembe	Ulsan Hyundai FC	16.0	1.000
	RB Salzburg	Auckland City FC	16.0	1.000

D.3 Match Times Equality Scores

Team Name	Equality Score
Real Madrid	0.962
Bayern München	0.962
Manchester City	0.971
Dinamo Zagreb	0.951
Paris Saint-Germain	0.962
Benfica	0.981
SE Palmeiras	0.955
RB Salzburg	0.961
Juventus	0.956
Al Ahly	0.957
PSV Eindhoven	0.928
Qarabag FK	0.967
TP Mazembe	0.958
LDU Quito	0.883
Johor FC	0.900
Ulsan Hyundai FC	0.893
Auckland City FC	0.825
UANL Tigres	0.853
CD Olimpia	0.872
Melbourne Heart FC	0.848

E Calculating probabilities

E.1 Probability of Leader Winning After X round

```
import numpy as np
from scipy.stats import norm
import matplotlib.pyplot as plt
import random

# Scores after certain rounds
scores_list = [[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
```

```
                [2.8311, 2.7477, 2.6775, 0.2523, 2.946, 0.3225, 2.5881, 0.1689,
   2.9379, 2.8905, 1.656, 2.475, 1.344, 0.4119, 0.525, 1.7229, 1.2771, 0.1095,
   0.054, 0.0621],
 [5.8245, 5.5107, 5.4807, 1.77, 5.6778, 1.8048, 2.8251, 1.7664, 4.4436, 4.3848,
   3.0585, 4.9287, 1.5408, 0.9582, 3.219, 1.9911, 3.7332, 0.1161, 0.5979,
   0.3681],
[8.6151, 8.5038, 8.433, 3.6972, 8.4723, 2.0142, 5.535, 3.3426, 6.9234, 5.8086,
   4.6374, 6.3498, 2.6136, 1.4784, 3.2667, 2.2812, 3.9387, 0.123, 2.1513,
   1.8147],
 [11.4078, 11.478, 11.1801, 5.4477, 8.6796, 3.9057, 8.448, 4.884, 8.382, 6.0615,
   5.8869, 8.8086, 3.7221, 4.1739, 3.8079, 2.307, 6.3945, 0.21, 2.4558, 2.3589],
[13.2315, 14.2827, 12.3564, 6.9819, 10.1454, 5.6433, 11.3946, 6.6162, 10.9194,
   7.3239, 8.6055, 9.0039, 4.9899, 5.9796, 4.2705, 3.5013, 6.6759, 2.1024,
   2.5092, 3.4665],
 [16.0092, 17.0901, 15.0972, 7.2042, 13.0965, 7.446, 14.3079, 6.8754, 12.6129,
   10.2606, 8.7981, 10.2012, 6.2964, 7.5624, 5.6877, 5.229, 7.9482, 2.1891,
   2.5581, 3.5298],
 [17.8497, 20.0841, 16.2567, 9.951, 14.8377, 10.3968, 17.0913, 8.4375, 14.2149,
   11.6985, 10.0569, 11.5992, 8.7312, 10.0956, 6.2529, 5.4822, 8.1648, 2.6559,
   2.6073, 3.5358],
 [20.6433, 22.8264, 19.2459, 12.9099, 16.7376, 10.6545, 17.2977, 10.9476,
   17.1534, 13.3005, 12.7164, 12.9972, 9.8313, 10.5855, 8.2356, 5.8227, 9.1821,
   2.6667, 2.6688, 3.5769],
 [23.5203, 25.8177, 22.2393, 14.4993, 18.4881, 12.3189, 18.7083, 12.2832,
   19.6497, 15.9525, 13.9659, 15.9306, 9.9543, 11.0892, 10.1952, 6.1707, 10.2225,
    2.6754, 2.7354, 3.5835],
 [26.5173, 28.6578, 25.0134, 16.0095, 21.4467, 13.8087, 20.3223, 13.6692,
   22.1766, 18.4659, 16.6182, 16.1565, 10.1142, 11.5758, 10.6683, 6.5184, 12.429,
    3.4689, 2.7384, 3.6249],
 [29.4894, 30.3327, 26.3385, 17.5419, 22.9143, 15.5004, 21.9354, 15.0561,
   23.4849, 21.357, 19.5495, 18.7635, 12.486, 12.204, 10.6962, 6.9114, 14.8728,
   3.5778, 3.2946, 3.6936],
 [32.3244, 33.0885, 29.3049, 20.1915, 23.1585, 17.0571, 23.3787, 17.8014,
   23.6499, 23.0148, 22.0503, 21.696, 13.8282, 12.7032, 11.0466, 6.945, 15.1275,
   5.4411, 4.4313, 3.7611],
 [33.9648, 34.4481, 32.0508, 23.1513, 24.8745, 19.7886, 25.1151, 20.3613, 26.544,
    24.2988, 22.3044, 22.9596, 16.7463, 14.7198, 11.4867, 7.2135, 16.1109, 5.547,
    4.5132, 3.8013],
 [36.8133, 37.4262, 34.7229, 25.9026, 25.2024, 21.6855, 26.7927, 23.3037,
   27.8664, 26.8353, 25.1907, 23.1111, 17.8494, 17.427, 11.9502, 7.4622, 16.1328,
    5.6607, 4.5708, 4.0941],
 [38.4567, 38.7828, 37.461, 27.6579, 27.8166, 23.4168, 28.5048, 23.5656, 29.1111,
    28.104, 26.4786, 25.8087, 20.7138, 17.8128, 13.6926, 8.7198, 16.4352, 5.7963,
    6.138, 5.5269],
 [41.3247, 41.7435, 40.4367, 29.4066, 29.3268, 24.9066, 30.267, 25.1151, 30.5616,
    29.3553, 29.4147, 27.0465, 20.8458, 20.511, 13.7319, 11.0952, 16.4595,
   6.4209, 6.4398, 5.5908],
 [44.3052, 44.5674, 43.3845, 32.3313, 31.0608, 27.8571, 33.216, 26.8656, 31.8276,
    32.0817, 31.9392, 27.2226, 22.0953, 20.5632, 14.2074, 11.1147, 16.7331,
   6.4962, 6.4893, 5.6418],
 [47.2692, 47.367, 46.0419, 32.6739, 32.793, 30.7785, 34.8942, 29.5266, 32.028,
   33.3495, 33.261, 28.8207, 23.4972, 20.5992, 16.8852, 11.4537, 19.1787, 6.5748,
    6.8115, 6.1962],
[50.2638, 50.1798, 48.7482, 34.4517, 35.5983, 33.3963, 35.1879, 32.4693, 33.6399,
    33.5367, 34.4832, 30.2088, 26.064, 20.9814, 19.5759, 11.8869, 19.3734,
   6.5802, 7.1208, 6.2535],
 [53.1036, 53.1747, 51.4029, 34.797, 38.5476, 35.0835, 36.8631, 33.7821, 35.3454,
    34.8615, 34.6434, 32.8263, 27.3585, 22.5429, 21.0144, 12.2694, 21.5481,
   7.4055, 7.1259, 6.3042],
[56.0829, 55.9794, 54.0798, 37.4463, 40.3512, 35.4066, 38.7267, 36.3276, 36.8823,
    35.0568, 36.1065, 34.0227, 28.4949, 22.9974, 21.3651, 14.8155, 21.5688,
   9.3123, 7.5798, 7.3974],
 [58.8636, 58.716, 57.0747, 37.7097, 40.5705, 37.0992, 41.3001, 39.0765, 38.1897,
```

```
    36.6336, 37.7763, 35.4459, 29.8251, 23.424, 24.0357, 17.1576, 21.8199,
   9.9702, 7.5849, 7.7268],
 [61.6926, 61.5666, 60.0459, 39.2871, 43.1943, 39.6978, 42.7227, 40.7247,
   38.3607, 38.1231, 39.2868, 36.7977, 29.9745, 23.8254, 24.4119, 19.7247,
   21.8487, 11.8314, 8.7237, 8.1597],
 [64.6734, 64.368, 62.9925, 42.2436, 44.7348, 42.4974, 44.1822, 43.6293, 38.5593,
    41.0589, 41.8026, 38.3574, 31.4148, 23.8788, 24.8961, 19.7439, 22.0491,
   11.9268, 8.7672, 8.2239],
 [67.6701, 67.1943, 65.7363, 45.1731, 46.4073, 44.3097, 46.0137, 44.9568,
   38.8155, 43.9968, 41.9763, 39.5451, 32.5833, 25.8624, 26.6181, 21.0219,
   23.0655, 11.9973, 8.8293, 8.2272],
 [70.5132, 69.9621, 68.4165, 48.1215, 46.7271, 46.0509, 46.2459, 47.5209,
   41.4663, 44.1537, 43.2351, 42.423, 35.2317, 28.575, 27.054, 21.3711, 23.4171,
   12.1194, 8.8809, 8.5146],
 [73.3416, 72.7512, 71.3793, 49.9548, 49.6443, 48.6723, 49.0353, 47.7318,
   44.4024, 46.6584, 43.4067, 43.5897, 38.1537, 29.0703, 27.4326, 21.4083,
   23.6277, 12.2022, 8.9448, 8.5926],
[76.3047, 75.7455, 74.1237, 51.6936, 51.5391, 50.2272, 50.4804, 49.2885, 44.658,
   47.9196, 44.85, 46.3059, 39.2589, 29.1072, 29.9406, 23.9976, 23.9115, 12.6942,
    8.9505, 9.0033],
 [79.164, 78.7029, 77.1114, 54.3108, 53.1078, 51.9609, 51.9117, 52.2141, 47.595,
   50.5839, 46.1163, 46.4466, 41.9379, 29.49, 29.9832, 24.3333, 24.2325, 12.7065,
    9.0249, 9.0663],
[82.008, 81.4404, 80.1063, 56.9358, 53.3703, 54.7506, 54.5193, 55.1145, 49.0947,
   50.7399, 47.6166, 49.3764, 44.487, 29.865, 30.3756, 24.7842, 24.4428, 12.8061,
    9.0951, 9.0714],
[84.8469, 84.3996, 83.0559, 59.7516, 54.8928, 56.2281, 57.4665, 55.2756, 51.7419,
    52.4043, 50.5479, 52.2585, 45.8226, 29.9058, 30.426, 25.137, 24.627, 12.924,
   9.1638, 9.1242],
 [87.6207, 87.3771, 85.8477, 59.9778, 57.5292, 59.1822, 59.1141, 56.9352, 54.63,
   53.7567, 53.0217, 53.5989, 46.0308, 30.432, 30.7896, 27.6933, 24.6495,
   13.0359, 9.6075, 9.1701],
 [90.6039, 90.3321, 88.6164, 61.914, 59.2344, 61.9167, 62.0595, 59.8833, 55.9248,
    55.2792, 54.4992, 53.8302, 47.0946, 30.477, 33.2853, 27.9588, 24.6663,
   13.5402, 9.6621, 9.222],
[93.3825, 93.3291, 91.3626, 63.5997, 61.0545, 62.1381, 64.7634, 61.1976, 58.6533,
    57.8052, 54.753, 55.0101, 50.007, 33.0192, 33.7593, 28.2549, 24.9378, 13.998,
    9.7497, 9.225],
 [96.381, 96.0621, 94.356, 65.3259, 63.6666, 62.4051, 67.3731, 63.8853, 59.9271,
   60.5343, 56.3103, 56.4528, 52.8804, 33.4071, 34.1496, 28.5672, 25.2087,
   14.1246, 9.7563, 9.2265],
 [99.3786, 99.0375, 97.101, 67.0281, 66.5904, 65.355, 69.0393, 65.1831, 61.2609,
   60.7893, 59.031, 58.8951, 55.3176, 33.9648, 34.7124, 28.5918, 25.488, 14.2008,
    9.7587, 9.2766],
[102.1665, 100.7262, 98.4123, 68.8695, 68.2341, 68.0049, 69.2514, 66.5394,
   63.9888, 63.7236, 61.9302, 60.0537, 58.2393, 35.7543, 35.0625, 29.8023,
   25.7601, 14.3016, 9.8244, 9.3549],
 [105.1398, 103.5081, 101.1276, 71.6811, 70.9863, 70.9293, 69.5361, 66.7575,
   65.5218, 65.1906, 64.8594, 62.9805, 60.6219, 36.3717, 35.0892, 30.0501,
   25.9485, 14.3772, 9.8952, 9.4281]]

#Elos of teams
elos = [2845.827448, 2808.935211, 2765.037686, 2407.793548, 2401.012962,
   2400.533903, 2387.270097, 2362.129444,
        2349.543101, 2348.058602, 2343.654686, 2326.412104, 2306.060436,
   2070.017405, 2055.383408, 1995.963647,
        1944.038018, 1774.099637, 1689.506583, 1678.791739]

#Function calculating win probability in a single game
def win_probability(elo_i, elo_j):
    return 1 / (1 + 10 ** ((elo_j - elo_i) / 400))

def expected_points_and_variance(elos, scores, x):
```

```
    n = len(elos)
    #Clear expected_points and variances
    expected_points = np.zeros(n)
    variances = np.zeros(n)
    gry = len(scores_list)
    q = 1 - x / gry

    # Calculate expected_points and variances for each team
    for i in range(n):
        for j in range(n):
            if i != j:
                p_win = win_probability(elos[i], elos[j])
                expected_points[i] += 3*q * p_win
                variances[i] += 9*q *q* p_win * (1 - p_win)
    for i in range(n):
        expected_points[i] += scores[i]

    return expected_points, variances

def probability_leader_wins(elos, scores, leader_index, x):
    #Obtain expected_points and variances
    expected_points, variances = expected_points_and_variance(elos, scores, x)
    n = len(elos)

    #Calculate the probability that leader will win
    probabilities = []
    for i in range(n):
        if i != leader_index:
            prob = 1 - norm.cdf(expected_points[leader_index], loc=
    expected_points[i], scale=np.sqrt(variances[i] + variances[leader_index]))
            probabilities.append(prob)
    total_prob = np.prod(1 - np.array(probabilities))
    return total_prob

#Calculate x values
x_values = list(range(0, len(scores_list)))

#Calculate probabilities for each x value
probabilities = [probability_leader_wins(elos, scores_list[i], leader_index=0, x=
    x_values[i]) for i in range(len(scores_list))]

# Plot the results
plt.figure(figsize=(10, 6))
plt.plot(x_values, probabilities, marker='o', linestyle='-', color='b', label="
    Probability that leader will win")
plt.xlabel("x (number of rounds already played)")
plt.ylabel("Probability that leader will win")
plt.legend()
plt.grid()
plt.show()
```

E.2 Probability of Leader Winning as a Function of Number of Round

```
import numpy as np
from scipy.stats import norm
import matplotlib.pyplot as plt

#Zeros as initial scores
scores_list = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

#Elos of the teams
elos = [2845.827448, 2808.935211, 2765.037686, 2407.793548, 2401.012962,
    2400.533903, 2387.270097, 2362.129444,
```

```
        2349.543101, 2348.058602, 2343.654686, 2326.412104, 2306.060436,
    2070.017405, 2055.383408, 1995.963647,
        1944.038018, 1774.099637, 1689.506583, 1678.791739]

#Function calculating win probability in a single game
def win_probability(elo_i, elo_j):
    return 1 / (1 + 10 ** ((elo_j - elo_i) / 400))

def expected_points_and_variance(elos, scores, x, gry):
    n = len(elos)
    #Clear expected_points and variances
    expected_points = np.zeros(n)
    variances = np.zeros(n)

    q = 1
    # Calculate expected_points and variances for each team
    for i in range(n):
        for j in range(n):
            if i != j:
                p_win = win_probability(elos[i], elos[j])
                expected_points[i] += 3*gry * p_win
                variances[i] += 9*gry*p_win * (1 - p_win)
    for i in range(n):
        expected_points[i] += scores[i]

    return expected_points, variances

def probability_leader_wins(elos, scores, leader_index, x, gry):
    #Obtain expected_points and variances
    expected_points, variances = expected_points_and_variance(elos, scores, x,
    gry)
    n = len(elos)

    #Calculate the probability that leader will win
    probabilities = []
    for i in range(n):
        if i != leader_index:
            prob = 1 - norm.cdf(expected_points[leader_index], loc=
    expected_points[i], scale=np.sqrt(variances[i] + variances[leader_index]))
            probabilities.append(prob)
    total_prob = np.prod(1 - np.array(probabilities))
    return total_prob

# Number of iterations
iter = 211

#Calculate the probabilities for different number of rounds
probabilities = [probability_leader_wins(elos, scores_list, leader_index=0, x=0,
    gry = i) for i in range(iter)]
gry = []

# Get the number of rounds
for i in range(iter):
    gry.append(i)

# Plot the results
plt.figure(figsize=(10, 6))
plt.plot(gry, probabilities, marker='o', linestyle='-', color='b', markersize =
    3, label="Probability that leader will win")
plt.xlabel("Number of all rounds")
plt.ylabel("Probability that leader will win")
plt.legend()
plt.grid()
```

```
plt.xlim(0, 210)
plt.ylim(0, 1.1)
plt.show()
```

F Balanced K-Means Schedule

F.1 Code

```
import json
import math
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from collections import Counter
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from scipy.spatial.distance import cdist
import pandas as pd
import plotly.graph_objects as go
from ortools.sat.python import cp_model

# Load JSON file
file_path = "clubs24.json" #"clubs.json" for 20 teams
with open(file_path, "r") as f:
    clubs = json.load(f)

# Earth's radius in kilometers
R = 6371

# Convert latitude and longitude to x, y, z
def lat_lon_to_xyz(lat, lon):
    lat_rad = math.radians(lat)
    lon_rad = math.radians(lon)

    x = R * math.cos(lat_rad) * math.cos(lon_rad)
    y = R * math.cos(lat_rad) * math.sin(lon_rad)
    z = R * math.sin(lat_rad)

    return x, y, z

# Compute coordinates for all clubs
x_vals, y_vals, z_vals, names, score, elo = [], [], [], [], [], []
for club in clubs:
    x, y, z = lat_lon_to_xyz(club["Latitude"], club["Longitude"])
    x_vals.append(x)
    y_vals.append(y)
    z_vals.append(z)
    names.append(club["Club"])
    score.append(club["ELO"])
    elo.append(club["ELO"])

#initialize helpful lists
single = np.array([])
multiple = np.array([])
none = np.array([])

data =  np.column_stack((x_vals, y_vals, z_vals, score))

cluster_holder = np.array([])
```

```

try:
    with open("teams.json", "r") as f:
        teamsData_pre = json.load(f)
except FileNotFoundError:
    print("Error: teams.json not found. Please create the input file.")
    exit()
except json.JSONDecodeError:
    print("Error: teams.json is not valid JSON.")
    exit()

print(teamsData_pre)

with open("data_dict.json", "r") as f:
    loaded_dict = json.load(f)


#cluster the teams
def perform_clustering(data, weights, num_clusters): #data, weights ([1,1,1,w]), number of clusters

    num_teams = len(data)
    num_features = len(weights) # x, y, z, score
    cluster_size = num_teams // num_clusters  # Ensures equal-sized clusters
    #number of clustering interations
    depth = 21

    #assign weights
    for i in range(len(weights)):
        for j in range(len(data)):
            data[j][i] = weights[i]*data[j][i]

    data_scaled = data

    # Run standard K-Means for initial clustering
    kmeans = KMeans(n_clusters=num_clusters, n_init=10, init="k-means++")
    initial_clusters = kmeans.fit_predict(data_scaled)

    # Compute distances of each point to centroids
    centroids = kmeans.cluster_centers_
    distances = cdist(data_scaled, centroids)  # Distance matrix

    #define distance between two points in our 4D space
    def haver_dist(point_a, point_b):
        a = point_a[3:]
        b = point_b[3:]

        return np.sqrt( (6371* np.arccos( (point_a[0]*point_b[0] + point_a[1]*point_b[1] + point_a[2]*point_b[2])/(6371*6371)     ))**2  + np.sum( (a-b)**2 )        )

    #find the centroid
    def haver_mean(points):
        radius = 6371
        points = np.asarray(points)

        # Compute the average of the first 3 coordinates and project onto the sphere.
        mean_xyz = np.mean(points[:, :3], axis=0)
        norm = np.linalg.norm(mean_xyz)

        if norm == 0:
            avg_xyz = np.zeros(3)
```

```
        else:
            avg_xyz = radius * mean_xyz / norm

        # Compute the mean for any remaining dimensions.
        if points.shape[1] > 3:
            avg_rest = np.mean(points[:, 3:], axis=0)
            return np.concatenate([avg_xyz, avg_rest])
        else:
            return avg_xyz

    ####################################
    # First clustering - warmup
    def balanced_kmeans(distances, num_clusters, cluster_size):

        d1 = np.min(distances, axis=1)  # Nearest centroid distances
        d2 = np.partition(distances, 1, axis=1)[:, 1]  # Second nearest centroid distances
        scores = (d2 - d1)  # Priority score: larger score means more uncertain point

        sorted_indices = np.argsort(scores)[::-1]  # Sort in decreasing order

        assigned_clusters = [-1] * len(distances)  # Placeholder (-1 means unassigned)
        cluster_counts = {i: 0 for i in range(num_clusters)}  # Track cluster sizes

        for i in sorted_indices:  # Process points in sorted order
            for cluster in np.argsort(distances[i]):  # Try the closest centroid first
                if cluster_counts[cluster] < cluster_size:  # If cluster isn't full
                    assigned_clusters[i] = cluster
                    cluster_counts[cluster] += 1
                    break  # Move to the next point once assigned

        return np.array(assigned_clusters)
    ################################

    # Apply Balanced K-Means Assignment
    balanced_clusters = balanced_kmeans(distances, num_clusters, cluster_size)

    #actual clustering alogirthm
    def calculate_centroids(points, cluster_numbers, cluster_size):

        num_clusters = np.max(cluster_numbers) + 1  # The number of clusters is one more than the maximum cluster number
        centroids = {}  # Dictionary to store centroids

        for cluster in range(num_clusters):
            # Get all points that belong to the current cluster
            cluster_points = points[cluster_numbers == cluster]

            # Calculate the centroid by taking the mean of the points in the cluster
            if len(cluster_points) > 0:  # Avoid empty clusters
                centroids[cluster] = haver_mean(cluster_points)

        centroid_array = np.array([centroids[i] for i in range(num_clusters)])

        centroid_distance = cdist(centroid_array, points, metric=haver_dist)  # distances to points 0, 1, ... 19
        #find 4 points closes
```

```
        closest_indices = np.argsort(centroid_distance, axis=1)[:, :cluster_size]

        global cluster_holder
        cluster_holder = closest_indices
        #print("CLOSEST INDICES", closest_indices)

        all_closest_points = closest_indices.flatten()
        # Count occurrences of each point index
        point_counts = Counter(all_closest_points)

        # Initialize lists of different types of points (relative to lister positions)
        only_one_centroid = np.zeros(num_teams, dtype=int)
        multiple_centroids = np.zeros(num_teams, dtype=int)
        none_centroid = np.zeros(num_teams, dtype=int)
        # Assign 1s to the appropriate lists
        for point, count in point_counts.items():
            if count == 1:
                only_one_centroid[point] = 1
            elif count > 1:
                multiple_centroids[point] = 1
        # Mark points that appear in none of the centroid lists
        all_points = set(range(num_teams))  # Assuming num_teams is the total number of points
        none_points = all_points - set(all_closest_points)
        for point in none_points:
            none_centroid[point] = 1

        global single; single = only_one_centroid
        global multiple; multiple = multiple_centroids
        global none; none = none_centroid

        # Compute the distances of each point to each centroid (haver)
        distances = cdist(points, centroid_array, metric=haver_dist)  # HAVER - DONE
        #print("DISTANCES", distances)

        #Distances also must be haver
        d1 = np.min(distances, axis=1)  # Nearest centroid distances
        d2 = np.partition(distances, 1, axis=1)[:, 1]  # Second nearest centroid distances
        #enusre the order of point assignment (instead of ifs we sort by number)
        scores = 10000000*only_one_centroid + 1000000*multiple_centroids + 100000*none_centroid +  (d2 - d1)  # Priority score: larger score means more uncertain point

        # Sort points based on score (d2 - d1), highest first
        sorted_indices = np.argsort(scores)[::-1]  # Sort in decreasing order

        assigned_clusters = [-1] * len(distances)  # Placeholder (-1 means unassigned)
        cluster_counts = {i: 0 for i in range(num_clusters)}  # Track cluster sizes

        for i in sorted_indices:  # Process points in sorted order
            #try the first-choice cluster first (if single)
            found = False
            if single[i]==1:
                for index, sublist in enumerate(closest_indices):
                    if i in sublist:
                        cluster = index
                        break  # Stop searching once found

```

```
                if cluster_counts[cluster] < cluster_size:  # If cluster isn't full
                    assigned_clusters[i] = cluster
                    cluster_counts[cluster] += 1
                    found = True # Move to the next point once assigned

            if found==True:
                continue
            else:

                for cluster in np.argsort(distances[i]):  # Try the closest centroid first
                    if cluster_counts[cluster] < cluster_size:  # If cluster isn't full
                        assigned_clusters[i] = cluster
                        cluster_counts[cluster] += 1
                        break  # Move to the next point once assigned

        return np.array(assigned_clusters)

    #perform Balanced K-means clustering
    for i in range(depth):
        balanced_clusters = calculate_centroids(data_scaled, balanced_clusters, cluster_size)

    for cluster in range(num_clusters):
            # Get all points that belong to the current cluster
            cluster_points = data_scaled[balanced_clusters == cluster]

            # Calculate the centroid by taking the haver mean of the points in the cluster
            if len(cluster_points) > 0:  # Avoid empty clusters
                #centroids[cluster] = np.mean(cluster_points, axis=0)
                centroids[cluster] = haver_mean(cluster_points)

    return [balanced_clusters, centroids]



from collections import defaultdict

#assign weights
weights = [1,1,1,0] #must be o for initial processing, or can be different
num_clusters = 6

#check if code works
clustered_teams = (perform_clustering(data, weights, num_clusters))

groups = clustered_teams[0]
print(groups)

grouped_indices = defaultdict(list)

# Iterate through the list and add the index of each element to the corresponding group
for index, value in enumerate(groups):
    grouped_indices[value].append(index)

# Convert to a list of lists
groups = list(grouped_indices.values())
print(groups)
teams =  np.column_stack((names, data, elo))

```

```
for i in range(len(teams)):
    teams[i][4] = 0
    teams[i][0] = str(teams[i][0])


teams = [[row[0]] + [float(x) for x in row[1:]] for row in teams]


def calc_dist(n_teams, teamIndices_pre):

    teamsData = [teamsData_pre[i] for i in teamIndices_pre]

    print(teamsData)

    # --- League Parameters derived from data ---
    teamsQuantity = n_teams
    if teamsQuantity % 2 != 0:
        raise ValueError("teamsQuantity must be even.")
    seasonWeeksQuantity = (teamsQuantity - 1) * 2 # Weeks needed for DRR

    # --- Scheduling Constraints ---
    # Set realistic streak limits (e.g., 3 or 4) - CP-SAT can handle tighter constraints
    maximumConsecutiveHomeGames = 20
    maximumConsecutiveAwayGames = 20
    # maximumConsecutiveHomeGames = seasonWeeksQuantity # Use high values to effectively disable
    # maximumConsecutiveAwayGames = seasonWeeksQuantity

    # --- Solver Parameters ---
    # CP-SAT Time Limit for finding first feasible solution
    CP_FEASIBILITY_TIME_LIMIT = 60.0 # Seconds

    # Simulated Annealing Parameters (Tune these)
    SA_INITIAL_TEMP = 10000.0 # Initial temperature - depends heavily on distance scale
    SA_COOLING_RATE = 0.995   # Cooling rate (closer to 1 means slower cooling)
    SA_MAX_ITERATIONS = 30000 # Increase for larger N or more thorough search

    # --- Prepare Data Structures ---
    # List of team names
    teamNames = [team['name'] for team in teamsData]
    # List of team indices [0, 1, ..., N-1]
    teamIndices = list(range(teamsQuantity))
    # Dictionary mapping team index to its location (latitude, longitude)
    teamLocations = {i: (team['latitude'], team['longitude']) for i, team in enumerate(teamsData)}

    # Week indices
    weeks_1_based = list(range(1, seasonWeeksQuantity + 1))
    weeks_0_based = list(range(seasonWeeksQuantity))
    weeks_minus_1_1_based = list(range(1, seasonWeeksQuantity)) # For SA verification if needed

    # --- Distance Calculation (Haversine Formula) ---
    def calculate_haversine_distance(lat1, lon1, lat2, lon2):
        """Calculates the great-circle distance between two points on the earth.
    """
        if (lat1, lon1) == (lat2, lon2):
            return 0.0

        R = 6371.0  # Earth radius in kilometers

```

```
        lat1_rad = math.radians(lat1)
        lon1_rad = math.radians(lon1)
        lat2_rad = math.radians(lat2)
        lon2_rad = math.radians(lon2)

        dlon = lon2_rad - lon1_rad
        dlat = lat2_rad - lat1_rad

        a = math.sin(dlat / 2)**2 + math.cos(lat1_rad) * math.cos(lat2_rad) *
    math.sin(dlon / 2)**2
        c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))

        distance = R * c
        return distance

    # Pre-compute Distance Matrix
    # distanceMatrix[i][j] = distance from team i's location to team j's location
    distanceMatrix = [[0.0] * teamsQuantity for _ in range(teamsQuantity)]
    for i in teamIndices:
        for j in teamIndices:
            if i != j:
                lat1, lon1 = teamLocations[i]
                lat2, lon2 = teamLocations[j]
                distanceMatrix[i][j] = calculate_haversine_distance(lat1, lon1,
    lat2, lon2)

    #print(f"Loaded {teamsQuantity} teams. Season length: {seasonWeeksQuantity}
    weeks.")
    #print(f"Max consecutive home/away games: {maximumConsecutiveHomeGames}/{
    maximumConsecutiveAwayGames}")

    # --- 2. Helper Functions ---

    def get_team_location_for_week(schedule, team_idx, week_1_based):
        """Finds the location index where team_idx plays in a given week (1-based
    )."""
        if week_1_based not in schedule or not schedule[week_1_based]:
            return None
        for home_team_idx, away_team_idx in schedule[week_1_based]:
            if home_team_idx == team_idx: return home_team_idx # Home game,
    location is own city index
            if away_team_idx == team_idx: return home_team_idx # Away game,
    location is opponent's city index
        return None # Team not found (should not happen in a valid schedule)

    def calculate_schedule_total_travel(schedule, dist_matrix, num_teams,
    num_weeks):
        """Calculates the total travel distance for a complete schedule (1-based
    weeks)."""
        total_distance = 0.0
        indices = list(range(num_teams))

        for k in indices:
            team_travel = 0.0
            current_loc_idx = k # Start at team k's home location index

            # Travel to first game (week 1)
            first_game_loc_idx = get_team_location_for_week(schedule, k, 1)
            if first_game_loc_idx is None: return float('inf') # Incomplete/
    invalid schedule
            team_travel += dist_matrix[current_loc_idx][first_game_loc_idx]
            current_loc_idx = first_game_loc_idx

```

```
            # Travel between games (week w location to week w+1 location)
            for w in range(1, num_weeks): # w goes from 1 to M-1
                next_loc_idx = get_team_location_for_week(schedule, k, w + 1) # Location for week w+1
                if next_loc_idx is None: return float('inf')
                team_travel += dist_matrix[current_loc_idx][next_loc_idx]
                current_loc_idx = next_loc_idx # Update location for next segment

            # Travel back home after last game (week M)
            team_travel += dist_matrix[current_loc_idx][k] # Travel from last game loc back to home k
            total_distance += team_travel

        return total_distance

    def check_schedule_feasibility(schedule, num_teams, num_weeks, max_home, max_away, verbose=True):
        """Checks if a schedule (1-based weeks) satisfies DRR and streaks."""
        team_indices = list(range(num_teams))
        weeks = list(range(1, num_weeks + 1))
        errors = []
        # Check structure
        for w in weeks:
            if w not in schedule or len(schedule.get(w, [])) != num_teams // 2:
                errors.append(f"Feasibility Error: Week {w} incomplete/missing.")
                if verbose: print("\n".join(errors)); return False
                return False
        # Check DRR
        required = set((i, j) for i in team_indices for j in team_indices if i != j)
        played = set()
        teams_per_week = {w: set() for w in weeks}
        for w in weeks:
            for h, a in schedule[w]:
                if (h,a) in played: errors.append(f"DRR Error: Match ({h},{a}) repeated.")
                played.add((h,a))
                if h in teams_per_week[w] or a in teams_per_week[w]: errors.append(f"DRR Error: Team in multiple games week {w}.")
                teams_per_week[w].add(h); teams_per_week[w].add(a)
        if played != required: errors.append(f"DRR Error: Mismatch. Missing={len(required-played)}, Extra={len(played-required)}")
        for w in weeks:
            if len(teams_per_week[w]) != num_teams: errors.append(f"DRR Error: Week {w} has {len(teams_per_week[w])} teams.")
        # Check Streaks
        for k in team_indices:
            ch, ca = 0, 0
            for w in weeks:
                is_home, is_playing = False, False
                for h, a in schedule[w]:
                    if k == h: is_home, is_playing = True, True; break
                    if k == a: is_home, is_playing = False, True; break
                if not is_playing: errors.append(f"Streak Error: Team {k} not playing week {w}"); continue
                if is_home:
                    ch+=1; ca=0
                    if ch > max_home: errors.append(f"Streak Error: Team {k} >{max_home} home end W{w}.")
                else:
                    ca+=1; ch=0
                    if ca > max_away: errors.append(f"Streak Error: Team {k} >{max_away} away end W{w}.")
```

```
        # Result
        if errors:
            if verbose: print("Feasibility Check Failed:"); [print(f"  - {e}") for e in errors]
            return False
        return True

    # --- 3. Phase 1: Find First Feasible Schedule using CP-SAT ---

    #print("\n--- Phase 1: Finding First Feasible Schedule (CP-SAT) ---")
    start_phase1 = time.time()

    cp_model_instance = cp_model.CpModel()

    # --- CP-SAT Variables ---
    opponent_vars = {} # opponent_vars[k, w_0based] -> opponent index
    for k in teamIndices:
        for w in weeks_0_based:
            opponent_vars[k, w] = cp_model_instance.NewIntVar(0, teamsQuantity - 1, f"opponent_{k}_w{w}")
            cp_model_instance.Add(opponent_vars[k, w] != k)

    is_home_vars = {} # is_home_vars[k, w_0based] -> 0 (away) or 1 (home)
    for k in teamIndices:
        for w in weeks_0_based:
            is_home_vars[k, w] = cp_model_instance.NewBoolVar(f"is_home_{k}_w{w}")

    # --- CP-SAT Constraints ---

    # Constraint 1: Game Pairing / Symmetry
    all_teams_list = list(range(teamsQuantity))
    for w in weeks_0_based:
        for k in teamIndices:
            opp_k_w = opponent_vars[k, w]
            opponent_at_opp_k_w = cp_model_instance.NewIntVar(0, teamsQuantity - 1, f"opp_at_opp_{k}_w{w}")
            cp_model_instance.AddElement(opp_k_w, [opponent_vars[j, w] for j in all_teams_list], opponent_at_opp_k_w)
            cp_model_instance.Add(opponent_at_opp_k_w == k) # opponent[opponent[k,w]][w] == k

            is_home_opp_k_w = cp_model_instance.NewBoolVar(f"is_home_opp_{k}_w{w}")
            cp_model_instance.AddElement(opp_k_w, [is_home_vars[j,w] for j in all_teams_list], is_home_opp_k_w)
            cp_model_instance.Add(is_home_vars[k, w] != is_home_opp_k_w) # is_home[k,w] + is_home[opponent[k,w]][w] == 1

    # Constraint 2: Double Round-Robin Fulfillment
    for i in teamIndices:
        for j in teamIndices:
            if i == j: continue
            plays_home_ij_w_vars = []
            for w in weeks_0_based:
                lit_opp = cp_model_instance.NewBoolVar(f'lit_opp_{i}_{j}_w{w}')
                cp_model_instance.Add(opponent_vars[i, w] == j).OnlyEnforceIf(lit_opp)
                cp_model_instance.Add(opponent_vars[i, w] != j).OnlyEnforceIf(lit_opp.Not())
                target_and_var = cp_model_instance.NewBoolVar(f"plays_home_{i}_{j}_w{w}")
                cp_model_instance.AddBoolAnd([lit_opp, is_home_vars[i, w]]).
```

```
    OnlyEnforceIf(target_and_var)
493                 cp_model_instance.AddBoolOr([lit_opp.Not(), is_home_vars[i, w].
    Not()]).OnlyEnforceIf(target_and_var.Not())
494                 plays_home_ij_w_vars.append(target_and_var)
495             cp_model_instance.Add(sum(plays_home_ij_w_vars) == 1) # i hosts j
    exactly once
496
497     # Constraint 3: Home/Away Streak Limits
498     for k in teamIndices:
499         if maximumConsecutiveHomeGames < seasonWeeksQuantity:
500             for w_start in range(seasonWeeksQuantity -
    maximumConsecutiveHomeGames):
501                 window = [is_home_vars[k, w] for w in range(w_start, w_start +
    maximumConsecutiveHomeGames + 1)]
502                 cp_model_instance.Add(sum(window) <= maximumConsecutiveHomeGames)
503         if maximumConsecutiveAwayGames < seasonWeeksQuantity:
504             for w_start in range(seasonWeeksQuantity -
    maximumConsecutiveAwayGames):
505                 window = [is_home_vars[k, w].Not() for w in range(w_start,
    w_start + maximumConsecutiveAwayGames + 1)]
506                 cp_model_instance.Add(sum(window) <= maximumConsecutiveAwayGames)
507
508
509     # --- CP-SAT Solution Callback ---
510     class FeasibleSolutionFinderCallback(cp_model.CpSolverSolutionCallback):
511         def __init__(self, opponent_vars, is_home_vars, num_teams, num_weeks):
512             cp_model.CpSolverSolutionCallback.__init__(self)
513             self._opponent_vars = opponent_vars
514             self._is_home_vars = is_home_vars
515             self._num_teams = num_teams
516             self._num_weeks = num_weeks
517             self.solution_schedule = None
518             self.solution_count = 0
519
520         def OnSolutionCallback(self):
521             self.solution_count += 1
522             #print(f"Feasible solution #{self.solution_count} found by CP-SAT.
    Stopping search.")
523             # Reconstruct schedule: {week_1_based: [(home_idx, away_idx), ...]}
524             schedule = {w + 1: [] for w in range(self._num_weeks)}
525             processed_in_week = {w: set() for w in range(self._num_weeks)}
526             for w_0based in range(self._num_weeks):
527                 w_1based = w_0based + 1
528                 for k in range(self._num_teams):
529                     if k not in processed_in_week[w_0based]:
530                         opp_k = self.Value(self._opponent_vars[k, w_0based])
531                         is_k_home = self.Value(self._is_home_vars[k, w_0based])
532                         if is_k_home: h, a = k, opp_k
533                         else:       h, a = opp_k, k
534                         schedule[w_1based].append((h, a))
535                         processed_in_week[w_0based].add(h)
536                         processed_in_week[w_0based].add(a)
537             self.solution_schedule = schedule
538             self.StopSearch()
539
540         def get_schedule(self):
541             return self.solution_schedule
542
543     # --- Solve with CP-SAT ---
544     cp_solver = cp_model.CpSolver()
545     cp_solver.parameters.max_time_in_seconds = CP_FEASIBILITY_TIME_LIMIT
546     # cp_solver.parameters.log_search_progress = True # Enable for debugging
547
```

```
    solution_finder = FeasibleSolutionFinderCallback(opponent_vars, is_home_vars, teamsQuantity, seasonWeeksQuantity)
    status = cp_solver.Solve(cp_model_instance, solution_finder)

    end_phase1 = time.time()
    #print(f"Phase 1 Time (CP-SAT Feasibility Search): {end_phase1 - start_phase1:.2f} seconds")
    #print(f"CP-SAT Solver Status: {cp_solver.StatusName(status)}")

    feasible_schedule_phase1 = solution_finder.get_schedule()

    if feasible_schedule_phase1 is None:
        # Handle failure based on status
        if status == cp_model.INFEASIBLE: print("\nCP-SAT determined INFEASIBLE constraints.")
        elif status == cp_model.MODEL_INVALID: print("\nCP-SAT reported INVALID model.")
        else: print(f"\nCP-SAT failed to find a feasible solution (Status: {cp_solver.StatusName(status)}).")
        #print("Cannot proceed to Phase 2.")
        exit()

    #print("Verifying Phase 1 schedule feasibility...")
    if not check_schedule_feasibility(feasible_schedule_phase1, teamsQuantity, seasonWeeksQuantity, maximumConsecutiveHomeGames, maximumConsecutiveAwayGames, verbose=True):
        print("Error: CP-SAT schedule failed feasibility check!")
        exit()
    #print("Phase 1 schedule (from CP-SAT) is FEASIBLE.")

    initial_travel_cost = calculate_schedule_total_travel(feasible_schedule_phase1, distanceMatrix, teamsQuantity, seasonWeeksQuantity)
    if initial_travel_cost == float('inf'):
        #print("Error calculating initial travel cost.")
        exit()
    # Clarification: This is the cost of the *first* solution found, not a theoretical lower bound.
    #print(f"Initial Feasible Schedule Total Travel: {initial_travel_cost:,.2f} km")


    # --- 4. Phase 2: Improve Travel Distance (Simulated Annealing) ---
    #print("\n--- Phase 2: Improving Schedule (Simulated Annealing) ---")
    start_phase2 = time.time()

    # Start SA from the feasible schedule found by CP-SAT
    current_schedule = copy.deepcopy(feasible_schedule_phase1)
    current_cost = initial_travel_cost
    best_schedule = copy.deepcopy(feasible_schedule_phase1)
    best_cost = initial_travel_cost

    T = SA_INITIAL_TEMP
    alpha = SA_COOLING_RATE
    accepted_moves = 0
    infeasible_neighbors_skipped = 0

    for i in range(SA_MAX_ITERATIONS):
        neighbor_schedule = copy.deepcopy(current_schedule)
        # Choose move type (add more sophisticated moves for better results)
        move_type = random.choice(["SwapHomes", "SwapRounds"])
        move_successful = False

        if move_type == "SwapHomes" and seasonWeeksQuantity > 0:
```

```
            w = random.choice(weeks_1_based)
            if w not in neighbor_schedule: continue # Skip if week somehow missing
            home_teams_in_week = [h for h, a in neighbor_schedule.get(w, [])]
            if len(home_teams_in_week) >= 2:
                t1, t2 = random.sample(home_teams_in_week, 2)
                match1 = next(((h, a) for h, a in neighbor_schedule[w] if h == t1), None)
                match2 = next(((h, a) for h, a in neighbor_schedule[w] if h == t2), None)
                if match1 and match2:
                    opp1, opp2 = match1[1], match2[1]
                    neighbor_schedule[w].remove(match1); neighbor_schedule[w].remove(match2)
                    neighbor_schedule[w].append((t1, opp2)); neighbor_schedule[w].append((t2, opp1))
                    move_successful = True

        elif move_type == "SwapRounds" and seasonWeeksQuantity >= 2:
            w1, w2 = random.sample(weeks_1_based, 2)
            if w1 in neighbor_schedule and w2 in neighbor_schedule:
                neighbor_schedule[w1], neighbor_schedule[w2] = neighbor_schedule[w2], neighbor_schedule[w1]
                move_successful = True

        if not move_successful: continue

        # Check Feasibility (use verbose=False for speed)
        if not check_schedule_feasibility(neighbor_schedule, teamsQuantity, seasonWeeksQuantity, maximumConsecutiveHomeGames, maximumConsecutiveAwayGames, verbose=False):
            infeasible_neighbors_skipped += 1
            continue

        neighbor_cost = calculate_schedule_total_travel(neighbor_schedule, distanceMatrix, teamsQuantity, seasonWeeksQuantity)
        if neighbor_cost == float('inf'): continue # Skip if calculation failed

        delta_cost = neighbor_cost - current_cost
        # Acceptance Criteria (Metropolis criterion)
        if delta_cost < 0 or (T > 1e-9 and random.random() < math.exp(-delta_cost / T)):
            accepted_moves += 1
            current_schedule = neighbor_schedule # Keep the copy
            current_cost = neighbor_cost
            if current_cost < best_cost:
                best_schedule = copy.deepcopy(current_schedule) # Store the absolute best
                best_cost = current_cost
                # print(f"Iter {i+1}: New best cost: {best_cost:,.2f}") # Optional

        T *= alpha # Cool down

    end_phase2 = time.time()
    #print(f"Phase 2 Time: {end_phase2 - start_phase2:.2f} seconds")
    #print(f"Phase 2: Accepted {accepted_moves} moves, skipped {infeasible_neighbors_skipped} infeasible neighbors.")
    #print(f"Final Optimized Schedule Total Travel: {best_cost:,.2f} km")

    # --- 5. Display Results ---
    #final_schedule_result = best_schedule
    final_travel_cost_result = best_cost
```

```
651
652     return final_travel_cost_result
653
654     print("-" * 30)
655     print("Final Optimized Schedule:")
656     for w in weeks_1_based:
657         print(f"\n--- Week {w} ---")
658         if w in final_schedule_result:
659             for home_idx, away_idx in final_schedule_result[w]: print(f"  {
    teamNames[home_idx]} vs {teamNames[away_idx]}")
660         else: print("  (Week missing from final schedule - Error!)")
661
662     print("\n" + "-" * 30); print("Team Itineraries & Travel Verification:")
663     per_team_travel = {k: 0.0 for k in teamIndices}
664     verification_possible = True
665     for k in teamIndices:
666         # print(f"\n{teamNames[k]} Itinerary:") # Can make output verbose if
    needed
667         current_loc_idx = k
668         try:
669             loc1 = get_team_location_for_week(final_schedule_result, k, 1);
    assert loc1 is not None
670             travel = distanceMatrix[current_loc_idx][loc1]; per_team_travel[k] +=
     travel
671             current_loc_idx = loc1
672             for w in range(1, seasonWeeksQuantity):
673                 loc_next = get_team_location_for_week(final_schedule_result, k, w
    +1); assert loc_next is not None
674                 travel = distanceMatrix[current_loc_idx][loc_next];
    per_team_travel[k] += travel
675                 current_loc_idx = loc_next
676             travel = distanceMatrix[current_loc_idx][k]; per_team_travel[k] +=
    travel # Return home
677             # print(f"  Total Travel for {teamNames[k]}: {per_team_travel[k]:,.2f
    } km") # Verbose
678         except Exception as e:
679             print(f"  Error calculating itinerary for {teamNames[k]}: {e}")
680             verification_possible = False
681
682 from itertools import combinations
683 import random
684
685 #simulate match outcome
686 def match(elo1, elo2):
687     return 1 / (1 + 10 ** ((elo2 - elo1) / 400))
688
689 def random_bool(p):
690     return random.random() < p
691
692 #sumulate any order single robin
693 def games_sim(teams):
694     #name, x, y, z, elo, score
695     teams_nums = [i for i in range(len(teams))]
696
697     matches = list(combinations(teams_nums, 2))
698
699     ranking = [0 for i in range(len(teams))]
700
701     for game in matches:
702         prob1 = match(  teams[game[0]][5], teams[game[1]][5]  )
703         result = random_bool(prob1)
704         if result:
705             ranking[game[0]] += 3
```

```
        else:
            ranking[game[1]] += 3

    return ranking

#sumulate any order double robin
def games_sim_double(teams):
    #name, x, y, z, elo, score
    teams_nums = [i for i in range(len(teams))]

    matches = list(combinations(teams_nums, 2))

    ranking = [0 for i in range(len(teams))]

    for game in matches:
        prob1 = match(  teams[game[0]][5], teams[game[1]][5]  )
        result = random_bool(prob1)
        if result:
            ranking[game[0]] += 3
        else:
            ranking[game[1]] += 3

    for game in matches:
        prob1 = match(  teams[game[0]][5], teams[game[1]][5]  )
        result = random_bool(prob1)
        if result:
            ranking[game[0]] += 3
        else:
            ranking[game[1]] += 3


    return ranking

#sumulate any order double robin returning average elo
def games_sim_double_elo(teams):
    #name, x, y, z, elo, score
    elo_diff = []

    teams_nums = [i for i in range(len(teams))]

    matches = list(combinations(teams_nums, 2))

    ranking = [0 for i in range(len(teams))]

    for game in matches:
        prob1 = match(  teams[game[0]][5], teams[game[1]][5]  )
        elo_diff.append(abs(teams[game[0]][5] - teams[game[1]][5]))
        result = random_bool(prob1)
        if result:
            ranking[game[0]] += 3
        else:
            ranking[game[1]] += 3

    for game in matches:
        prob1 = match(  teams[game[0]][5], teams[game[1]][5]  )
        elo_diff.append(abs(teams[game[0]][5] - teams[game[1]][5]))
        result = random_bool(prob1)
        if result:
            ranking[game[0]] += 3
        else:
            ranking[game[1]] += 3


```

```
    return [ranking, elo_diff]

#simulation of the season with clustering
def season(game_weight, duration, do_dist):
    global teams
    ranking = [0 for i in range(len(teams))]
    distances = [0 for i in range(len(teams))]
    dist = 0 #total distance travelled

    teams =  np.column_stack((names, data, elo))

    for i in range(len(teams)):
        teams[i][4] = 0
        teams[i][0] = str(teams[i][0])


    teams = [[row[0]] + [float(x) for x in row[1:]] for row in teams]

    weights = [1, 1, 1, game_weight]

    rounds = duration

    elo_diff = []


    for i in range(rounds):
        #simulate a round (between clustering)


        new_data = np.array([row[1:-1] for row in teams])
        #print(new_data)

        clustered_teams = (perform_clustering(new_data, weights, num_clusters))
        groups = clustered_teams[0]
        grouped_indices = defaultdict(list)
        # Iterate through the list and add the index of each element to the
   corresponding group
        for index, value in enumerate(groups):
            grouped_indices[value].append(index)
        groups = list(grouped_indices.values())
        #print(groups)
        #print(groups)
        for group in groups:
            grouped_teams = [teams[k] for k in group]
            #print(grouped_teams)
            #print(teams[:4])
            output = games_sim_double_elo(grouped_teams)
            gains = output[0] ##################
            elo_diff = elo_diff + output[1]
            for j in range(len(gains)):
                ranking[group[j]] += gains[j]

            if do_dist==True:
                key_str = ",".join(map(str, sorted(group)))
                cluster_dist =  loaded_dict.get(key_str)
                dist += cluster_dist[0]
                indices = [int(x) for x in key_str.split(',')]
                for i in range(len(indices)):
                    distances[indices[i]] += cluster_dist[i+1]

        for l in range(len(teams)):
            teams[l][4] = ranking[l]

```

```
        #print(elo_diff)

    mean_elo_diff = np.mean(elo_diff)

    #print(teams)
    #print(ranking)
    #print(sum(distances))
    #print(dist)
    return [ranking, distances, float(mean_elo_diff)]

#season(50000, 6, True)

simulation for weights
interesting_weights = [i for i in range(0, 18000, 3000)]
res = []
dist = []
elo_diff =[]

for i in range(len(interesting_weights)):
    print(interesting_weights[i])
    res_local = []
    dist_local = []
    elo_local = []
    for j in range(100):
        output = season(interesting_weights[i], 6, True)
        res_local.append( output[0]  )
        dist_local.append( output[1]  )
        elo_local.append( output[2]  )

    res.append(res_local)
    dist.append(dist_local)
    elo_diff.append(elo_local)

#simulation2 for w = 500
duration = [i for i in range(1, 8)]
res2 = []
dist2 = []
elo_diff2 =[]

for i in range(len(duration)):
    print(duration[i])
    res_local = []
    dist_local = []
    elo_local = []
    for j in range(100):
        output = season(500, duration[i], True)
        res_local.append( output[0]  )
        dist_local.append( output[1]  )
        elo_local.append( output[2]  )

    res2.append(res_local)
    dist2.append(dist_local)
    elo_diff2.append(elo_local)

    #simulation3 for w = 7000
duration = [i for i in range(1, 8)]
res3 = []
dist3 = []
elo_diff3 =[]

for i in range(len(duration)):
    print(duration[i])
    res_local = []
```

```
    dist_local = []
    elo_local = []
    for j in range(100):
        output = season(7000, duration[i], True)
        res_local.append( output[0]  )
        dist_local.append( output[1]  )
        elo_local.append( output[2]  )

    res3.append(res_local)
    dist3.append(dist_local)
    elo_diff3.append(elo_local)


#simulation4 for w = 15000
duration = [i for i in range(1, 8)]
res4 = []
dist4 = []
elo_diff4 =[]

for i in range(len(duration)):
    print(duration[i])
    res_local = []
    dist_local = []
    elo_local = []
    for j in range(100):
        output = season(15000, duration[i], True)
        res_local.append( output[0]  )
        dist_local.append( output[1]  )
        elo_local.append( output[2]  )

    res4.append(res_local)
    dist4.append(dist_local)
    elo_diff4.append(elo_local)

res_avg = []

for i in range(len(res)):
    average_list = [sum(x) / len(x) for x in zip(*res[i])]
    res_avg.append(average_list)

dist_avg = []
dist_avg_avg = []

for i in range(len(dist)):
    average_list = [sum(x) / len(x) for x in zip(*dist[i])]
    dist_avg.append(average_list)
    dist_avg_avg.append(np.mean(dist_avg))

avg_elo_diff = np.mean(elo_diff)

print(dist_avg_avg)
print(avg_elo_diff)

for i, series in enumerate(res_avg):
    plt.plot(names, series, marker='o', linestyle='-', label=f"w = {i*3000}")
# Rotate x-axis labels vertically
plt.xticks(rotation=90)

# Add labels and title
plt.ylabel("Points scored in the season")
plt.ylim(top = 90 , bottom=20 )
# Show legend
plt.legend()
```

```
# Show grid
plt.grid(True, linestyle="--", alpha=0.5)
#plt.gca().set_aspect(0.2, adjustable='box')
plt.savefig('my_plot.pdf')
# Display the plot
plt.show()


plt.plot(interesting_weights, dist_avg_avg, marker='o', linestyle='-', label="
    average distance travelled by a team")
plt.ylabel("Average distance travelled by a team")
plt.xlabel("Weight w")
# Show legend
plt.legend()
# Show grid
plt.grid(True, linestyle="--", alpha=0.5)
# Display the plot
plt.show()

#to file save

data = {
    "res5": res5,
    "dist5": dist5,
    "elo_diff5": elo_diff5
}

# Save to a JSON file
with open("data5.json", "w") as f:
    json.dump(data, f)

res_avg = []

for i in range(len(res2)):
    average_list = [sum(x) / len(x) for x in zip(*res2[i])]
    res_avg.append(average_list)

dist_avg = []

for i in range(len(dist2)):
    average_list = [sum(x) / len(x) for x in zip(*dist2[i])]
    dist_avg.append(average_list)

print(dist_avg)

avg_elo_diff = np.mean(elo_diff2)

print(avg_elo_diff)

for i, series in enumerate(res_avg):
    plt.plot(names, series, marker='o', linestyle='-', label=f"{i+1} rounds")
# Rotate x-axis labels vertically
plt.xticks(rotation=90)

# Add labels and title
plt.ylabel("Points scored in the season")
plt.ylim(top = 105  )
# Show legend
plt.legend()

# Show grid
plt.grid(True, linestyle="--", alpha=0.5)

```

```
# Display the plot
plt.savefig('my_plot.pdf')
plt.show()

#Plot for different lengths of season

res_avg = []

for i in range(len(res)):
    average_list = [sum(x) / len(x) for x in zip(*res[i])]
    res_avg.append(average_list)

print(res_avg)
print(names)
print(elo)


for i, series in enumerate(res_avg):
    plt.plot(names, series, marker='o', linestyle='-', label=f"Series {i+1}")
# Rotate x-axis labels vertically
plt.xticks(rotation=90)

# Add labels and title
plt.xlabel("Names")
plt.ylabel("Points")
plt.title("Results by Name (Multiple Series)")

# Show legend
plt.legend()

# Show grid
plt.grid(True, linestyle="--", alpha=0.5)

# Display the plot
plt.show()

#simulation of double robin MONTE CARLO

def double_robin():

    teams =  np.column_stack((names, data, elo))
    teams = [[row[0]] + [float(x) for x in row[1:]] for row in teams]
    #print(teams)
    #name, x, y, z, elo, score
    teams_nums = [i for i in range(len(teams))]

    matches = list(combinations(teams_nums, 2))

    ranking = [0 for i in range(len(teams))]

    #round 1
    for game in matches:
        prob1 = match(  teams[game[0]][5], teams[game[1]][5]  )
        result = random_bool(prob1)
        if result:
            ranking[game[0]] += 3
        else:
            ranking[game[1]] += 3

    #round 2
    for game in matches:
        prob1 = match(  teams[game[0]][5], teams[game[1]][5]  )
        result = random_bool(prob1)
```

```
            if result:
                ranking[game[0]] += 3
            else:
                ranking[game[1]] += 3


    return ranking

double_robin()

simulate

res_robin = []
simulations = 10000

for j in range(simulations):
    res_robin.append(double_robin())

#print(res_robin)
res_avg = res_robin[0]

for i in range(1, len(res_robin)):
    for j in range(len(res_robin[i])):
        res_avg[j] += res_robin[i][j]

for i in range(len(res_avg)):
    res_avg[i] = res_avg[i]/simulations

print(res_avg)
#print(names)
#print(elo)

#simulate
from collections import Counter

res_robin = []
simulations = 10000

for j in range(simulations):
    res_robin.append(double_robin())

print(res_robin)
#print(res_robin)
max_indices = [sublist.index(max(sublist)) for sublist in res_robin]

print(max_indices)

winners = [0 for i in range(len(names))]
for i in range(len(max_indices)):
    winners[max_indices[i]] += 1

print(winners)
#print(names)
#print(elo)

#simulation to determine the schedule

def season_schedule(game_weight, duration, do_dist):
    global teams
    ranking = [0 for i in range(len(teams))]
    distances = [0 for i in range(len(teams))]
    dist = 0 #total distance travelled

```

```
    schedule = []

    teams =  np.column_stack((names, data, elo))

    for i in range(len(teams)):
        teams[i][4] = 0
        teams[i][0] = str(teams[i][0])


    teams = [[row[0]] + [float(x) for x in row[1:]] for row in teams]

    weights = [1, 1, 1, game_weight]

    rounds = duration

    elo_diff = []

    for i in range(rounds):

        new_data = np.array([row[1:-1] for row in teams])

        clustered_teams = (perform_clustering(new_data, weights, num_clusters))
        groups = clustered_teams[0]
        grouped_indices = defaultdict(list)
        # Iterate through the list and add the index of each element to the corresponding group
        for index, value in enumerate(groups):
            grouped_indices[value].append(index)
        groups = list(grouped_indices.values())

        schedule.append(groups)

        for group in groups:
            grouped_teams = [teams[k] for k in group]

            output_loc = []
            res_loc = []

            output = []
            for i in range(2000):
                output_loc.append(games_sim_double_elo(grouped_teams))
                res_loc.append(output_loc[i][1])
                output_loc[i] = output_loc[i][0]

            output = [[sum(col) / len(col) for col in zip(*output_loc)]]
            output.append([sum(col) / len(col) for col in zip(*res_loc)])

            gains = output[0] ##################
            elo_diff = elo_diff + output[1]
            for j in range(len(gains)):
                ranking[group[j]] += gains[j]

            if do_dist==True:
                key_str = ",".join(map(str, sorted(group)))
                cluster_dist =  loaded_dict.get(key_str)
                dist += cluster_dist[0]
                indices = [int(x) for x in key_str.split(',')]
                for i in range(len(indices)):
                    distances[indices[i]] += cluster_dist[i+1]

        for l in range(len(teams)):
            teams[l][4] = ranking[l]

```

```
    mean_elo_diff = np.mean(elo_diff)

    return [ranking, distances, float(mean_elo_diff), schedule]

#season_schedule(6000, 6, True)

obtian schedule

group_schedule = season_schedule(9000, 6, True)
#points of the teams
print(group_schedule[0])
#groups
print(group_schedule[3])
#mean elo difference
print(np.mean(group_schedule[1]))

grouped_games = group_schedule[3]

import json
import math
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from collections import Counter

# Load JSON file
#file_path = "football_clubs_with_coords.json"
file_path = "distance_compute.json"
with open(file_path, "r") as f:
    clubs = json.load(f)

# Earth's radius in kilometers
R = 6371

# Convert latitude and longitude to x, y, z
def lat_lon_to_xyz(lat, lon):
    lat_rad = math.radians(lat)
    lon_rad = math.radians(lon)

    x = R * math.cos(lat_rad) * math.cos(lon_rad)
    y = R * math.cos(lat_rad) * math.sin(lon_rad)
    z = R * math.sin(lat_rad)

    return x, y, z

# Compute coordinates for all clubs
x_vals, y_vals, z_vals, names, score, elo = [], [], [], [], [], []
for club in clubs:
    x, y, z = lat_lon_to_xyz(club["Latitude"], club["Longitude"])
    x_vals.append(x)
    y_vals.append(y)
    z_vals.append(z)
    names.append(club["Club"])
    score.append(club["ELO"])
    elo.append(club["ELO"])

single = np.array([])
multiple = np.array([])
none = np.array([])



data =  np.column_stack((x_vals, y_vals, z_vals, score))
```

```

cluster_holder = np.array([])

try:
    with open("distance_compute.json", "r") as f:
        teamsData_pre = json.load(f)
except FileNotFoundError:
    print("Error: distance_compute.json not found. Please create the input file."
    )
    exit()
except json.JSONDecodeError:
    print("Error: distance_compute.json is not valid JSON.")
    exit()

print(teamsData_pre)

def calc_dist2(n_teams, teamIndices_pre):

    teamsData = [teamsData_pre[i] for i in teamIndices_pre]

    #print(teamsData)

    # --- League Parameters derived from data ---
    teamsQuantity = n_teams
    if teamsQuantity % 2 != 0:
        raise ValueError("teamsQuantity must be even.")
    seasonWeeksQuantity = (teamsQuantity - 1) * 2 # Weeks needed for DRR

    # --- Scheduling Constraints ---
    # Set realistic streak limits (e.g., 3 or 4) - CP-SAT can handle tighter
    constraints
    maximumConsecutiveHomeGames = 20
    maximumConsecutiveAwayGames = 20
    # maximumConsecutiveHomeGames = seasonWeeksQuantity # Use high values to
    effectively disable
    # maximumConsecutiveAwayGames = seasonWeeksQuantity

    # --- Solver Parameters ---
    # CP-SAT Time Limit for finding first feasible solution
    CP_FEASIBILITY_TIME_LIMIT = 60.0 # Seconds

    # Simulated Annealing Parameters (Tune these)
    SA_INITIAL_TEMP = 10000.0 # Initial temperature - depends heavily on distance
    scale
    SA_COOLING_RATE = 0.995   # Cooling rate (closer to 1 means slower cooling)
    SA_MAX_ITERATIONS = 30000 # Increase for larger N or more thorough search

    # --- Prepare Data Structures ---
    # List of team names
    teamNames = [team['Club'] for team in teamsData]
    # List of team indices [0, 1, ..., N-1]
    teamIndices = list(range(teamsQuantity))
    # Dictionary mapping team index to its location (latitude, longitude)
    teamLocations = {i: (team['Latitude'], team['Longitude']) for i, team in
    enumerate(teamsData)}

    # Week indices
    weeks_1_based = list(range(1, seasonWeeksQuantity + 1))
    weeks_0_based = list(range(seasonWeeksQuantity))
    weeks_minus_1_1_based = list(range(1, seasonWeeksQuantity)) # For SA
    verification if needed

    # --- Distance Calculation (Haversine Formula) ---
```

```
    def calculate_haversine_distance(lat1, lon1, lat2, lon2):
        """Calculates the great-circle distance between two points on the earth.
    """
        if (lat1, lon1) == (lat2, lon2):
            return 0.0

        R = 6371.0  # Earth radius in kilometers

        lat1_rad = math.radians(lat1)
        lon1_rad = math.radians(lon1)
        lat2_rad = math.radians(lat2)
        lon2_rad = math.radians(lon2)

        dlon = lon2_rad - lon1_rad
        dlat = lat2_rad - lat1_rad

        a = math.sin(dlat / 2)**2 + math.cos(lat1_rad) * math.cos(lat2_rad) *
    math.sin(dlon / 2)**2
        c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))

        distance = R * c
        return distance

    # Pre-compute Distance Matrix
    # distanceMatrix[i][j] = distance from team i's location to team j's location
    distanceMatrix = [[0.0] * teamsQuantity for _ in range(teamsQuantity)]
    for i in teamIndices:
        for j in teamIndices:
            if i != j:
                lat1, lon1 = teamLocations[i]
                lat2, lon2 = teamLocations[j]
                distanceMatrix[i][j] = calculate_haversine_distance(lat1, lon1,
    lat2, lon2)

    #print(f"Loaded {teamsQuantity} teams. Season length: {seasonWeeksQuantity}
    weeks.")
    #print(f"Max consecutive home/away games: {maximumConsecutiveHomeGames}/{
    maximumConsecutiveAwayGames}")

    # --- 2. Helper Functions ---

    def get_team_location_for_week(schedule, team_idx, week_1_based):
        """Finds the location index where team_idx plays in a given week (1-based
    )."""
        if week_1_based not in schedule or not schedule[week_1_based]:
            return None
        for home_team_idx, away_team_idx in schedule[week_1_based]:
            if home_team_idx == team_idx: return home_team_idx # Home game,
    location is own city index
            if away_team_idx == team_idx: return home_team_idx # Away game,
    location is opponent's city index
        return None # Team not found (should not happen in a valid schedule)

    def calculate_schedule_total_travel(schedule, dist_matrix, num_teams,
    num_weeks):
        """Calculates the total travel distance for a complete schedule (1-based
    weeks)."""
        total_distance = 0.0
        indices = list(range(num_teams))

        for k in indices:
            team_travel = 0.0
            current_loc_idx = k # Start at team k's home location index
```

```
            # Travel to first game (week 1)
            first_game_loc_idx = get_team_location_for_week(schedule, k, 1)
            if first_game_loc_idx is None: return float('inf') # Incomplete/
    invalid schedule
            team_travel += dist_matrix[current_loc_idx][first_game_loc_idx]
            current_loc_idx = first_game_loc_idx

            # Travel between games (week w location to week w+1 location)
            for w in range(1, num_weeks): # w goes from 1 to M-1
                next_loc_idx = get_team_location_for_week(schedule, k, w + 1) #
    Location for week w+1
                if next_loc_idx is None: return float('inf')
                team_travel += dist_matrix[current_loc_idx][next_loc_idx]
                current_loc_idx = next_loc_idx # Update location for next segment

            # Travel back home after last game (week M)
            team_travel += dist_matrix[current_loc_idx][k] # Travel from last
    game loc back to home k
            total_distance += team_travel

        return total_distance

    def check_schedule_feasibility(schedule, num_teams, num_weeks, max_home,
    max_away, verbose=True):
        """Checks if a schedule (1-based weeks) satisfies DRR and streaks."""
        team_indices = list(range(num_teams))
        weeks = list(range(1, num_weeks + 1))
        errors = []
        # Check structure
        for w in weeks:
            if w not in schedule or len(schedule.get(w, [])) != num_teams // 2:
                errors.append(f"Feasibility Error: Week {w} incomplete/missing.")
                if verbose: print("\n".join(errors)); return False
                return False
        # Check DRR
        required = set((i, j) for i in team_indices for j in team_indices if i !=
     j)
        played = set()
        teams_per_week = {w: set() for w in weeks}
        for w in weeks:
            for h, a in schedule[w]:
                if (h,a) in played: errors.append(f"DRR Error: Match ({h},{a})
    repeated.")
                played.add((h,a))
                if h in teams_per_week[w] or a in teams_per_week[w]: errors.
    append(f"DRR Error: Team in multiple games week {w}.")
                teams_per_week[w].add(h); teams_per_week[w].add(a)
        if played != required: errors.append(f"DRR Error: Mismatch. Missing={len(
    required-played)}, Extra={len(played-required)}")
        for w in weeks:
            if len(teams_per_week[w]) != num_teams: errors.append(f"DRR Error:
    Week {w} has {len(teams_per_week[w])} teams.")
        # Check Streaks
        for k in team_indices:
            ch, ca = 0, 0
            for w in weeks:
                is_home, is_playing = False, False
                for h, a in schedule[w]:
                    if k == h: is_home, is_playing = True, True; break
                    if k == a: is_home, is_playing = False, True; break
                if not is_playing: errors.append(f"Streak Error: Team {k} not
    playing week {w}"); continue
```

```
                if is_home:
                    ch+=1; ca=0
                    if ch > max_home: errors.append(f"Streak Error: Team {k} >{max_home} home end W{w}.")
                else:
                    ca+=1; ch=0
                    if ca > max_away: errors.append(f"Streak Error: Team {k} >{max_away} away end W{w}.")
        # Result
        if errors:
            if verbose: print("Feasibility Check Failed:"); [print(f"  - {e}") for e in errors]
            return False
        return True

    # --- 3. Phase 1: Find First Feasible Schedule using CP-SAT ---

    #print("\n--- Phase 1: Finding First Feasible Schedule (CP-SAT) ---")
    start_phase1 = time.time()

    cp_model_instance = cp_model.CpModel()

    # --- CP-SAT Variables ---
    opponent_vars = {} # opponent_vars[k, w_0based] -> opponent index
    for k in teamIndices:
        for w in weeks_0_based:
            opponent_vars[k, w] = cp_model_instance.NewIntVar(0, teamsQuantity - 1, f"opponent_{k}_w{w}")
            cp_model_instance.Add(opponent_vars[k, w] != k)

    is_home_vars = {} # is_home_vars[k, w_0based] -> 0 (away) or 1 (home)
    for k in teamIndices:
        for w in weeks_0_based:
            is_home_vars[k, w] = cp_model_instance.NewBoolVar(f"is_home_{k}_w{w}")

    # --- CP-SAT Constraints ---

    # Constraint 1: Game Pairing / Symmetry
    all_teams_list = list(range(teamsQuantity))
    for w in weeks_0_based:
        for k in teamIndices:
            opp_k_w = opponent_vars[k, w]
            opponent_at_opp_k_w = cp_model_instance.NewIntVar(0, teamsQuantity - 1, f"opp_at_opp_{k}_w{w}")
            cp_model_instance.AddElement(opp_k_w, [opponent_vars[j, w] for j in all_teams_list], opponent_at_opp_k_w)
            cp_model_instance.Add(opponent_at_opp_k_w == k) # opponent[opponent[k,w]][w] == k

            is_home_opp_k_w = cp_model_instance.NewBoolVar(f"is_home_opp_{k}_w{w}")
            cp_model_instance.AddElement(opp_k_w, [is_home_vars[j,w] for j in all_teams_list], is_home_opp_k_w)
            cp_model_instance.Add(is_home_vars[k, w] != is_home_opp_k_w) # is_home[k,w] + is_home[opponent[k,w]][w] == 1

    # Constraint 2: Double Round-Robin Fulfillment
    for i in teamIndices:
        for j in teamIndices:
            if i == j: continue
            plays_home_ij_w_vars = []
            for w in weeks_0_based:
```

```
                lit_opp = cp_model_instance.NewBoolVar(f'lit_opp_{i}_{j}_w{w}')
                cp_model_instance.Add(opponent_vars[i, w] == j).OnlyEnforceIf(
    lit_opp)
                cp_model_instance.Add(opponent_vars[i, w] != j).OnlyEnforceIf(
    lit_opp.Not())
                target_and_var = cp_model_instance.NewBoolVar(f"plays_home_{i}_{j
    }_w{w}")
                cp_model_instance.AddBoolAnd([lit_opp, is_home_vars[i, w]]).
    OnlyEnforceIf(target_and_var)
                cp_model_instance.AddBoolOr([lit_opp.Not(), is_home_vars[i, w].
    Not()]).OnlyEnforceIf(target_and_var.Not())
                plays_home_ij_w_vars.append(target_and_var)
            cp_model_instance.Add(sum(plays_home_ij_w_vars) == 1) # i hosts j
    exactly once

    # Constraint 3: Home/Away Streak Limits
    for k in teamIndices:
        if maximumConsecutiveHomeGames < seasonWeeksQuantity:
            for w_start in range(seasonWeeksQuantity -
    maximumConsecutiveHomeGames):
                window = [is_home_vars[k, w] for w in range(w_start, w_start +
    maximumConsecutiveHomeGames + 1)]
                cp_model_instance.Add(sum(window) <= maximumConsecutiveHomeGames)
        if maximumConsecutiveAwayGames < seasonWeeksQuantity:
            for w_start in range(seasonWeeksQuantity -
    maximumConsecutiveAwayGames):
                window = [is_home_vars[k, w].Not() for w in range(w_start,
    w_start + maximumConsecutiveAwayGames + 1)]
                cp_model_instance.Add(sum(window) <= maximumConsecutiveAwayGames)


    # --- CP-SAT Solution Callback ---
    class FeasibleSolutionFinderCallback(cp_model.CpSolverSolutionCallback):
        def __init__(self, opponent_vars, is_home_vars, num_teams, num_weeks):
            cp_model.CpSolverSolutionCallback.__init__(self)
            self._opponent_vars = opponent_vars
            self._is_home_vars = is_home_vars
            self._num_teams = num_teams
            self._num_weeks = num_weeks
            self.solution_schedule = None
            self.solution_count = 0

        def OnSolutionCallback(self):
            self.solution_count += 1
            #print(f"Feasible solution #{self.solution_count} found by CP-SAT.
    Stopping search.")
            # Reconstruct schedule: {week_1_based: [(home_idx, away_idx), ...]}
            schedule = {w + 1: [] for w in range(self._num_weeks)}
            processed_in_week = {w: set() for w in range(self._num_weeks)}
            for w_0based in range(self._num_weeks):
                w_1based = w_0based + 1
                for k in range(self._num_teams):
                    if k not in processed_in_week[w_0based]:
                        opp_k = self.Value(self._opponent_vars[k, w_0based])
                        is_k_home = self.Value(self._is_home_vars[k, w_0based])
                        if is_k_home: h, a = k, opp_k
                        else:       h, a = opp_k, k
                        schedule[w_1based].append((h, a))
                        processed_in_week[w_0based].add(h)
                        processed_in_week[w_0based].add(a)
            self.solution_schedule = schedule
            self.StopSearch()

```

```
        def get_schedule(self):
            return self.solution_schedule

    # --- Solve with CP-SAT ---
    cp_solver = cp_model.CpSolver()
    cp_solver.parameters.max_time_in_seconds = CP_FEASIBILITY_TIME_LIMIT
    # cp_solver.parameters.log_search_progress = True # Enable for debugging

    solution_finder = FeasibleSolutionFinderCallback(opponent_vars, is_home_vars, teamsQuantity, seasonWeeksQuantity)
    status = cp_solver.Solve(cp_model_instance, solution_finder)

    end_phase1 = time.time()
    #print(f"Phase 1 Time (CP-SAT Feasibility Search): {end_phase1 - start_phase1:.2f} seconds")
    #print(f"CP-SAT Solver Status: {cp_solver.StatusName(status)}")

    feasible_schedule_phase1 = solution_finder.get_schedule()

    if feasible_schedule_phase1 is None:
        # Handle failure based on status
        if status == cp_model.INFEASIBLE: print("\nCP-SAT determined INFEASIBLE constraints.")
        elif status == cp_model.MODEL_INVALID: print("\nCP-SAT reported INVALID model.")
        else: print(f"\nCP-SAT failed to find a feasible solution (Status: {cp_solver.StatusName(status)}).")
        #print("Cannot proceed to Phase 2.")
        exit()

    #print("Verifying Phase 1 schedule feasibility...")
    if not check_schedule_feasibility(feasible_schedule_phase1, teamsQuantity, seasonWeeksQuantity, maximumConsecutiveHomeGames, maximumConsecutiveAwayGames, verbose=True):
        print("Error: CP-SAT schedule failed feasibility check!")
        exit()
    #print("Phase 1 schedule (from CP-SAT) is FEASIBLE.")

    initial_travel_cost = calculate_schedule_total_travel(feasible_schedule_phase1, distanceMatrix, teamsQuantity, seasonWeeksQuantity)
    if initial_travel_cost == float('inf'):
        #print("Error calculating initial travel cost.")
        exit()
    # Clarification: This is the cost of the *first* solution found, not a theoretical lower bound.
    #print(f"Initial Feasible Schedule Total Travel: {initial_travel_cost:,.2f} km")


    # --- 4. Phase 2: Improve Travel Distance (Simulated Annealing) ---
    #print("\n--- Phase 2: Improving Schedule (Simulated Annealing) ---")
    start_phase2 = time.time()

    # Start SA from the feasible schedule found by CP-SAT
    current_schedule = copy.deepcopy(feasible_schedule_phase1)
    current_cost = initial_travel_cost
    best_schedule = copy.deepcopy(feasible_schedule_phase1)
    best_cost = initial_travel_cost

    T = SA_INITIAL_TEMP
    alpha = SA_COOLING_RATE
    accepted_moves = 0
    infeasible_neighbors_skipped = 0
```

```
    for i in range(SA_MAX_ITERATIONS):
        neighbor_schedule = copy.deepcopy(current_schedule)
        # Choose move type (add more sophisticated moves for better results)
        move_type = random.choice(["SwapHomes", "SwapRounds"])
        move_successful = False

        if move_type == "SwapHomes" and seasonWeeksQuantity > 0:
            w = random.choice(weeks_1_based)
            if w not in neighbor_schedule: continue # Skip if week somehow missing
            home_teams_in_week = [h for h, a in neighbor_schedule.get(w, [])]
            if len(home_teams_in_week) >= 2:
                t1, t2 = random.sample(home_teams_in_week, 2)
                match1 = next(((h, a) for h, a in neighbor_schedule[w] if h == t1), None)
                match2 = next(((h, a) for h, a in neighbor_schedule[w] if h == t2), None)
                if match1 and match2:
                    opp1, opp2 = match1[1], match2[1]
                    neighbor_schedule[w].remove(match1); neighbor_schedule[w].remove(match2)
                    neighbor_schedule[w].append((t1, opp2)); neighbor_schedule[w].append((t2, opp1))
                    move_successful = True

        elif move_type == "SwapRounds" and seasonWeeksQuantity >= 2:
            w1, w2 = random.sample(weeks_1_based, 2)
            if w1 in neighbor_schedule and w2 in neighbor_schedule:
                neighbor_schedule[w1], neighbor_schedule[w2] = neighbor_schedule[w2], neighbor_schedule[w1]
                move_successful = True

        if not move_successful: continue

        # Check Feasibility (use verbose=False for speed)
        if not check_schedule_feasibility(neighbor_schedule, teamsQuantity, seasonWeeksQuantity, maximumConsecutiveHomeGames, maximumConsecutiveAwayGames, verbose=False):
            infeasible_neighbors_skipped += 1
            continue

        neighbor_cost = calculate_schedule_total_travel(neighbor_schedule, distanceMatrix, teamsQuantity, seasonWeeksQuantity)
        if neighbor_cost == float('inf'): continue # Skip if calculation failed

        delta_cost = neighbor_cost - current_cost
        # Acceptance Criteria (Metropolis criterion)
        if delta_cost < 0 or (T > 1e-9 and random.random() < math.exp(-delta_cost / T)):
            accepted_moves += 1
            current_schedule = neighbor_schedule # Keep the copy
            current_cost = neighbor_cost
            if current_cost < best_cost:
                best_schedule = copy.deepcopy(current_schedule) # Store the absolute best
                best_cost = current_cost
                # print(f"Iter {i+1}: New best cost: {best_cost:,.2f}") # Optional

        T *= alpha # Cool down

    end_phase2 = time.time()
```

```
    #print(f"Phase 2 Time: {end_phase2 - start_phase2:.2f} seconds")
    #print(f"Phase 2: Accepted {accepted_moves} moves, skipped {infeasible_neighbors_skipped} infeasible neighbors.")
    #print(f"Final Optimized Schedule Total Travel: {best_cost:,.2f} km")

    # --- 5. Display Results ---
    final_schedule_result = best_schedule
    final_travel_cost_result = best_cost

    #return final_travel_cost_result

    #print("-" * 30)
    #print("Final Optimized Schedule:")
    for w in weeks_1_based:
        #print(f"\n--- Week {w} ---")
        if w in final_schedule_result:
            for home_idx, away_idx in final_schedule_result[w]: pass #print(f"  {teamNames[home_idx]} vs {teamNames[away_idx]}")
        else: pass #print("  (Week missing from final schedule - Error!)")

    #print("\n" + "-" * 30); print("Team Itineraries & Travel Verification:")
    per_team_travel = {k: 0.0 for k in teamIndices}
    verification_possible = True
    for k in teamIndices:
        # print(f"\n{teamNames[k]} Itinerary:") # Can make output verbose if needed
        current_loc_idx = k
        try:
            loc1 = get_team_location_for_week(final_schedule_result, k, 1); assert loc1 is not None
            travel = distanceMatrix[current_loc_idx][loc1]; per_team_travel[k] += travel
            current_loc_idx = loc1
            for w in range(1, seasonWeeksQuantity):
                loc_next = get_team_location_for_week(final_schedule_result, k, w+1); assert loc_next is not None
                travel = distanceMatrix[current_loc_idx][loc_next]; per_team_travel[k] += travel
                current_loc_idx = loc_next
            travel = distanceMatrix[current_loc_idx][k]; per_team_travel[k] += travel # Return home
            # print(f"  Total Travel for {teamNames[k]}: {per_team_travel[k]:,.2f} km") # Verbose
        except Exception as e:
            #print(f"  Error calculating itinerary for {teamNames[k]}: {e}")
            verification_possible = False

    return best_schedule

    #return [final_travel_cost_result, per_team_travel]

    chedule = []

for i in range(len(grouped_games)):
    six_rounds = [[] for i in range(6)]
    for j in range(len(grouped_games[i])):

        dist = calc_dist2(4, grouped_games[i][j])

        for key in dist:
            pair_list = dist[key]
            six_rounds[key-1].append( tuple( [grouped_games[i][j][pair_list[0][0]] , grouped_games[i][j][pair_list[0][1]]]          ))
```

```
            six_rounds[key-1].append(tuple( [grouped_games[i][j][pair_list[1][0]]
    , grouped_games[i][j][pair_list[1][1]]]          ))


    for j in range(len(six_rounds)):
        schedule.append(six_rounds[j])

print(schedule)

clubs = [
    ['0', 'Real Madrid'],
    ['1', 'Bayern M nchen'],
    ['2', 'Manchester City'],
    ['3', 'Dinamo Zagreb'],
    ['4', 'Paris Saint-Germain'],
    ['5', 'Benfica'],
    ['6', 'SE Palmeiras'],
    ['7', 'RB Salzburg'],
    ['8', 'Juventus'],
    ['9', 'Al Ahly'],
    ['10', 'PSV Eindhoven'],
    ['11', 'Qarabag FK'],
    ['12', 'Shakhtar Donetsk'],
    ['13', 'TP Mazembe'],
    ['14', 'Mamelodi Sundowns'],
    ['15', 'Celtic FC'],
    ['16', 'Esperance Tunis'],
    ['17', 'LDU Quito'],
    ['18', 'Johor FC'],
    ['19', 'Ulsan Hyundai FC'],
    ['20', 'Auckland City FC'],
    ['21', 'UANL Tigres'],
    ['22', 'CD Olimpia'],
    ['23', 'Melbourne Heart FC']
]

# Convert the list to a dictionary
club_dict = {int(id_): name for id_, name in clubs}

# Sample input: Replace this with your full list of rounds
matches = schedule

# Convert numeric IDs in each match to club names
final_schedule_teams = [
    [(club_dict[home], club_dict[away]) for (home, away) in match_day]
    for match_day in matches
]

#printing the schedule
print(final_schedule_teams)

clubs_time_zones = {
    "Real Madrid": 1,
    "Bayern M nchen": 1,
    "Manchester City": 0,
    "Dinamo Zagreb": 1,
    "Paris Saint-Germain": 1,
    "Benfica": 0,
    "SE Palmeiras": -3,
    "RB Salzburg": 1,
    "Juventus": 1,
    "Al Ahly": 2,
    "PSV Eindhoven": 1,
```

```
    "Qarabag FK": 4,
    "TP Mazembe": 2,
    "LDU Quito": -5,
    "Johor FC": 8,
    "Ulsan Hyundai FC": 9,
    "Auckland City FC": 12,
    "UANL Tigres": -6,
    "CD Olimpia": -6,
    "Melbourne Heart FC": 11,
  "Shakhtar Donetsk": 2,
  "Mamelodi Sundowns": 2,
  "Celtic FC": 0,
  "Esperance Tunis": 1
}

def match_hours(final_schedule_teams, clubs_time_zones):
    next_timezone = clubs_time_zones.copy()
    team_timezone = clubs_time_zones.copy()

    def calc_next_timezone(next_timezone, curr_week, final_schedule_teams,
    clubs_time_zones):
        for matchup in final_schedule_teams[curr_week]:
            next_timezone[matchup[0]] = clubs_time_zones[matchup[0]]
            next_timezone[matchup[1]] = clubs_time_zones[matchup[0]]
        return next_timezone

    def adjust_team_timezone(next_timezone, team_timezone):
        for team in team_timezone:
            next = next_timezone[team]
            if abs(next - team_timezone[team]) > 4:
                if next - team_timezone[team] > 0:
                    team_timezone[team] += 4
                else:
                    team_timezone[team] -= 4
            else:
                team_timezone[team] = next
        return team_timezone

    from math import cos, pi

    def match_time(matchup, timezone_shift, clubs_time_zones):
        club_1 = matchup[0]
        club_2 = matchup[1]

        def calc_time(timezone_shift, club_1, club_2, clubs_time_zones):
            a = timezone_shift[club_1]
            b = timezone_shift[club_2]

            def trig(x, a):
                return -cos(2*pi/24 * (x-4+a))

            if a == b:
                x = 16 - a
                y = 1
                return (x, y)

            else:
                solutions = []
                solutions.append((4 - (a+b)/2)%24)
                solutions.append((16 - (a+b)/2)%24)
                solutions[0] = (solutions[0], trig(solutions[0], a))
                solutions[1] = (solutions[1], trig(solutions[1], a))
                if abs(a-b) == 12:
```

```
                    time_1 = (solutions[0][0] + clubs_time_zones[club_1])%24
                    time_2 = (solutions[1][0] + clubs_time_zones[club_1])%24
                    if abs(time_1 - 16) < abs(time_2 - 16):
                        return solutions[0]
                    elif abs(time_1 - 16) > abs(time_2 - 16):
                        return solutions[1]
                    elif time_1 > time_2:
                        return solutions[0]
                    else:
                        return solutions[1]


                else:
                    if solutions[0][1] > solutions[1][1]:
                        return solutions[0]
                    elif solutions[0][1] < solutions[1][1]:
                        return solutions[1]

        time, probability = calc_time(timezone_shift, club_1, club_2, clubs_time_zones)
        final_time = (time + clubs_time_zones[club_1])%24

        return (matchup, final_time, probability)


    matches_w_hours = []
    counter = 1
    for week in range(len(final_schedule_teams)):
        #print('Week ' + str(counter))
        next_timezone = calc_next_timezone(next_timezone, week, final_schedule_teams, clubs_time_zones)
        team_timezone = adjust_team_timezone(next_timezone, team_timezone)
        for matchup in final_schedule_teams[week]:
            time = match_time(matchup, team_timezone, clubs_time_zones)
            matches_w_hours.append(time)
            #print(time)
        #print('-' * 50)
        counter += 1

    return matches_w_hours

matches_w_hours = match_hours(final_schedule_teams, clubs_time_zones)
equality = {k: [] for k in clubs_time_zones.keys()}

import statistics as st

for matches in matches_w_hours:
    for team in matches[0]:
        equality[team].append(matches[2])

for team in equality:
    equality[team] = st.mean(equality[team])

print(equality)

##calculate match hourse

matches_w_hours = match_hours(final_schedule_teams, clubs_time_zones)
equality = {k: [] for k in clubs_time_zones.keys()}

import statistics as st

for matches in matches_w_hours:
```

```
    for team in matches[0]:
        equality[team].append(matches[2])

for team in equality:
    equality[team] = st.mean(equality[team])

print(equality)
```

F.2 Match Times

Week	Home Team	Away Team	Hour	CMTS
1	Benfica	Real Madrid	16.0	1.000
	Esperance Tunis	Juventus	16.0	1.000
	RB Salzburg	Bayern München	16.0	1.000
	PSV Eindhoven	Paris Saint-Germain	16.0	1.000
	CD Olimpia	Manchester City	15.0	0.966
	UANL Tigres	Celtic FC	15.0	0.966
	Dinamo Zagreb	Al Ahly	16.0	1.000
	Qarabag FK	Shakhtar Donetsk	16.0	1.000
	TP Mazembe	SE Palmeiras	16.5	0.991
	Ulsan Hyundai FC	LDU Quito	21.0	0.259
	Mamelodi Sundowns	Auckland City FC	13.0	0.707
	Johor FC	Melbourne Heart FC	16.0	1.000
2	Real Madrid	Esperance Tunis	16.0	1.000
	Benfica	Juventus	16.0	1.000
	Bayern München	Paris Saint-Germain	16.0	1.000
	RB Salzburg	PSV Eindhoven	16.0	1.000
	UANL Tigres	Manchester City	16.0	1.000
	CD Olimpia	Celtic FC	16.0	1.000
	Shakhtar Donetsk	Dinamo Zagreb	16.0	1.000
	Qarabag FK	Al Ahly	16.0	1.000
	SE Palmeiras	LDU Quito	16.0	1.000
	TP Mazembe	Ulsan Hyundai FC	14.5	0.924
	Mamelodi Sundowns	Melbourne Heart FC	15.0	0.966

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Johor FC	Auckland City FC	16.0	1.000
3	Real Madrid	Juventus	16.0	1.000
	Benfica	Esperance Tunis	16.0	1.000
	PSV Eindhoven	Bayern München	16.0	1.000
	RB Salzburg	Paris Saint-Germain	16.0	1.000
	Celtic FC	Manchester City	18.0	1.000
	CD Olimpia	UANL Tigres	16.0	1.000
	Qarabag FK	Dinamo Zagreb	16.0	1.000
	Shakhtar Donetsk	Al Ahly	16.0	1.000
	SE Palmeiras	Ulsan Hyundai FC	14.0	0.866
	TP Mazembe	LDU Quito	16.5	0.991
	Johor FC	Mamelodi Sundowns	17.0	0.966
	Melbourne Heart FC	Auckland City FC	17.5	0.924
4	Real Madrid	Benfica	16.0	1.000
	Juventus	Esperance Tunis	16.0	1.000
	Paris Saint-Germain	Bayern München	16.0	1.000
	PSV Eindhoven	RB Salzburg	16.0	1.000
	Manchester City	UANL Tigres	17.0	0.966
	Celtic FC	CD Olimpia	17.0	0.966
	Al Ahly	Dinamo Zagreb	16.0	1.000
	Shakhtar Donetsk	Qarabag FK	16.0	1.000
	SE Palmeiras	TP Mazembe	15.5	0.991
	LDU Quito	Ulsan Hyundai FC	14.0	1.000
	Auckland City FC	Mamelodi Sundowns	17.0	0.966
	Melbourne Heart FC	Johor FC	16.0	1.000
5	Esperance Tunis	Real Madrid	16.0	1.000
	Juventus	Benfica	16.0	1.000
	Bayern München	PSV Eindhoven	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Paris Saint-Germain	RB Salzburg	16.0	1.000
	Manchester City	CD Olimpia	16.0	1.000
	Celtic FC	UANL Tigres	16.0	1.000
	Dinamo Zagreb	Qarabag FK	16.0	1.000
	Al Ahly	Shakhtar Donetsk	16.0	1.000
	Ulsan Hyundai FC	SE Palmeiras	0.0	1.000
	LDU Quito	TP Mazembe	16.0	1.000
	Melbourne Heart FC	Mamelodi Sundowns	16.0	1.000
	Auckland City FC	Johor FC	16.0	1.000
6	Juventus	Real Madrid	16.0	1.000
	Esperance Tunis	Benfica	16.0	1.000
	Bayern München	RB Salzburg	16.0	1.000
	Paris Saint-Germain	PSV Eindhoven	16.0	1.000
	Manchester City	Celtic FC	16.0	1.000
	UANL Tigres	CD Olimpia	14.0	1.000
	Dinamo Zagreb	Shakhtar Donetsk	16.0	1.000
	Al Ahly	Qarabag FK	16.0	1.000
	LDU Quito	SE Palmeiras	15.0	0.966
	Ulsan Hyundai FC	TP Mazembe	23.0	0.707
	Mamelodi Sundowns	Johor FC	10.5	0.966
	Auckland City FC	Melbourne Heart FC	16.0	1.000
7	Bayern München	Real Madrid	16.0	1.000
	Mamelodi Sundowns	SE Palmeiras	16.0	0.966
	Esperance Tunis	Manchester City	16.0	1.000
	Auckland City FC	Al Ahly	19.0	0.707
	TP Mazembe	Dinamo Zagreb	16.0	1.000
	Johor FC	Celtic FC	18.0	0.866
	Paris Saint-Germain	Shakhtar Donetsk	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Benfica	Qarabag FK	16.0	1.000
	PSV Eindhoven	RB Salzburg	16.0	1.000
	LDU Quito	Juventus	15.0	0.966
	Ulsan Hyundai FC	Melbourne Heart FC	16.0	1.000
	UANL Tigres	CD Olimpia	16.0	1.000
8	Real Madrid	SE Palmeiras	16.0	1.000
	Bayern München	Mamelodi Sundowns	16.0	1.000
	Auckland City FC	Manchester City	19.5	0.609
	Esperance Tunis	Al Ahly	15.5	0.991
	Dinamo Zagreb	Celtic FC	16.0	1.000
	TP Mazembe	Johor FC	15.0	0.966
	Paris Saint-Germain	Qarabag FK	16.0	1.000
	Benfica	Shakhtar Donetsk	16.0	1.000
	LDU Quito	RB Salzburg	15.0	0.966
	PSV Eindhoven	Juventus	16.0	1.000
	Ulsan Hyundai FC	CD Olimpia	21.5	0.131
	UANL Tigres	Melbourne Heart FC	10.5	0.131
9	Real Madrid	Mamelodi Sundowns	16.0	1.000
	Bayern München	SE Palmeiras	16.0	1.000
	Al Ahly	Manchester City	16.0	1.000
	Auckland City FC	Esperance Tunis	19.5	0.609
	Dinamo Zagreb	Johor FC	16.0	1.000
	TP Mazembe	Celtic FC	16.0	1.000
	Benfica	Paris Saint-Germain	16.0	1.000
	Shakhtar Donetsk	Qarabag FK	16.0	1.000
	Juventus	RB Salzburg	16.0	1.000
	LDU Quito	PSV Eindhoven	15.0	0.966
	Ulsan Hyundai FC	UANL Tigres	21.5	0.131

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	CD Olimpia	Melbourne Heart FC	12.5	0.609
10	Real Madrid	Bayern München	16.0	1.000
	SE Palmeiras	Mamelodi Sundowns	16.0	1.000
	Manchester City	Auckland City FC	12.0	0.500
	Al Ahly	Esperance Tunis	16.0	1.000
	Dinamo Zagreb	TP Mazembe	16.0	1.000
	Celtic FC	Johor FC	16.0	1.000
	Qarabag FK	Paris Saint-Germain	16.0	1.000
	Shakhtar Donetsk	Benfica	16.0	1.000
	RB Salzburg	PSV Eindhoven	16.0	1.000
	Juventus	LDU Quito	16.0	1.000
	UANL Tigres	Ulsan Hyundai FC	10.5	0.924
	Melbourne Heart FC	CD Olimpia	23.5	0.131
11	Real Madrid	Manchester City	16.0	1.000
	Bayern München	Dinamo Zagreb	16.0	1.000
	UANL Tigres	Paris Saint-Germain	14.5	0.924
	Johor FC	Esperance Tunis	18.0	0.866
	SE Palmeiras	Benfica	16.0	1.000
	Mamelodi Sundowns	Al Ahly	16.0	1.000
	RB Salzburg	Qarabag FK	16.0	1.000
	PSV Eindhoven	Shakhtar Donetsk	16.0	1.000
	Celtic FC	Juventus	16.0	1.000
	Ulsan Hyundai FC	TP Mazembe	16.5	0.991
	CD Olimpia	LDU Quito	14.5	0.924
	Melbourne Heart FC	Auckland City FC	18.5	0.793
12	Dinamo Zagreb	Real Madrid	16.0	1.000
	Manchester City	Bayern München	16.0	1.000
	Johor FC	Paris Saint-Germain	19.5	0.609

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	UANL Tigres	Esperance Tunis	13.0	0.707
	Benfica	Mamelodi Sundowns	16.0	1.000
	SE Palmeiras	Al Ahly	15.5	0.991
	RB Salzburg	Shakhtar Donetsk	16.0	1.000
	PSV Eindhoven	Qarabag FK	16.0	1.000
	Ulsan Hyundai FC	Juventus	18.5	0.793
	Celtic FC	TP Mazembe	14.0	0.866
	Auckland City FC	LDU Quito	22.5	0.383
	Melbourne Heart FC	CD Olimpia	10.5	0.131
13	Bayern München	Real Madrid	16.0	1.000
	Paris Saint-Germain	Manchester City	16.0	1.000
	TP Mazembe	Dinamo Zagreb	16.0	1.000
	Juventus	RB Salzburg	16.0	1.000
	Benfica	LDU Quito	16.0	1.000
	SE Palmeiras	Celtic FC	16.5	0.991
	Al Ahly	Shakhtar Donetsk	16.0	1.000
	Qarabag FK	Mamelodi Sundowns	16.0	1.000
	PSV Eindhoven	Esperance Tunis	12.5	0.991
	Johor FC	Ulsan Hyundai FC	16.0	1.000
	Auckland City FC	UANL Tigres	11.0	0.259
	CD Olimpia	Melbourne Heart FC	15.5	0.991
14	Real Madrid	Manchester City	16.0	1.000
	Bayern München	PSV Eindhoven	16.0	1.000
	Dinamo Zagreb	Ulsan Hyundai FC	15.5	0.991
	Johor FC	Auckland City FC	19.5	0.609
	TP Mazembe	Paris Saint-Germain	16.0	1.000
	Celtic FC	Juventus	15.5	0.991
	SE Palmeiras	Benfica	15.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Al Ahly	RB Salzburg	16.0	1.000
	Qarabag FK	Shakhtar Donetsk	16.0	1.000
	Mamelodi Sundowns	Esperance Tunis	16.0	1.000
	CD Olimpia	LDU Quito	16.0	1.000
	Melbourne Heart FC	UANL Tigres	21.0	0.259
15	Real Madrid	PSV Eindhoven	16.0	1.000
	Bayern München	Manchester City	16.0	1.000
	Dinamo Zagreb	Auckland City FC	14.5	0.924
	Johor FC	Ulsan Hyundai FC	18.5	0.991
	Paris Saint-Germain	Juventus	16.0	1.000
	TP Mazembe	Celtic FC	16.0	1.000
	Benfica	RB Salzburg	16.0	1.000
	SE Palmeiras	Al Ahly	15.5	0.991
	Qarabag FK	Mamelodi Sundowns	16.0	1.000
	Shakhtar Donetsk	Esperance Tunis	16.0	1.000
	LDU Quito	Melbourne Heart FC	10.5	0.131
	CD Olimpia	UANL Tigres	14.5	0.924
16	Real Madrid	Bayern München	16.0	1.000
	SE Palmeiras	Mamelodi Sundowns	16.0	0.966
	Manchester City	Auckland City FC	12.0	0.500
	Al Ahly	Esperance Tunis	16.0	1.000
	Dinamo Zagreb	TP Mazembe	16.0	1.000
	Celtic FC	Johor FC	16.0	1.000
	Qarabag FK	Paris Saint-Germain	16.0	1.000
	Shakhtar Donetsk	Benfica	16.0	1.000
	RB Salzburg	PSV Eindhoven	16.0	1.000
	Juventus	LDU Quito	16.0	1.000
	UANL Tigres	Ulsan Hyundai FC	10.5	0.924

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Melbourne Heart FC	CD Olimpia	23.5	0.131
17	Real Madrid	Manchester City	16.0	1.000
	Bayern München	Dinamo Zagreb	16.0	1.000
	UANL Tigres	Paris Saint-Germain	14.5	0.924
	Johor FC	Esperance Tunis	18.0	0.866
	SE Palmeiras	Benfica	16.0	1.000
	Mamelodi Sundowns	Al Ahly	16.0	1.000
	RB Salzburg	Qarabag FK	16.0	1.000
	PSV Eindhoven	Shakhtar Donetsk	16.0	1.000
	Celtic FC	Juventus	16.0	1.000
	Ulsan Hyundai FC	TP Mazembe	16.5	0.991
	CD Olimpia	LDU Quito	14.5	0.924
	Melbourne Heart FC	Auckland City FC	18.5	0.793
18	Dinamo Zagreb	Real Madrid	16.0	1.000
	Manchester City	Bayern München	16.0	1.000
	Johor FC	Paris Saint-Germain	19.5	0.609
	UANL Tigres	Esperance Tunis	13.0	0.707
	Benfica	Mamelodi Sundowns	16.0	1.000
	SE Palmeiras	Al Ahly	15.5	0.991
	RB Salzburg	Shakhtar Donetsk	16.0	1.000
	PSV Eindhoven	Qarabag FK	16.0	1.000
	Ulsan Hyundai FC	Juventus	18.5	0.793
	Celtic FC	TP Mazembe	14.0	0.866
	Auckland City FC	LDU Quito	22.5	0.383
	Melbourne Heart FC	CD Olimpia	10.5	0.131
19	Bayern München	Real Madrid	16.0	1.000
	Mamelodi Sundowns	SE Palmeiras	16.0	0.965
	Esperance Tunis	Manchester City	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Auckland City FC	Al Ahly	19.0	0.707
	TP Mazembe	Dinamo Zagreb	16.0	1.000
	Johor FC	Celtic FC	18.0	0.866
	Paris Saint-Germain	Shakhtar Donetsk	16.0	1.000
	Benfica	Qarabag FK	16.0	1.000
	PSV Eindhoven	RB Salzburg	16.0	1.000
	LDU Quito	Juventus	15.0	0.966
	Ulsan Hyundai FC	Melbourne Heart FC	16.0	1.000
	UANL Tigres	CD Olimpia	16.0	1.000
20	Real Madrid	SE Palmeiras	16.0	1.000
	Bayern München	Mamelodi Sundowns	16.0	1.000
	Auckland City FC	Manchester City	19.5	0.609
	Esperance Tunis	Al Ahly	15.5	0.991
	Dinamo Zagreb	Celtic FC	16.0	1.000
	TP Mazembe	Johor FC	15.0	0.966
	Paris Saint-Germain	Qarabag FK	16.0	1.000
	Benfica	Shakhtar Donetsk	16.0	1.000
	LDU Quito	RB Salzburg	15.0	0.966
	PSV Eindhoven	Juventus	16.0	1.000
	Ulsan Hyundai FC	CD Olimpia	21.5	0.131
	UANL Tigres	Melbourne Heart FC	10.5	0.131
21	Real Madrid	Mamelodi Sundowns	16.0	1.000
	Bayern München	SE Palmeiras	16.0	1.000
	Al Ahly	Manchester City	16.0	1.000
	Auckland City FC	Esperance Tunis	19.5	0.609
	Dinamo Zagreb	Johor FC	16.0	1.000
	TP Mazembe	Celtic FC	16.0	1.000
	Benfica	Paris Saint-Germain	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Shakhtar Donetsk	Qarabag FK	16.0	1.000
	RB Salzburg	PSV Eindhoven	16.0	1.000
	LDU Quito	Juventus	15.0	0.966
	Ulsan Hyundai FC	Melbourne Heart FC	16.0	1.000
	UANL Tigres	CD Olimpia	16.0	1.000
22	Real Madrid	Mamelodi Sundowns	16.0	1.000
	Bayern München	SE Palmeiras	16.0	1.000
	Al Ahly	Manchester City	16.0	1.000
	Auckland City FC	Esperance Tunis	19.5	0.609
	Dinamo Zagreb	Johor FC	16.0	1.000
	TP Mazembe	Celtic FC	16.0	1.000
	Benfica	Paris Saint-Germain	16.0	1.000
	Shakhtar Donetsk	Qarabag FK	16.0	1.000
	LDU Quito	RB Salzburg	15.0	0.966
	PSV Eindhoven	Juventus	16.0	1.000
	Ulsan Hyundai FC	CD Olimpia	21.5	0.131
	UANL Tigres	Melbourne Heart FC	10.5	0.131
23	Real Madrid	Mamelodi Sundowns	16.0	1.000
	Bayern München	SE Palmeiras	16.0	1.000
	Al Ahly	Manchester City	16.0	1.000
	Auckland City FC	Esperance Tunis	14.5	0.924
	Dinamo Zagreb	Johor FC	16.0	1.000
	TP Mazembe	Celtic FC	16.0	1.000
	Benfica	Paris Saint-Germain	16.0	1.000
	Shakhtar Donetsk	Qarabag FK	16.0	1.000
	RB Salzburg	PSV Eindhoven	16.0	1.000
	LDU Quito	Juventus	15.0	0.966
	Ulsan Hyundai FC	Melbourne Heart FC	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	UANL Tigres	CD Olimpia	16.0	1.000
24	Real Madrid	SE Palmeiras	16.0	1.000
	Bayern München	Mamelodi Sundowns	16.0	1.000
	Al Ahly	Manchester City	16.0	1.000
	Auckland City FC	Esperance Tunis	19.5	0.609
	Dinamo Zagreb	Johor FC	16.0	1.000
	TP Mazembe	Celtic FC	16.0	1.000
	Benfica	Paris Saint-Germain	16.0	1.000
	Shakhtar Donetsk	Qarabag FK	16.0	1.000
	RB Salzburg	PSV Eindhoven	16.0	1.000
	LDU Quito	Juventus	15.0	0.966
	Ulsan Hyundai FC	Melbourne Heart FC	16.0	1.000
	UANL Tigres	CD Olimpia	16.0	1.000
25	Real Madrid	Mamelodi Sundowns	16.0	1.000
	Bayern München	SE Palmeiras	16.0	1.000
	Al Ahly	Manchester City	16.0	1.000
	Auckland City FC	Esperance Tunis	19.5	0.609
	Dinamo Zagreb	Johor FC	16.0	1.000
	TP Mazembe	Celtic FC	16.0	1.000
	Benfica	Paris Saint-Germain	16.0	1.000
	Shakhtar Donetsk	Qarabag FK	16.0	1.000
	LDU Quito	RB Salzburg	15.0	0.966
	PSV Eindhoven	Juventus	16.0	1.000
	Ulsan Hyundai FC	CD Olimpia	21.5	0.131
	UANL Tigres	Melbourne Heart FC	10.5	0.131
26	Real Madrid	Mamelodi Sundowns	16.0	1.000
	Bayern München	SE Palmeiras	16.0	1.000
	Al Ahly	Manchester City	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Auckland City FC	Esperance Tunis	14.5	0.924
	Dinamo Zagreb	Johor FC	16.0	1.000
	TP Mazembe	Celtic FC	16.0	1.000
	Benfica	Paris Saint-Germain	16.0	1.000
	Shakhtar Donetsk	Qarabag FK	16.0	1.000
	RB Salzburg	PSV Eindhoven	16.0	1.000
	LDU Quito	Juventus	15.0	0.966
	Ulsan Hyundai FC	Melbourne Heart FC	16.0	1.000
	UANL Tigres	CD Olimpia	16.0	1.000
27	Real Madrid	SE Palmeiras	16.0	1.000
	Bayern München	Mamelodi Sundowns	16.0	1.000
	Al Ahly	Manchester City	16.0	1.000
	Auckland City FC	Esperance Tunis	14.5	0.924
	Dinamo Zagreb	Johor FC	16.0	1.000
	TP Mazembe	Celtic FC	16.0	1.000
	Benfica	Paris Saint-Germain	16.0	1.000
	Shakhtar Donetsk	Qarabag FK	16.0	1.000
	RB Salzburg	PSV Eindhoven	16.0	1.000
	LDU Quito	Juventus	15.0	0.966
	Ulsan Hyundai FC	Melbourne Heart FC	16.0	1.000
	UANL Tigres	CD Olimpia	16.0	1.000
28	Real Madrid	Mamelodi Sundowns	15.5	0.991
	Bayern München	SE Palmeiras	16.0	1.000
	Manchester City	Al Ahly	16.0	1.000
	Esperance Tunis	Auckland City FC	14.5	0.924
	Dinamo Zagreb	Celtic FC	16.0	1.000
	TP Mazembe	Johor FC	15.0	0.966
	Paris Saint-Germain	Qarabag FK	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Benfica	Shakhtar Donetsk	16.0	1.000
	LDU Quito	RB Salzburg	15.0	0.966
	PSV Eindhoven	Juventus	16.0	1.000
	Ulsan Hyundai FC	CD Olimpia	21.5	0.131
	UANL Tigres	Melbourne Heart FC	10.5	0.131
29	Real Madrid	Mamelodi Sundowns	16.0	1.000
	Bayern München	SE Palmeiras	16.0	1.000
	Al Ahly	Manchester City	16.0	1.000
	Johor FC	Dinamo Zagreb	19.5	0.991
	Celtic FC	TP Mazembe	16.0	1.000
	Shakhtar Donetsk	Paris Saint-Germain	16.0	1.000
	Qarabag FK	Benfica	16.0	1.000
	RB Salzburg	PSV Eindhoven	16.0	1.000
	LDU Quito	Juventus	15.0	0.966
	Ulsan Hyundai FC	Melbourne Heart FC	16.0	1.000
	UANL Tigres	CD Olimpia	16.0	1.000
	Auckland City FC	Al Ahly	19.0	0.707
30	Johor FC	Dinamo Zagreb	16.0	1.000
	Ulsan Hyundai FC	Auckland City FC	16.5	0.991
	Paris Saint-Germain	Celtic FC	16.0	1.000
	Juventus	TP Mazembe	16.0	1.000
	Benfica	Al Ahly	16.0	1.000
	SE Palmeiras	RB Salzburg	16.0	1.000
	Qarabag FK	Esperance Tunis	16.0	1.000
	Shakhtar Donetsk	Mamelodi Sundowns	16.0	1.000
	LDU Quito	UANL Tigres	16.0	1.000
	Melbourne Heart FC	CD Olimpia	10.5	0.383
	Bayern München	Real Madrid	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	Paris Saint-Germain	Manchester City	16.0	1.000
31	Dinamo Zagreb	TP Mazembe	16.0	1.000
	RB Salzburg	Juventus	16.0	1.000
	LDU Quito	Benfica	16.0	1.000
	Celtic FC	SE Palmeiras	16.5	0.991
	Al Ahly	Shakhtar Donetsk	16.0	1.000
	Qarabag FK	Mamelodi Sundowns	16.0	1.000
	PSV Eindhoven	Ulsan Hyundai FC	14.5	0.924
	Esperance Tunis	Johor FC	14.5	0.924
	CD Olimpia	Auckland City FC	21.0	0.259
	Melbourne Heart FC	UANL Tigres	2.5	0.924
	Real Madrid	Paris Saint-Germain	16.0	1.000
	Bayern München	Manchester City	16.0	1.000
32	Dinamo Zagreb	Juventus	16.0	1.000
	RB Salzburg	TP Mazembe	16.0	1.000
	Benfica	LDU Quito	16.0	1.000
	SE Palmeiras	Celtic FC	16.0	1.000
	Shakhtar Donetsk	Al Ahly	16.0	1.000
	Mamelodi Sundowns	Qarabag FK	16.0	1.000
	Johor FC	PSV Eindhoven	16.0	1.000
	Ulsan Hyundai FC	Esperance Tunis	16.0	1.000
	Auckland City FC	CD Olimpia	11.0	0.259
	Melbourne Heart FC	UANL Tigres	2.5	0.793
	Real Madrid	Bayern München	16.0	1.000
	Paris Saint-Germain	Manchester City	16.0	1.000
33	Dinamo Zagreb	RB Salzburg	16.0	1.000
	Juventus	TP Mazembe	16.0	1.000
	SE Palmeiras	Benfica	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	LDU Quito	Celtic FC	15.5	0.991
	Al Ahly	Qarabag FK	16.0	1.000
	Shakhtar Donetsk	Mamelodi Sundowns	16.0	1.000
	PSV Eindhoven	Esperance Tunis	12.5	0.991
	Johor FC	Ulsan Hyundai FC	16.0	1.000
	UANL Tigres	Auckland City FC	11.0	0.707
	CD Olimpia	Melbourne Heart FC	15.5	0.991
	Real Madrid	Dinamo Zagreb	16.0	1.000
	Bayern München	Manchester City	16.0	1.000
34	Paris Saint-Germain	Johor FC	14.5	0.924
	UANL Tigres	Esperance Tunis	16.0	1.000
	SE Palmeiras	Benfica	16.0	1.000
	Mamelodi Sundowns	Al Ahly	16.5	0.991
	Shakhtar Donetsk	RB Salzburg	16.0	1.000
	PSV Eindhoven	Qarabag FK	16.0	1.000
	Ulsan Hyundai FC	Juventus	16.0	1.000
	TP Mazembe	Celtic FC	16.0	1.000
	LDU Quito	Auckland City FC	9.5	0.991
	CD Olimpia	Melbourne Heart FC	21.5	0.131
	Real Madrid	Bayern München	16.0	1.000
	Manchester City	Dinamo Zagreb	16.0	1.000
35	Paris Saint-Germain	UANL Tigres	16.0	1.000
	Esperance Tunis	Johor FC	16.0	1.000
	Mamelodi Sundowns	Benfica	16.0	1.000
	Al Ahly	SE Palmeiras	16.0	1.000
	Qarabag FK	RB Salzburg	16.0	1.000
	Shakhtar Donetsk	PSV Eindhoven	16.0	1.000
	Celtic FC	Juventus	16.0	1.000

Continued on next page

Week	Home Team	Away Team	Hour	CMTS
	TP Mazembe	Ulsan Hyundai FC	16.0	1.000
	LDU Quito	Melbourne Heart FC	11.0	0.707
	Auckland City FC	CD Olimpia	2.0	0.500
	Real Madrid	Manchester City	16.0	1.000
	Bayern München	Dinamo Zagreb	16.0	1.000
36	Paris Saint-Germain	Al Ahly	16.0	1.000
	RB Salzburg	Qarabag FK	16.0	1.000
	TP Mazembe	Benfica	16.0	1.000
	Juventus	SE Palmeiras	16.0	1.000
	Mamelodi Sundowns	PSV Eindhoven	16.0	1.000
	LDU Quito	Shakhtar Donetsk	13.5	0.991
	Celtic FC	Esperance Tunis	16.0	1.000
	Melbourne Heart FC	Auckland City FC	16.5	0.991
	UANL Tigres	Johor FC	12.5	0.609
	Ulsan Hyundai FC	CD Olimpia	12.5	0.609
	Dinamo Zagreb	Real Madrid	16.0	1.000
	Bayern München	Manchester City	16.0	1.000

F.3 Match Times Equality Scores

Team Name	Equality Score
Real Madrid	0.999
Bayern München	0.999
Manchester City	0.973
Dinamo Zagreb	0.985
Paris Saint-Germain	0.985
Benfica	0.999
SE Palmeiras	0.992
RB Salzburg	0.998
Juventus	0.993
Al Ahly	0.991
PSV Eindhoven	0.988
Qarabag FK	1.000
TP Mazembe	0.979
LDU Quito	0.880
Johor FC	0.900
Ulsan Hyundai FC	0.824
Auckland City FC	0.734
UANL Tigres	0.815
CD Olimpia	0.700
Melbourne Heart FC	0.679
Shakhtar Donetsk	0.999
Mamelodi Sundowns	0.985
Celtic FC	0.955
Esperance Tunis	0.947